

INDIAN INSTITUTE OF TECHNOLOGY DELHI

Department of Electrical Engineering

ELL305: Computer Architecture — Term Paper

Modern Processor Design Beyond Monolithic Scaling

Chiplet Architectures, Cache Hierarchies, and 3D-Stacked Cache — A Case Study of AMD Zen Processors

Second Full-Paper Working Draft (initial-draft-2)

Submitted by

Ishan Krishna Agrawal	2024MT60860	mt6240860@maths.iitd.ac.in
Vaidik Goyal	2024EE11030	ee1241030@ee.iitd.ac.in
Moon Saraswat	2024EE11163	ee1241163@ee.iitd.ac.in

30 October 2026

Copyright and Intellectual Property Rights Assignment

All intellectual property rights in the original content of this report belong to the Indian Institute of Technology Delhi, including its Department of Electrical Engineering.

Intellectual property rights in material cited or otherwise attributed to external sources remain with the respective authors, publishers, or other rights holders.

The Authors
30 October 2026

Acknowledgements

The authors acknowledge Prof. Sumantra Dutta Roy as the faculty advisor for this term-paper project.

The authors also acknowledge the Department of Electrical Engineering and the Indian Institute of Technology Delhi for the academic environment, facilities, and institutional resources supporting this work.

This paper is prepared collectively by Ishan Krishna Agrawal, Vaidik Goyal, and Moon Saraswat.

Abstract

For several decades, processor performance improved largely by integrating more transistors on a single, monolithic die. That approach is now constrained by die size limits, manufacturing yield, the uneven scaling of logic, SRAM and analog circuits, and the power and thermal budget of the package. This paper studies two linked architectural responses to these constraints: chiplet-based processor organisation, which partitions a processor into separately manufactured dies, and three-dimensional stacking of cache, which adds last-level-cache capacity without enlarging the compute die. AMD Zen processors serve as the principal case study, tracing the progression from the compute-die and I/O-die partitioning of Zen 2, through the unified eight-core L3 organisation of Zen 3, to the hybrid-bonded 3D V-Cache products. For each technique the paper asks which bottleneck it addresses, what benefit it provides, and what new cost or trade-off it introduces. The analysis is grounded in the memory-hierarchy and multiprocessor foundations of the course textbook and evaluates published research and benchmark evidence rather than new hardware measurements. **[TODO:** Revise the abstract once Parts V and VI contain completed analysis.]

Keywords: processor architecture, chiplets, memory hierarchy, last-level cache, 3D-stacked cache, hybrid bonding, AMD Zen, ELL305.

Contents

List of Tables **9**

List of Figures **11**

Glossary **14**

I Introduction and Research Framing 16

1 Introduction 16

1.1 Motivation: why monolithic scaling is becoming difficult 16
1.2 A thought experiment: one die or eight? 17
1.3 Two linked scaling problems 17
1.4 Research questions and objectives 19
1.5 Scope 20
1.6 Methodology 20
1.7 Contributions 21
1.8 Organisation of the paper 21

II Architectural Foundations 22

2 Processor Organisation and Performance Metrics 22

2.1 The performance equation 22
2.2 Baseline processor organisation 22

3 Pipelining, Power and Thermal Limits 23

3.1 Pipelining and its limits 23
3.2 Dynamic and leakage power 24
3.3 Temperature 24
3.4 Combining energy and delay: the ED^2 metric 25

4 The Memory Hierarchy 25

4.1 Why a hierarchy is needed 25

4.2 SRAM and DRAM arrays . . . 26
4.3 Average memory access time . . . 26
4.4 Classifying cache misses . . . 28
4.5 Cache organisation and policies . . . 28
4.6 Beyond SRAM: alternative LLC technologies . . . 30

5 Multicore Processors, Coherence and Interconnects 30
5.1 From one core to many . . . 31
5.2 How far parallelism goes: Amdahl's law . . . 31
5.3 Cache coherence . . . 31
5.4 On-chip and on-package interconnection networks . . . 34

III Chiplet-Based Processor Architecture 36

6 Why Large Monolithic Dies Become Costly 36
6.1 The reticle limit and die size . . . 36
6.2 Manufacturing yield . . . 36
6.3 Uneven scaling of logic, SRAM and analog circuits . . . 37

7 Compute Dies, I/O Dies and Core Complexes 38
7.1 Inside the I/O die: memory, I/O and NUMA settings . . . 39

8 Infinity Fabric and Die-to-Die Communication 40
8.1 Latency, bandwidth and interconnect power . . . 41
8.2 Scalability . . . 42
8.3 How the dies are connected: organic substrate, 2.5D and 3D . . . 43

9 Yield, Process-Node Specialisation and Cost 44
9.1 Worked example: good dies per wafer . . . 45

10 Summary of Part III 46

IV 3D-Stacked Cache and AMD 3D V-Cache 47

11 Why Cache Capacity Matters **47**

12 Why Expanding Cache in Two Dimensions Became Difficult **48**

12.1 Area and yield 48

12.2 Wire length and access latency 48

12.3 Power 49

12.4 Summary of the planar limits 49

13 Side-by-Side Versus Vertically Stacked Cache Dies **49**

13.1 Placing the cache die beside the compute die 49

13.2 Stacking the cache die on the compute die 50

14 Stacked-Die Integration and Hybrid Bonding **51**

14.1 The first-generation 3D V-Cache stack 51

14.2 Hybrid bonding versus micro-bumps 52

14.3 Worked example: what a finer pitch buys 53

14.4 How a hybrid bond is made 53

15 Cache Access, Latency, Bandwidth and Hierarchy **55**

15.1 Worked example: applying the trade-off to a measured workload 55

15.2 Inside the cache interface 56

16 Thermal Management, Power Delivery and Manufacturing Complexity **57**

16.1 Power delivery through the stack 58

16.2 Evidence of the thermal cost and AMD's response 59

16.3 Manufacturing: thinning, assembly, yield and test 60

17 Workloads That Benefit From a Larger LLC **61**

18 Summary of Part IV **63**

V AMD Case Study and Quantitative Analysis **64**

19 From Zen 2 Chiplets to Zen 3 and 3D V-Cache **64**

19.1 Zen 2 to Zen 3: a same-core-count comparison 64

20 Comparing Architectural Configurations **66**

21 Method for Analysing Published Evidence **66**

22 Published Benchmarks and Performance Measurements **67**

22.1 Latency through the whole hierarchy 69

22.2 A working-set sweep: MiniFE 69

22.3 OpenFOAM in detail 70

22.4 Breadth: 350 desktop results 71

23 Cache-Sensitive Versus Cache-Insensitive Workloads **71**

24 Separating Cache Effects From Other Factors **72**

24.1 Worked example: correcting for clock speed 73

25 Summary of Part V **74**

VI Comparative Synthesis, Limitations and Future Directions **75**

26 Monolithic, Chiplet-Based and Vertically Stacked Designs Compared **75**

27 Other Ways to Enlarge the Last-Level Cache **75**

28 Interactions Between Chiplet Organisation and Cache Design **77**

29 Workload-Dependent Trade-Offs and Remaining Bottlenecks **78**

30 Future Directions in Heterogeneous Integration and Advanced Packaging **80**

31 Limitations of This Study **80**

32 Conclusions **81**

33 Open Research Questions **82**

References **83**

A Plagiarism Report **86**

B Reading Indices **87**

C Yield and Cost-per-Good-Die Model **88**

C.1 From Poisson to Murphy yield 88

C.2 Cost per good die 88

C.3 Splitting a design into chiplets 89

Index **90**

List of Tables

1 One die or eight: the 64-core thought experiment, answered from later sections (model results are illustrative) . . . 18

2 Where the research questions are answered . . . 20

3 Worked example: AMAT from measured latencies [12] and assumed miss rates ($m_1 = 5\%$, $m_2 = 30\%$, $m_3 = 40\%$) . . . 27

4 Worked example: address fields for the Zen 3 caches (64-byte lines; 48-bit physical address, as documented for EPYC 7003 with memory encryption off) . 29

5 Selected memory-system optimisations and their costs, after Sarangi [1, Table 11.2] 30

6 Private versus shared caches, after Sarangi [1, Table 12.3] . . . 32

7 Worked example: block states and bus traffic for two cores sharing one block (write-invalidate MSI) . . . 33

8 Diameter and bisection bandwidth of common topologies with N nodes, from Sarangi [1, sec. 12.6.3] . . . 34

9 Worked example: diameter and bisection bandwidth from the formulas of table 8 35

10 Illustrative Murphy-model yield for $D_0 = 0.2$ defects/cm^2 and $Y_{\text{wafer}} = 1$ . . . 37

11 NUMA settings of an EPYC 7003 processor with eight CCDs and eight memory channels, from AMD's description [11], with peak interleaved bandwidth worked out for DDR4-3200 (25.6 GB/s per channel) . . . 40

12 Memory latency by NUMA distance, Naples (EPYC 7001) and Rome (EPYC 7002), as reported by AMD [2, Fig. 2.2.6] . . . 41

13 Worked example: total CCD–IOD link bandwidth against peak memory bandwidth (55 GB/s per link [2]; 8 channels of DDR4-3200) . . . 43

14 Die-to-die connection methods, with AMD's interconnect pitch targets [22, slide 13] and AMD examples . . . 44

15 Worked example: dies and good dies per 300 mm wafer ($D_0 = 0.2\,\text{cm}^{-2}$, $Y_{\text{wafer}} = 1$; illustrative) . . . 45

16 Qualitative comparison of side-by-side and vertically stacked cache dies . . . 51

17 Worked example: silicon consumed per good part for a planar 96 MB CCD and the stacked design (Murphy model, $D_0 = 0.2\,\text{cm}^{-2}$; illustrative) . . . 61

18 L3 organisation per compute die across the generations studied (AMD-published figures) . . . 65

19 3D V-Cache across Zen generations (AMD-published figures unless noted) . . . 66

20 Processor pairs that differ mainly in L3 capacity . . . 67

21 AMD-reported performance figures relevant to this study (vendor claims, not independent measurements) . . . 68

22 Independent measurements of the effect of 3D V-Cache 68

23 Which confounding factors each available comparison controls (✓ = controlled, × = not controlled, ∼ = estimated and normalised, ? = not stated; AMD-reported rows above the line, independent below) 73

24 Worked example: speedups bounded by a clock correction (corrected = measured ÷ clock ratio) 74

25 Comparison of the three design approaches (model values illustrative; measured values from the cited sections) 76

26 Ways to enlarge the last-level cache, with the figures the sources report (different studies, nodes and conditions; compare orders of magnitude, not exact values) . 77

27 Worked example: IPC gain of the V-Cache CCD at equal clock [18] against the plain CCD's 7% clock advantage (net = $(1 + \text{gain})/1.07 - 1$) 79

List of Figures

1 The steps of AMD's Zen design studied in this paper, with the year each was disclosed or launched [2], [3], [4], [5], [6], [7], [8], [9], [10]. 17

2 Research framework: two linked scaling problems, the AMD responses studied in this paper, and the evaluation question applied to each. 19

3 The heat path from a processor die to the air, redrawn after Sarangi [1, sec. 10.10.1, Fig. 10.36]. Each layer adds thermal resistance; stacking a second die on the processor adds another layer between the transistors and the spreader (section 16). 25

4 A multi-level cache hierarchy with the quantities used in the recursive AMAT expression (equation (6)). Capacity grows and speed falls moving down the hierarchy. 26

5 (a) A six-transistor SRAM cell: the word line (WL) turns on the access transistors W1 and W2, connecting the cross-coupled inverters to the complementary bit lines. (b) Cells are arranged in a two-dimensional array; a decoder raises one word line, every cell in that row drives its bit lines, and a column multiplexer and sense amplifiers select and read the bits required. Redrawn after Sarangi [1, sec. 7.4.1, Figs. 7.26–7.27]. 27

6 Lookup in a 4-way set-associative cache. The index bits select one set; the tags stored in all four ways of that set are compared with the address tag in parallel; a match identifies the way holding the block, and the offset selects the bytes within it. After Sarangi [1, sec. 11.2.2]. 29

7 Amdahl's law, equation (7), for three sequential fractions, after Sarangi [1, sec. 12.2.3, Fig. 12.4]. The marked points are the worked example for $f_{\text{seq}} = 5\%$. 31

8 The write-invalidate (MSI) snooping protocol for one cache block, redrawn after Sarangi [1, sec. 12.4.6, Figs. 12.13–12.14]. (a) Transitions caused by this cache's own processor; (b) transitions caused by messages that other caches place on the bus. A block may be held by many caches in S, but by only one in M. 33

9 Three topologies. The ring and mesh follow Sarangi's definitions [1, sec. 12.6.3]; the star is the arrangement of Rome's eight CCDs around one I/O die (section 7). Within a CCD, Zen 3 and Zen 4 connect cores to L3 slices by a ring [18], [19]. 35

10 Murphy yield model (equation (8), $Y_{\text{wafer}} = 1$) for three illustrative defect densities. The marked points reproduce the worked example in section 6.2. Illustrative calculation, not measured processor data. 37

11 Schematic contrast between a monolithic processor die and a chiplet package with compute dies (CCDs) and an I/O die (IOD). Illustrative only; not to scale and not an actual AMD floorplan. 38

12 Schematic of an AMD EPYC 7002/7003-style package: eight CCDs in two-deep columns around a central I/O die, each with its own link to the I/O die [2], [11]. Positions, link routes and memory-controller placement are illustrative, not an actual AMD floorplan. 39

13 AMD's packaging steps as presented by AMD [22, slide 5]. The slide gives no separate year for the 2.5D bridge (EFB). The CPUs studied in this paper use the 2019 chiplet organisation and, from 2021, 3D-stacked cache. 44

14 Cost against core count for Rome-generation chiplet products and hypothetical monolithic 7 nm equivalents, normalised to the 64-core chiplet part. Values were read by eye from AMD's chart [2, Fig. 2.2.2] and are accurate only to about ±0.05. AMD shows no monolithic 64-core bar; the text states that such a die would exceed the maximum reticle size. 46

15 Conceptual shape of the L3 miss rate against working-set size for a workload with reuse, for a 32 MB and a 96 MB L3. The curves are illustrative, not measured; real transitions are smoother and depend on access pattern, associativity and sharing. They show the three workload classes of section 17. 48

16 Measured extra latency, in cycles beyond the 40-cycle minimum, for a Zen 3 core to reach each of the eight slices of its L3, on a Ryzen 5 5600G [13]. The arrangement of slices follows the source's own table; it is a logical layout, not a floorplan. 49

17 Two ways to add cache capacity to a compute die. Illustrative arrangements only; not actual AMD package layouts and not to scale. 50

18 Schematic cross-section of the first-generation 3D V-Cache stack, drawn from AMD's descriptions [6, slides 10–12, 14, 16–17] and [11]. Not to scale; layer thicknesses and the number of TSVs are illustrative. 52

19 Worked example: connections per square millimetre for a square grid of pads at each pitch, $(1000/p)^2$ for a pitch p in micrometres. Pitches are AMD's stated limits for C4 bumps and 2.5D [22, slide 13], the finest micro-bump pitch AMD quotes, and the 9 µm hybrid-bond pitch [5]. Real layouts are not full square grids, so the values are upper bounds for comparison only. 54

20 Schematic of one hybrid-bond connection in the first-generation stack, redrawn from AMD's description [6, slide 16]. Copper bond-pad metal (BPM) on the two dies is bonded directly; on the top die a bond-pad via (BPV) connects the pad to the top copper layer (M13), and on the bottom die the pad lands on a TSV. In the Zen 4 stack the BPV lands on the top aluminium pad instead [8, slides 11–12]. Not to scale. 54

21 Top view of how the L3D connects to the CCD, redrawn from AMD's descriptions [6, slides 10–11, 19–21, 25]. Two columns of signal TSVs run along the CCD's central L3, and matching interface columns on the L3D lie directly above them. Each core reaches the L3 over a 32 B/cycle interface. Positions and sizes are schematic; the L3D outline is drawn only approximately over the L3 region. . . 58

22 The two stack arrangements of 3D V-Cache, drawn from AMD's comparison [22, slide 7] and descriptions [6], [8], [23]. In (a) the cache die sits over the CCD's central L3 and the cores lie under structural silicon dies; in (b) the cache die, now the full width of the CCD, carries the through-silicon vias and the cores sit directly beneath a single structural layer. Schematic; not to scale. 59

23 IPC gain from 3D V-Cache with clock, core design and memory held equal: the two CCDs of one Ryzen 9 7950X3D, both fixed at 4.2 GHz, measured with performance counters by Lam [18]. Cyberpunk 2077 is computed from the reported IPC of 1.26 and 1.43; the Call of Duty value is reported only as "over 19%". A sixth workload, a flight simulator with very few L3 misses, lost a small, unquantified amount of IPC and is not plotted. 62

24 Schematic progression of L3 organisation per compute die from Zen 2 to Zen 3 and Zen 3 with 3D V-Cache, based on AMD's descriptions [3], [6], [11]. Logical organisation only; not a floorplan. 64

25 Planned evidence-analysis procedure for Part V (published research and benchmarks only; no new hardware measurement). 67

26 Cache and memory latency against data size for Milan and Milan-X, redrawn from Chips and Cheese's measurements [32]. The labelled values (L1 4 cycles, L2 about 12 cycles, L3 47.49 and 51.14 cycles, memory 342 and 313 cycles) are taken from the chart's own labels; the transitions between them were read by eye and are approximate. In the shaded range, from 32 MB to about 96 MB, Milan-X still hits in its L3 while Milan already goes to memory. 70

27 MiniFE on a dual-socket EPYC 7773X (Milan-X, 2×768 MiB L3) against a dual-socket EPYC 7763 (Milan, 2×256 MiB L3) as the problem size grows. Values were read by eye from Domke *et al.*'s figure [26, Fig. 1] and are accurate to about ±0.05. Milan-X runs at a lower base clock (2.20 against 2.45 GHz). . . 71

28 Distribution of the 350 statistically significant differences among Phoronix's 420 benchmarks of the Ryzen 7 5800X3D and 5800X [28]. The winner of each test was identified from the colour of its bar in Phoronix's comparison chart and matched to the chart's printed percentages; this classification is ours. Each bin includes its lower bound (for example, 5–10% means at least 5% and less than 10%). 72

29 When does a stacked L3 pay off? A decision sequence summarising the evidence of Parts IV and V; the examples are measured cases from [18], [26], [27]. Cost, which also favours the plain die, is not shown. 79

Glossary

3D V-Cache AMD's name for a design that uses three-dimensional die integration to add stacked cache capacity above a processor die.

AMAT (average memory access time) A weighted average of memory-access latencies that accounts for cache hits, cache misses, and the penalties of accessing lower levels of the memory hierarchy.

cache coherence The mechanisms and rules used to maintain a consistent view of shared data when multiple caches can hold copies of the same memory location.

cache hierarchy A set of cache levels organised to exploit locality by balancing capacity, access latency, and implementation cost.

CCD (Core Complex Die) AMD terminology for a compute die containing processor cores and associated cache resources in selected Zen-based processors.

CCX (Core Complex) AMD terminology for a group of cores that share one L3 cache; in Zen 2 a CCD contains two four-core CCXs, and in Zen 3 a CCD contains one eight-core CCX.

chiplet A separately fabricated die designed to operate with one or more other dies as part of a packaged system.

CPI (cycles per instruction) The average number of clock cycles taken per dynamic instruction; its reciprocal is IPC.

defect density The average number of yield-killing manufacturing defects per unit area of silicon, written D_0 in simple yield models.

hybrid bonding A fine-pitch die-stacking technique that forms direct mechanical and electrical connections between aligned die surfaces.

IFOP (Infinity Fabric On-Package) The on-package serial link used by AMD to connect compute dies to the I/O die over short organic-package routes.

Infinity Fabric AMD's interconnect architecture for communication among processor components and, in relevant products, among separate dies.

IOD (I/O Die) AMD terminology for a die that centralises input/output, memory-interface, and supporting interconnect functions in selected chiplet-based Zen processors.

IPC (instructions per cycle) The average number of dynamic instructions completed per clock cycle; the reciprocal of CPI.

LLC (last-level cache) The final cache level checked before a request proceeds to main memory in the cache hierarchy under discussion.

memory hierarchy An organisation of storage levels with different capacities, costs, and access times, arranged to exploit temporal and spatial locality.

micro-bump A small solder-based bump used to connect stacked or adjacent dies; its pitch limits the achievable interconnect density compared with hybrid bonding.

monolithic die A single semiconductor die on which the principal processor resources under discussion are integrated rather than partitioned across multiple dies.

reticle limit The maximum die area that a lithography system can pattern in a single exposure field, which bounds the size of a monolithic die.

SRAM (static random-access memory) A memory built from bistable cells that retain data while powered without refresh; the usual technology for on-chip caches.

TSV (through-silicon via) A vertical electrical connection passing through a silicon die to support communication in a stacked-die assembly.

yield The fraction of manufactured dies that function correctly; larger dies generally yield worse for a given defect density.

Part I

Introduction and Research Framing

Target length: about 10 pages. Draft status: opening argument, research questions, scope and method drafted; contributions finalised.

1 Introduction

1.1 Motivation: why monolithic scaling is becoming difficult

For most of the history of the microprocessor, a new product generation could be built by placing more transistors on a single piece of silicon. The course textbook summarises this trend as Moore's law: the number of transistors on a chip has roughly doubled every one to two years since the 1960s [1, sec. 12.1.1].

How processors used to scale

The textbook describes three phases in how designers spent those transistors [1, sec. 12.1.2]. First, a smaller transistor was faster and more power-efficient, so each generation built a bigger processor: more complex units, larger caches, wider issue and more functional units. Second, until about 2002, pipelines also grew deeper and clock frequencies rose with them. Third, after about 2002, power and temperature became the limit: processors began to exceed 100 W and on-chip temperatures 100 °C, which effectively ended the growth in core complexity and clock frequency. Designers instead began to place more cores of an unchanged design on each chip, so that the number of cores could double with each process generation [1, sec. 12.1.2].

Over the same period the smallest feature that could be manufactured shrank by a factor of about $\sqrt{2}$ every two years, from 130 nm in 2001 to 22 nm in 2011, doubling the number of transistors per unit area each time [1, sec. 12.1.1, Table 12.1]. Writing around 2012, the textbook expected processors with 32 or 64 cores within five to ten years [1, sec. 12.1.2]. The prediction came true almost exactly: AMD's second-generation EPYC offered 64 cores in 2019. But it did not arrive in the way the multicore era had assumed. Those 64 cores were not on one larger die but on eight small ones, joined in one package to a separate I/O die [2]. Explaining why is the starting point of this paper.

Three pressures now make it harder to keep growing a single monolithic die:

- **Die size and yield.** A lithography system can only pattern a die up to the reticle limit, and the probability that a die contains a fatal defect rises with its area. AMD states that

the 64-core server configuration of its second-generation EPYC processors would lie well beyond the maximum reticle size if built as one chip [2]. Section 6.2 quantifies the yield effect with a simple model.

- **Uneven scaling of circuit types.** Logic, SRAM (static random-access memory) and analog/I-O circuits do not shrink equally when moving to a new process node. Naffziger *et al.* note that analog circuitry shrinks far less than digital logic and SRAM, so memory interfaces and SerDes gain little from an advanced node [2]. More recent AMD material indicates that SRAM itself now scales more slowly than logic (section 11).
- **Power and temperature.** Dynamic power grows with switched capacitance, the square of the supply voltage and clock frequency, while leakage power is consumed even when circuits are idle and itself rises with temperature [1, sec. 10.10]. Every additional square millimetre of active silicon must be powered and cooled within a fixed package budget.

These pressures do not stop progress, but they change its form (figure 1). Instead of one ever-larger die, designers increasingly assemble a processor from several dies inside one package, and increasingly use the vertical dimension as well as the horizontal one. This paper studies that shift through two concrete techniques and one principal case study.

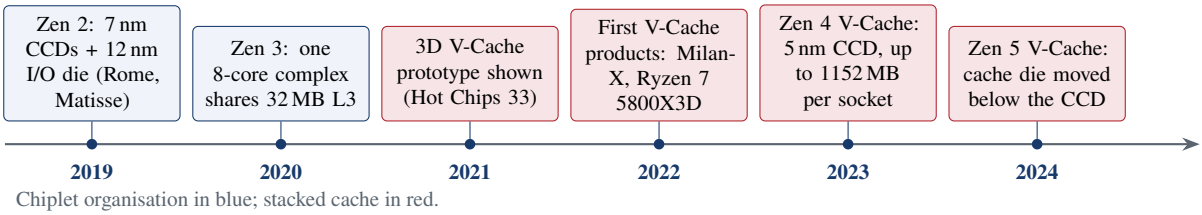


Figure 1: The steps of AMD's Zen design studied in this paper, with the year each was disclosed or launched [2], [3], [4], [5], [6], [7], [8], [9], [10].

1.2 A thought experiment: one die or eight?

Consider two ways to build a 64-core processor. Processor A places all 64 cores on one large die. Processor B places them on eight smaller dies of eight cores each, connected inside one package to a separate I/O die. Both offer 64 cores. Are they equally expensive to make? Do they communicate equally quickly? Can they use the same manufacturing process?

Using AMD's Zen 2 dimensions, with the compute dies at 74 mm^2 each and a monolithic equivalent of $8 \times 74 = 592$ mm^2 for the compute area alone, the later parts of this paper answer these questions (table 1).

The answers are not all in favour of B. It is far cheaper and the only option that can be built at all, but it pays for that with a die-to-die hop on every memory access and with caches that are private to each die. The rest of the paper examines both sides of this trade, and how 3D stacking changes the cache side of it.

1.3 Two linked scaling problems

The paper is organised around two related problems (figure 2).

Table 1: One die or eight: the 64-core thought experiment, answered from later sections (model results are illustrative)

Question	A: one 64-core die	B: eight 8-core dies + I/O die	Where
Can it be built?	For 64 cores with I/O, beyond the maximum reticle size [2]	Yes; each die is small	section 6
Modelled yield ($D_0 = 0.2\,\mathrm{cm}^{-2}$)	38%	87% per compute die	section 6.2
64-core sets per 300 mm wafer	about 35	about 95	section 9.1
AMD's cost estimate	roughly 2× at 32 and 48 cores	1× (reference)	figure 14
Manufacturing process	everything on the newest node	cores on 7 nm, I/O on cheaper 12 nm	section 7
Communication	on-die network only	every memory access crosses a die-to-die link (under 9 ns round trip)	section 8
Memory latency	not available for comparison	Rome's average about 0.81× that of its multi-die predecessor	table 12
Cache per core	one shared design choice	each die has its own L3; can be enlarged per die by stacking	Part IV

Problem 1: scaling processor resources beyond a single die. If a product needs more cores, memory channels and I/O than one economical die can hold, the processor can be partitioned into chiplets. AMD's Zen 2-based products separate eight-core compute dies (CCDs) built on a 7 nm process from an I/O die (IOD) built on a mature 12 nm process [2], [3].

Problem 2: scaling cache capacity without spending planar die area. A larger LLC (last-level cache) can reduce costly trips to main memory, but on a planar die every extra megabyte of SRAM enlarges the die, lowers yield and lengthens wires. AMD's 3D V-Cache instead stacks an additional SRAM die on top of the compute die, expanding the Zen 3 L3 from 32 MB to 96 MB per CCD [6], [11].

The problems are linked: the chiplet organisation makes the CCD a small, self-contained compute die with its own L3, and that die is the natural host for a stacked cache. The paper therefore follows the argument in four steps:

1. Monolithic CPUs face die-size, yield and scaling limits; chiplets address compute scaling [2].
2. Chiplets alone leave the cost of large on-die caches unsolved: AMD's ISSCC 2022 presentation names SRAM scaling and cost as the barriers to large on-die caches, in cost, product flexibility and latency, and argues that 2.5D chiplets address only part of the problem [6, slides 6–7].
3. Vertical stacking addresses those cache-specific limits [5], [6].
4. Evaluation evidence shows where the stacked cache improves performance, and which thermal, packaging and cost trade-offs it introduces (Part V).

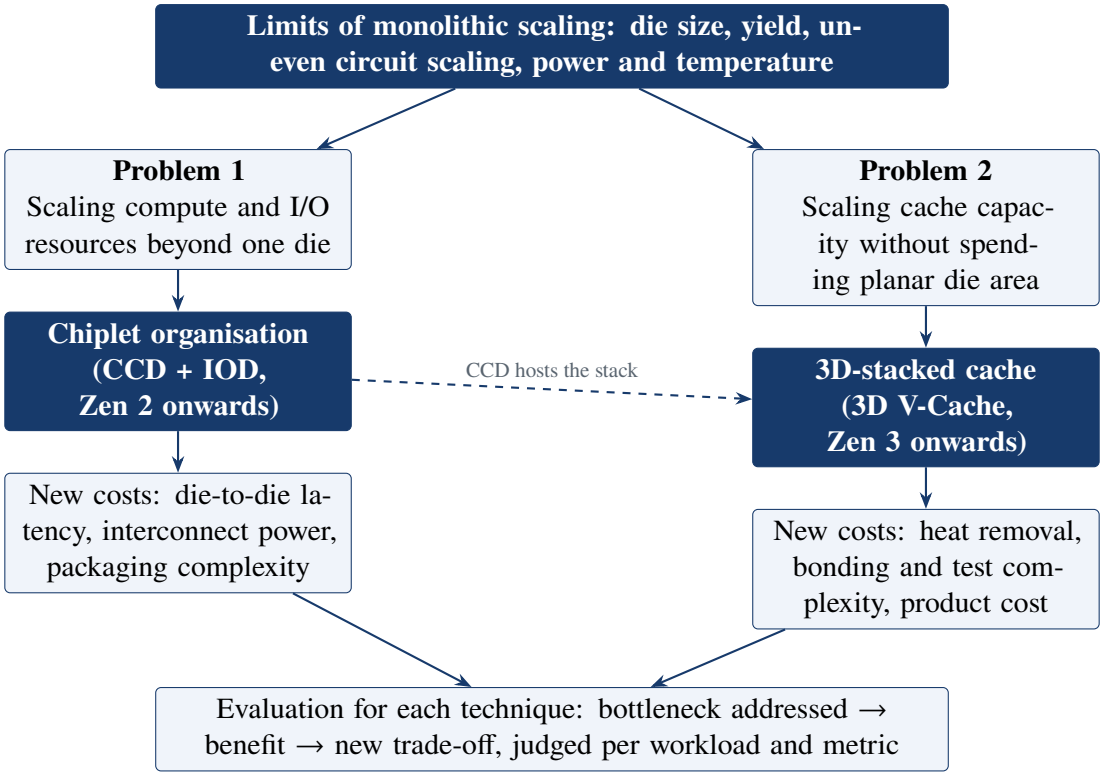


Figure 2: Research framework: two linked scaling problems, the AMD responses studied in this paper, and the evaluation question applied to each.

1.4 Research questions and objectives

The central research question is:

> *How do chiplet-based organisation and 3D-stacked cache help modern processors address limits in performance, scalability, memory latency, power and die-level integration, and what new trade-offs do they introduce?*

It is divided into four sub-questions, the second with a supporting question:

RQ1 Which architectural and manufacturing limits make a single large die an increasingly costly way to scale a processor?

RQ2 How does partitioning a processor into compute and I/O dies change memory latency, bandwidth, power and product flexibility?

RQ2a How does AMD divide processor functionality between the CCDs and the IOD, and why does it connect them through the organic package substrate rather than a 2.5D or 3D interconnect?

RQ3 Why does last-level-cache capacity matter, and what does vertical stacking change compared with expanding the cache in the plane of the die?

RQ4 For which workloads does a larger stacked L3 produce measurable benefit, and how can that benefit be separated from differences in frequency, core count and memory configuration?

For every technique the paper applies the same three-part test: *which bottleneck does it address, what benefit does it provide, and what new bottleneck, cost or trade-off does it create?* Table 2 shows where each question is answered.

Table 2: Where the research questions are answered

Question	Main evidence	Sections
RQ1	Reticle limit, yield and dies-per-wafer models, uneven scaling of circuit types	Part III (sections 6, 6.2 and 9.1)
RQ2, RQ2a	AMD's chiplet disclosures, die-to-die link figures, NUMA latencies, packaging options	Part III (sections 7, 8 and 8.3)
RQ3	Planar limits, hybrid bonding, the stacked-cache interface, thermal and manufacturing evidence	Part IV
RQ4	AMD-reported and independent measurements, graded for confounding factors	Part V
All	Comparison, interactions and conclusions	Part VI

1.5 Scope

AMD Zen processors are the principal case study because AMD has published unusually detailed technical descriptions of both its chiplet organisation and its stacked cache [2], [3], [5], [11]. They are, however, a case study and not the whole subject: the architectural principles apply to chiplet and die-stacked designs in general.

The following are deliberately outside scope:

- fabrication chemistry and process details beyond what is needed to explain an architectural decision;
- reverse engineering of undisclosed Zen internals;
- a catalogue of product specifications;
- vendor contests. Comparisons are always tied to a stated workload, metric and time period, and the paper makes no general claim that one vendor "beats" another.

1.6 Methodology

The study combines three kinds of evidence:

1. **Textbook foundations.** Chapters 9–12 of Sarangi's *Basic Computer Architecture* supply the models used throughout: the performance equation, pipelining limits, power and temperature, the memory hierarchy and average memory access time, and multicore coherence and interconnects [1].

2. **Primary technical disclosures.** Peer-reviewed conference papers and vendor technical presentations describe what was built and why. Vendor presentations are attributed to the vendor and treated as engineering claims rather than independent measurements.

3. **Published research and benchmarks.** Part V analyses published measurements of chiplet and stacked-cache processors. The paper does not perform new hardware measurements; its quantitative contribution lies in organising, normalising and critically comparing existing results (section 21).

Two practices run through the analysis. First, *worked examples* apply the textbook's models to published numbers: yield and cost per wafer, connection density, average memory access time, the stacked-cache trade-off and clock correction. Each states its inputs, and any assumed value is labelled as such. Second, sources are graded. Vendor presentations are used for what was built; independent measurements, preferably with fixed clocks or performance counters, are used for what it achieves; and where a value had to be read from a published chart rather than a table, the text says so and gives the reading accuracy.

1.7 Contributions

This paper contributes no new measurements; its contributions are analytical:

- a single argument linking chiplet partitioning and 3D-stacked cache as responses to two related scaling limits, die cost and last-level cache capacity;
- an explanation of both techniques in terms of the textbook's performance, power, thermal and memory-hierarchy models;
- a structured comparison of planar, side-by-side and vertically stacked cache expansion;
- a synthesis of AMD-reported and independent performance evidence for stacked cache across three Zen generations, graded by which confounding factors each comparison controls (table 23);
- an account of how AMD's move to an inverted stack in Zen 5 bears on the thermal cost of the first design.

1.8 Organisation of the paper

Part II reviews the architectural foundations. Part III analyses chiplet-based organisation. Part IV examines 3D-stacked cache and AMD 3D V-Cache. Part V presents the AMD case study and quantitative analysis, and Part VI compares the approaches, identifies limitations and outlines future directions. Self-contained derivations longer than about half a page appear as appendices.

Part II

Architectural Foundations

Target length: about 25 pages. Draft status: all sections drafted from Sarangi Ch. 9–12; expand worked examples and figures.

This part sets out the models that the rest of the paper uses to judge chiplets and stacked cache. It follows Chapters 9–12 of the course textbook [1]. The textbook supplies the theory; it does not describe modern compute-die/I/O-die partitioning or stacked cache, which later parts support with external sources.

2 Processor Organisation and Performance Metrics

2.1 The performance equation

The textbook defines the performance of a processor only with respect to a given program, as the reciprocal of its execution time [1, sec. 10.9.1]. Execution time τ can be split into three factors:

$$\tau = \frac{\#\text{instructions} \times \text{CPI}}{f}, \qquad P \propto \frac{\text{IPC} \times f}{\#\text{instructions}}, \tag{1}$$

where f is the clock frequency, CPI (cycles per instruction) is the average number of cycles per dynamic instruction, and IPC (instructions per cycle) is its reciprocal [1, sec. 10.9.1].

Equation (1) is the lens for the whole paper. A larger cache does not change the instruction count or, directly, the clock frequency; it acts through IPC by reducing the cycles spent waiting for memory. A chiplet organisation, by contrast, can change all three: it can enable more cores (more throughput), alter memory latency (IPC), and change the frequency and power each die can sustain. When published results compare two processors, they must therefore be checked for differences in f as well as in cache size (section 24).

2.2 Baseline processor organisation

The textbook builds its baseline processor, SimpleRisc, from five stages: instruction fetch (IF), operand fetch (OF), execute (EX), memory access (MA) and register writeback (RW) [1, sec. 9.9.1]. The *data path* holds and processes information (registers, memory elements, the ALU), while the *control path* generates the signals that move instructions and data through it; the control path can be a hardwired control unit or a microprogrammed one, which trades speed for flexibility [1, secs. 9.2–9.8].

Two features of this baseline matter for the rest of the paper. First, the MA stage is the processor's only contact with memory: every load and store passes through it, so the speed of the memory system directly sets how long that stage can take. Second, the organisation assumes a single processor on a single piece of silicon. Everything that follows, from pipelining to multicore processors and finally to chiplets, can be read as a series of departures from this baseline, each made to use a larger transistor budget without lengthening the clock cycle or exceeding the power budget.

3 Pipelining, Power and Thermal Limits

3.1 Pipelining and its limits

In the basic processor, most of the hardware is idle at any moment: when an instruction is in the MA stage, the IF stage has nothing to do. Pipelining overlaps instructions so that each of the five stages works on a different instruction in the same cycle, with pipeline registers between stages carrying each instruction's state forward [1, sec. 10.12.1]. Ideally this completes one instruction per cycle while allowing the clock frequency to rise roughly in proportion to the number of stages.

The ideal is broken by *hazards*. A data (read-after-write) hazard occurs when an instruction needs a value that an earlier instruction has not yet produced; a control hazard occurs because the outcome of a branch is not known when the following instructions are fetched; a structural hazard occurs when two instructions need the same hardware unit [1, sec. 10.4]. Interlocks resolve hazards by stalling, and forwarding passes results directly between stages, which removes every data hazard except the load-use hazard [1, sec. 10.12.1]. The effect on performance is captured by

$$\mathrm{CPI} = \mathrm{CPI}_{\mathrm{ideal}} + \text{stall rate} \times \text{stall penalty}, \tag{2}$$

so that, for example, if 10% of instructions suffer a one-cycle load-use stall and 20% are taken branches costing two cycles each, the CPI rises from 1 to 1.5 [1, sec. 10.9.3, Example 140]. Deeper pipelines raise the frequency but also the stall penalty, which is why the textbook derives an optimal pipeline depth rather than an unlimited one [1, sec. 10.9.3].

Worked example: how deep should a pipeline be?

The textbook models a pipeline of k stages whose total logic delay $t_{\max}$ is divided equally among the stages, each stage adding a latch delay l, and whose stall penalty grows in proportion to depth, ck, for a stall rate of r stalls per instruction. Maximising performance over k gives

$$k_{\mathrm{opt}} \approx \sqrt{\frac{t_{\max}}{l\, r\, c}} \tag{3}$$

[1, sec. 10.9.3]. With $t_{\max}/l = 20$, $r = 0.2$ and $c = 0.6$, $k_{\mathrm{opt}} = \sqrt{20/0.12} \approx 13$ stages [1, sec. 10.9.3, Example 142]. The textbook draws three conclusions: programs with many dependencies favour shallower pipelines; better forwarding, which lowers c, favours deeper ones; and because logic and latches have been getting faster at about the same rate, the optimal depth

has changed little over time [1, sec. 10.9.3]. The same reasoning explains why designers stopped raising clock frequency by deepening pipelines (section 1), and why a design that reduces the stall rate, such as a larger cache that removes memory stalls, is a more durable source of performance than a higher clock.

Equation (2) is the link between pipelining and the memory hierarchy. A load that misses in the cache is a stall whose penalty is not one or two cycles but the miss penalty of equation (6), which can be tens or hundreds of cycles. Out-of-order processors soften this: they fetch and decode in order, rename registers to remove false dependencies, and issue any instruction whose operands are ready from an instruction window that typically holds 64–128 entries [1, sec. 10.11.4]. Independent work can therefore proceed while a load waits. But the window is finite, so a long miss to main memory eventually fills it and stalls the pipeline. Reducing the number of such misses, which is what a larger last-level cache does, therefore still matters even for aggressive out-of-order cores.

3.2 Dynamic and leakage power

The textbook models dynamic power, the energy dissipated when circuit nodes switch, as

$$P_{\text{dyn}} \propto \sum_i \alpha_i C_i V^2 f, \tag{4}$$

where α_i is the activity factor and C_i the capacitance of node i, V the supply voltage and f the frequency [1, sec. 10.10.2]. Leakage (static) power arises because transistors do not switch off perfectly; it is consumed all the time regardless of activity, and the textbook notes that it can account for roughly 20–40% of a high-performance processor's power budget [1, sec. 10.10.3].

Both terms matter for this paper:

- **Wires are capacitance.** Long on-die or die-to-die wires increase C in equation (4), so moving data farther costs energy as well as time. This is one reason the distance between cores and cache matters (section 12).
- **Large SRAM arrays leak.** A large cache adds leakage even when most of it is idle, which is part of the cost of simply growing the LLC on a planar die.

3.3 Temperature

Power that is dissipated becomes heat, which must flow from the transistors through the die, a thermal interface material and a heat spreader to the cooler [1, sec. 10.10.1] (figure 3). The textbook gives a sense of the scale: a 200–400 mm^2 die dissipating 60–100 W would heat up to around 200 °C without these measures, and even with them processors run at 60–100 °C, reaching 120 °C under heavy load [1, sec. 10.10.1].

Higher temperature raises leakage, which raises power further, and accelerates ageing [1, sec. 10.10.1]. Stacking a die on top of a compute die changes this heat path, which is the main thermal question raised in section 16.

Heat is not produced evenly. The textbook notes that the ALUs dissipate a lot of heat while memory elements stay relatively cool, and that the heat spreader exists to even out the resulting

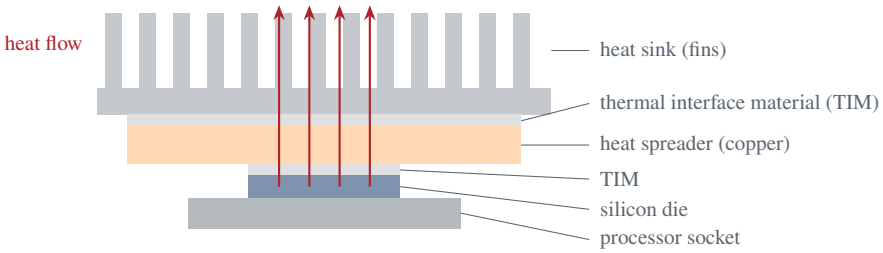


Figure 3: The heat path from a processor die to the air, redrawn after Sarangi [1, sec. 10.10.1, Fig. 10.36]. Each layer adds thermal resistance; stacking a second die on the processor adds another layer between the transistors and the spreader (section 16).

hot spots [1, sec. 10.10.1]. It models the steady-state temperature by dividing the die into a grid of m points:

$$T - T_{\text{amb}} \;=\; \Delta T \;=\; A \times P_{\text{tot}}, \tag{5}$$

where P_{tot} is the vector of power dissipated at each grid point, T_{amb} the ambient temperature and A the $m \times m$ *thermal resistance matrix* [1, sec. 10.10.4]. Because $P_{\text{tot}} = P_{\text{dyn}} + P_{\text{leak}}$ and leakage itself grows with temperature, equation (5) and the leakage model form a feedback loop that must be iterated until it converges [1, sec. 10.10.4]. Two consequences follow for stacking. Placing an extra layer over a hot region increases the entries of A for that region, so the same power produces a higher temperature; and a cool structure such as a cache array is the cheaper thing to put in the heat path. 3D V-Cache follows both rules: its cache die sits over the CCD's L3, and only structural silicon covers the cores (section 16).

3.4 Combining energy and delay: the ED^2 metric

A design that is faster but uses more energy is not necessarily better. To compare two designs fairly, the textbook equalises their delays by voltage–frequency scaling, assuming delay $D \propto 1/f$, $f \propto V$ and energy $E \propto V^2$; under these assumptions the product ED^2 is independent of the voltage and frequency setting and depends only on the design itself [1, sec. 10.10.5]. A design with lower ED^2 is therefore better at any operating point, and the textbook observes that many performance-enhancing schemes fail this test because they increase energy disproportionately [1, sec. 10.10.5]. The metric is a useful lens for 3D V-Cache, which raises the power budget and lowers the clock of some products (section 16.2) while reducing the time and energy spent on off-chip memory accesses; whether it lowers ED^2 therefore depends on the workload.

4 The Memory Hierarchy

4.1 Why a hierarchy is needed

The textbook frames memory design as a conflict between capacity and speed. Per unit of silicon, DRAM stores far more data than SRAM but is much slower; a memory built entirely of DRAM would make every load and store wait for main memory, while a memory built entirely of fast on-chip SRAM cannot hold a program's data [1, sec. 11.1.1]. As an illustration, if one third of instructions access a 100-cycle memory, the effective CPI of an otherwise ideal pipeline rises to

$1 + \frac{1}{3}(100 - 1) = 34$ [1, sec. 11.1.1].

The way out is *locality*: programs tend to reuse recently used data (temporal locality) and to access data near recently used data (spatial locality) [1, sec. 11.1.2–11.1.6]. A memory hierarchy keeps recently used blocks in small, fast caches close to the core and backs them with progressively larger, slower levels (figure 4).

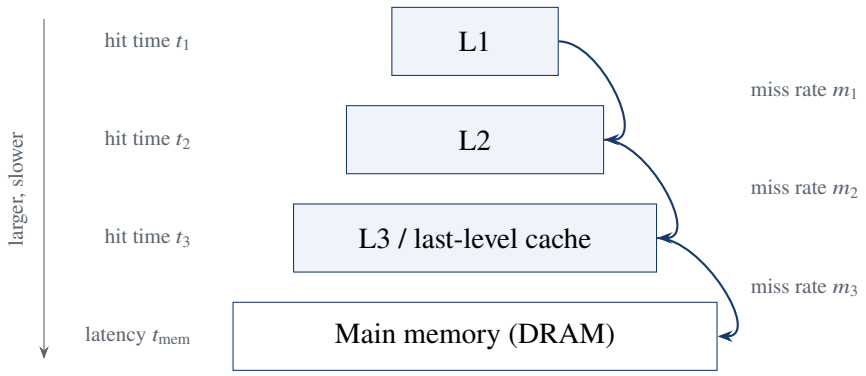


Figure 4: A multi-level cache hierarchy with the quantities used in the recursive AMAT expression (equation (6)). Capacity grows and speed falls moving down the hierarchy.

4.2 SRAM and DRAM arrays

Caches are built from SRAM. A standard SRAM cell uses six transistors: two cross-coupled inverters hold the bit, and two access transistors, switched by a shared *word line*, connect it to a pair of complementary *bit lines* [1, sec. 7.4.1]. Cells are laid out as a two-dimensional array: a decoder raises one word line, every cell in that row drives its bit lines, and a column multiplexer selects the bits of interest [1, sec. 7.4.1]. Because hundreds of cells share a bit line, the bit line has a high capacitance and would be slow to charge fully. Reads therefore pre-charge both bit lines to half the supply voltage and let *sense amplifiers* detect a small voltage difference [1, sec. 7.4.1]. The textbook also notes that as an array grows it must be kept roughly square, otherwise the capacitive load on its word lines or bit lines becomes prohibitive [1, sec. 7.4.1]. Large caches are therefore built as many smaller arrays and banks (section 5), connected by wires whose length grows with total capacity. Figure 5 shows the cell and the array.

Cell size sets the density gap between SRAM and DRAM. Using 2012-era figures, the textbook gives an SRAM cell an area of about $0.08\ \mu\text{m}^2$ and an array access time of 1–5 cycles, against about $0.005\ \mu\text{m}^2$ and 50–200 cycles for a DRAM cell [1, sec. 11.1.1, Table 11.1]. SRAM is thus roughly sixteen times larger per bit but far faster, which is why it is used for on-chip caches and why a cache of tens of megabytes takes a large share of a die (section 12).

4.3 Average memory access time

The average memory access time (AMAT (average memory access time)) of a hierarchy is defined recursively [1, sec. 11.3.1]:

$$\begin{aligned} \text{AMAT} &= t_1 + m_1 \times \text{penalty}_1, \\ \text{penalty}_k &= t_{k+1} + m_{k+1} \times \text{penalty}_{k+1}, \end{aligned} \tag{6}$$

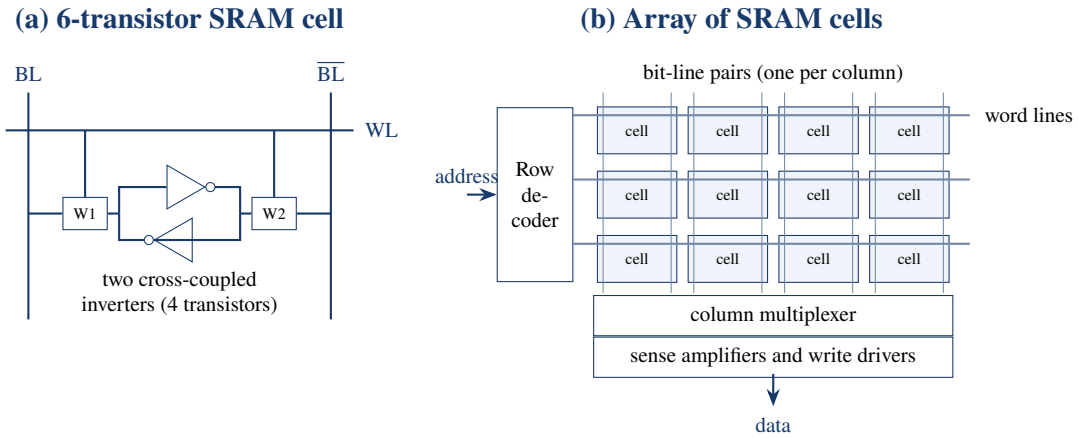


Figure 5: (a) A six-transistor SRAM cell: the word line (WL) turns on the access transistors W1 and W2, connecting the cross-coupled inverters to the complementary bit lines. (b) Cells are arranged in a two-dimensional array; a decoder raises one word line, every cell in that row drives its bit lines, and a column multiplexer and sense amplifiers select and read the bits required. Redrawn after Sarangi [1, sec. 7.4.1, Figs. 7.26–7.27].

where t_k is the hit time and m_k the local miss rate of level k, and the penalty of the last cache level is the main-memory latency. The textbook's worked example uses an L1 with 10% miss rate and 1-cycle latency, an L2 with 10% miss rate and 10-cycle latency, and a 100-cycle memory, giving $T = 1 + 0.1(10 + 0.1 \times 100) = 3$ cycles against 100 cycles for a flat memory [1, sec. 11.1.7, Example 147].

Worked example: AMAT on two real processors

Equation (6) can be evaluated with measured latencies. The FPS Review ran AIDA64's cache and memory benchmark on a Zen 2 Ryzen 7 3700X and a Zen 3 Ryzen 7 5800X with identical DDR4-3600 memory [12]; table 3 uses those latencies. The benchmark does not report miss rates, so the example assumes illustrative local miss rates of $m_1 = 5\%$ for L1, $m_2 = 30\%$ for L2 and $m_3 = 40\%$ for L3; real values vary widely between programs.

Table 3: Worked example: AMAT from measured latencies [12] and assumed miss rates ($m_1 = 5\%$, $m_2 = 30\%$, $m_3 = 40\%$)

	t_1	t_2	t_3	t_{mem}	**AMAT**	**L3 + mem.**
Ryzen 7 3700X (Zen 2)	0.9 ns	2.7 ns	9.9 ns	68.8 ns	1.60 ns	0.56 ns
Ryzen 7 5800X (Zen 3)	0.8 ns	2.5 ns	10.9 ns	61.8 ns	1.46 ns	0.53 ns
5800X with $m_3 = 30\%$ and $t_3 + 4$ cycles	0.8 ns	2.5 ns	11.7 ns	61.8 ns	1.38 ns	0.45 ns

For the 5800X, AMAT $= 0.8 + 0.05\,[\,2.5 + 0.3\,(10.9 + 0.4 \times 61.8)\,] \approx 1.46$ ns. Three lessons follow. First, the L1 hit time dominates: 0.8 of the 1.46 ns is paid on every access. Second, the L3 and memory terms are weighted by $m_1 m_2 = 1.5\%$, so even a 62 ns memory latency adds only about half a nanosecond on average; that average, however, hides long stalls for the individual loads that miss, which an out-of-order core can only partly overlap. Third, the last row anticipates Part IV: if a larger L3 cut its local miss rate from 40% to 30% at a cost of four extra cycles (about

0.8 ns at the 5800X's measured 4.85 GHz), the L3-and-memory share would fall from 0.53 to 0.45 ns and the AMAT by about 5%. Whether that matters depends entirely on how often a program reaches the L3, which is why the benefit of a larger LLC is so workload-dependent.

Equation (6) shows precisely what a larger last-level cache can and cannot do:

- it can lower m_{LLC}, so fewer accesses pay the main-memory latency;
- but if making the cache larger also lengthens its hit time t_{LLC}, part of the gain is lost on every LLC hit.

The design problem for a large LLC is therefore to add capacity while keeping the added hit latency small. That is the problem 3D stacking attacks (Part IV).

4.4 Classifying cache misses

The textbook divides misses into three classes [1, sec. 11.3.2]:

- **Compulsory (cold) misses** occur the first time a block is accessed.
- **Capacity misses** occur when the program's working set—the blocks it uses over a typical interval—is larger than the cache.
- **Conflict misses** occur in direct-mapped and set-associative caches when more blocks map to one set than the set can hold.

A larger LLC primarily removes capacity misses. It therefore helps workloads whose working set exceeds the original cache but fits in the larger one, and helps little when the working set either already fits or is far larger than either size. This simple argument predicts the strongly workload-dependent results examined in Part V.

4.5 Cache organisation and policies

A cache stores fixed-size *blocks* (typically 32–128 bytes) and records which memory addresses it holds in a *tag array* beside the *data array*. An address is divided into a tag, an index and an offset within the block. In a *fully associative* cache a block may be placed anywhere, so every tag must be compared on each access; this is practical only for small structures because of its area and power. A *direct-mapped* cache allows each block exactly one location, selected by the index bits, which is simple and fast but suffers conflict misses. A *set-associative* cache is the compromise used almost universally: the index selects a set of K locations (the associativity, typically 2, 4 or 8 in the textbook's examples), and the tags of that set are compared in parallel [1, sec. 11.2.2]. AMD's Zen 3 L3, for comparison, is 16-way set associative both with and without V-Cache [6, slides 19–20]. Figure 6 shows the lookup.

Worked example: splitting an address in the Zen 3 caches

The textbook's method divides an address into offset, index and tag [1, sec. 11.2.2]. For a cache of capacity S, associativity K and block size B, the number of sets is $S/(KB)$; the offset needs $\log_2 B$ bits, the index $\log_2$ of the number of sets, and the tag the remaining bits. Zen 3

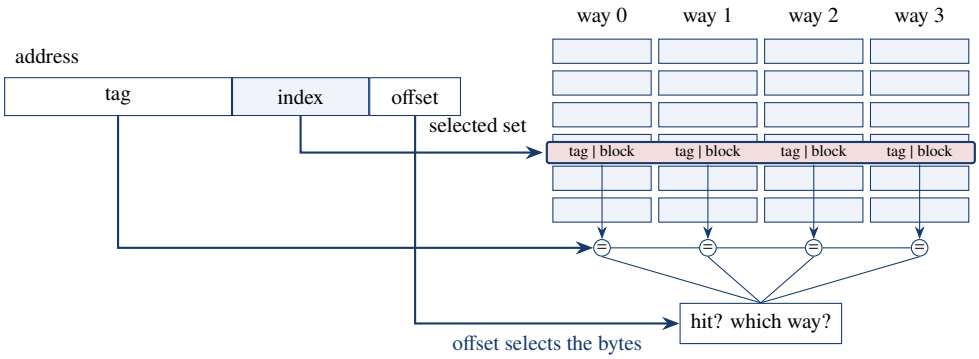


Figure 6: Lookup in a 4-way set-associative cache. The index bits select one set; the tags stored in all four ways of that set are compared with the address tag in parallel; a match identifies the way holding the block, and the offset selects the bytes within it. After Sarangi [1, sec. 11.2.2].

uses 64-byte lines throughout [11], an 8-way 32 KB L1 data cache, an 8-way 512 KB L2 and a 16-way 32 MB L3 per CCD [6, slide 19]; independent measurements confirm the L1, L2 and L3 organisation, with 64-byte lines throughout [13]. AMD documents 48-bit physical addressing for its Zen 3 EPYC 7003 processors when memory encryption is off; enabling Secure Encrypted Virtualization reduces the usable width to 43 or 44 bits, because some address bits are then used for encryption keys [14, p. 10]. Table 4 applies the method with 48 bits.

Table 4: Worked example: address fields for the Zen 3 caches (64-byte lines; 48-bit physical address, as documented for EPYC 7003 with memory encryption off)

Cache	Capacity, ways	Sets	Offset	Index	Tag
L1 data	32 KB, 8	$32\,\mathrm{K}/(8\times64) = 64$	6 bits	6 bits	36 bits
L2	512 KB, 8	$512\,\mathrm{K}/(8\times64) = 1024$	6 bits	10 bits	32 bits
L3 (one CCD)	32 MB, 16	$32\,\mathrm{M}/(16\times64) = 32\,768$	6 bits	15 bits	27 bits

Two observations matter later. The L1's 64 sets and 6-bit offset together use only the low 12 bits of the address, the size of a 4 KB page; this is a common design choice that lets the cache be indexed in parallel with address translation, although AMD's sources used here do not state it for Zen 3. And a bigger cache does not shrink the tag much: the 32 MB L3 still stores 27 bits per line before any state, replacement or error-correction bits, so tag storage grows almost in proportion to capacity (section 15.2).

Organisation can also be matched to how a program walks through its data. Wang, for example, proposes an L1 cache for two-dimensional data, such as matrices and images, that stores data in a hierarchical Z-order layout across multiple banks so that either a square tile or a row can be read in one access; in matrix-multiplication simulations this reduced both L1 cache and TLB misses compared with a conventional row-by-row layout [15]. The example is far from AMD's last-level cache, but it illustrates the same principle from the other direction: instead of adding capacity to hold a working set, it rearranges data so that the existing capacity is used better.

When a set is full, a *replacement policy* chooses a victim: random replacement is simplest, FIFO evicts the oldest block, and least-recently-used (LRU), which follows directly from the stack-distance view of temporal locality, is among the most effective [1, sec. 11.2.5]. Writes are handled either *write-through*, sending every write to the next level, or *write-back*, marking

a block as modified and writing it back only on eviction, which reduces traffic at the cost of tracking state [1, sec. 11.2.3]. The stacked V-Cache keeps this organisation intact: each die's portion of the L3 has its own data, tag and LRU arrays [6, slide 20].

The textbook summarises the main cache optimisations and their side effects; table 5 reproduces the entries most relevant to this paper. The table makes the paper's central tension explicit: the standard remedy for capacity misses is a larger cache, and its listed costs are higher latency, higher power and more area.

Table 5: Selected memory-system optimisations and their costs, after Sarangi [1, Table 11.2]

Technique	Targets	Disadvantages
Larger block size	compulsory misses	fewer blocks in the cache
Prefetching	compulsory and capacity misses	complexity; may displace useful data
Larger cache	**capacity misses**	**higher latency, higher power, more area**
Higher associativity	conflict misses	higher latency, higher power
Victim cache	conflict misses	extra complexity
Small, simple cache	hit time	higher miss rate
Write buffer, critical word first	miss penalty	extra complexity

3D V-Cache can be read as an attempt to take the third row of table 5 while avoiding most of its disadvantages: the area cost moves to a separate die, and the latency cost is held to four cycles (section 15).

4.6 Beyond SRAM: alternative LLC technologies

Stacking SRAM is not the only way to obtain a larger last-level cache. Research has also explored denser memory technologies. Mittal and Vetter's survey notes that DRAM offers roughly eight times the density of SRAM, and that die-stacked DRAM can provide 8 to 15 times the bandwidth of off-chip DRAM at one half to one third of its latency, which has made stacked DRAM a candidate for very large LLCs [16, sec. 1]. Its longer access latency, the size of the tag store such a cache needs, and thermal management are the main design challenges the survey identifies [16, secs. 1–2]. Non-volatile memories such as STT-RAM offer high density and low leakage, but with much longer write latency and higher write energy than SRAM. Kim *et al.* therefore study hybrid LLCs that combine an SRAM region for write-intensive blocks with an STT-RAM region for the rest, and report a 26% average speedup in simulation from a better block-placement policy [17]. These approaches change the *technology* of the cache, whereas AMD's 3D V-Cache keeps SRAM and changes its *placement*; the comparison is taken up in Part VI.

5 Multicore Processors, Coherence and Interconnects

5.1 From one core to many

The textbook traces the move to multicore processors from the mid-2000s as transistor budgets continued to grow while single-core power and complexity limits tightened [1, sec. 12.1]. Multicore processors raise two questions that matter directly for chiplets: how cores keep a consistent view of shared data, and how requests travel between cores, caches and memory.

5.2 How far parallelism goes: Amdahl's law

More cores help only to the extent that a program can use them. If a fraction f_{seq} of a program's running time is sequential and the rest divides perfectly among P processors, the speedup is

$$S = \frac{1}{f_{\text{seq}} + (1 - f_{\text{seq}})/P}, \tag{7}$$

which approaches $1/f_{\text{seq}}$ however many processors are added [1, sec. 12.2.3]. With $f_{\text{seq}} = 5\%$, eight cores give a speedup of $1/(0.05 + 0.95/8) \approx 5.9$, sixteen give 9.1 and sixty-four give 15.4, against a limit of 20 (figure 7). The textbook's own observation is that for $f_{\text{seq}} = 5\%$ there is no appreciable difference between 35 and 200 processors [1, sec. 12.2.3].

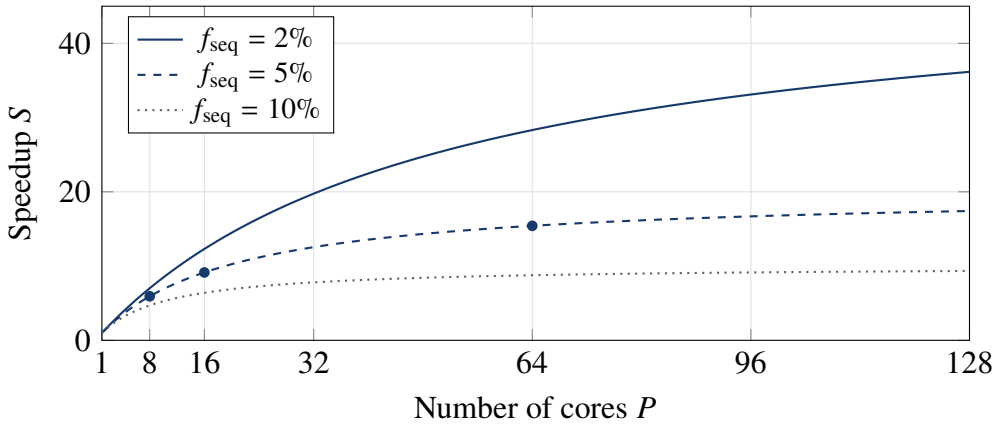


Figure 7: Amdahl's law, equation (7), for three sequential fractions, after Sarangi [1, sec. 12.2.3, Fig. 12.4]. The marked points are the worked example for $f_{\text{seq}} = 5\%$.

Amdahl's law has two consequences for this paper. First, a 64-core chiplet processor does not mainly serve one parallel program: its typical use is to run many unrelated programs or requests at once, which the textbook calls loosely coupled multiprocessing and which scales almost perfectly because the programs barely interact [1, sec. 12.2.1]. That is the market the chiplet organisation serves. Second, for any single program the sequential part still runs on one core, so the performance of each core, set by its clock, its IPC and the cache it can reach, still decides the limit. Stacked cache improves exactly that per-core performance, which is why it complements rather than competes with adding cores.

5.3 Cache coherence

When several caches can hold copies of the same address, cache coherence rules ensure that all threads observe writes to that address in a consistent order [1, sec. 12.4.2]. In a chiplet

processor, a coherence transaction between cores on different compute dies must cross the die-to-die interconnect, so the physical partitioning becomes visible as latency (section 8).

A multicore memory system can use *shared* caches, which every core accesses, or *private* caches, each serving one core or a small group of cores. The textbook compares them directly (table 6): private caches are small, fast and close to their cores but must be kept coherent and replicate shared data, while shared caches avoid replication and coherence traffic but are large, slow and far from most cores [1, sec. 12.4.4]. Its conclusion is that the first levels should be private and the lower, larger levels shared.

Table 6: Private versus shared caches, after Sarangi [1, Table 12.3]

Attribute	Private cache	Shared cache
Area	low	high
Speed	fast	slow
Proximity to the processor	near	far
Scalability in size	low	high
Data replication	yes	no
Complexity	high (needs coherence)	low

A large shared cache is not built as one monolithic array. To serve many cores at once, it is split into *banks*, each a smaller cache in its own right selected by some of the address bits; banking reduces contention and makes each bank faster and more power-efficient [1, sec. 12.4.5].

Private caches are made to look like one shared cache by a coherence protocol. The textbook states the requirement as two axioms: *completion*, every write must eventually complete, and *order*, all threads must see the writes to one address in the same order [1, sec. 12.4.2]. If one thread writes 1 and then 2 to x while another reads x twice, the reader may see (0,1), (1,2) or (2,2), but never (2,1), because that would show the writes in the opposite order to the writer's [1, sec. 12.4.2, Example 151].

In a *snooping* protocol the caches share a bus and broadcast read misses, write misses and writes; each block is tracked in a small state machine with the states Modified, Shared and Invalid [1, sec. 12.4.6]. Because the bus carries one message at a time, every cache sees the same order of messages, which satisfies the order axiom. The textbook develops two such protocols. In *write-update*, every write is broadcast so that all copies are updated. In *write-invalidate*, a cache that wants to write first invalidates all other copies; a block may then be shared by many readers or owned by a single writer, but never both, and writes to an owned block need no messages at all [1, sec. 12.4.6]. Figure 8 shows the write-invalidate state machine.

Worked example: one shared variable, two cores

Table 7 traces a write-invalidate protocol through five accesses by two cores, A and B, to the same block, using the transitions of figure 8. Every step generates a bus message, and steps 2 and 4 also move the block's data between the caches.

Two points carry over to chiplets. First, every message and every data transfer in the table must travel between the two caches. If A and B are on the same Zen CCD, the transfer stays on the die;

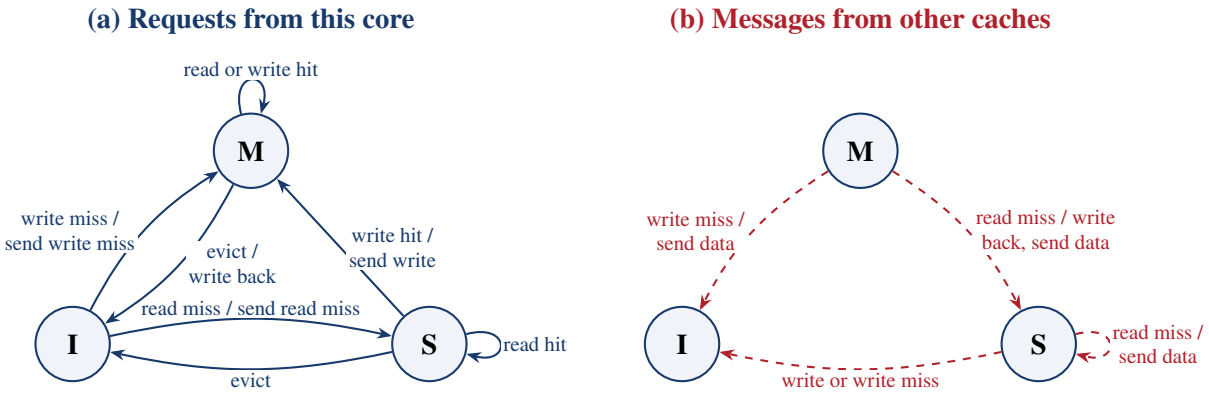


Figure 8: The write-invalidate (MSI) snooping protocol for one cache block, redrawn after Sarangi [1, sec. 12.4.6, Figs. 12.13–12.14]. (a) Transitions caused by this cache's own processor; (b) transitions caused by messages that other caches place on the bus. A block may be held by many caches in S, but by only one in M.

Table 7: Worked example: block states and bus traffic for two cores sharing one block (write-invalidate MSI)

	Access	A	B	Bus message and data movement
1	A reads x	I → S	I	read miss; data from the lower level
2	B reads x	S	I → S	read miss; A or the lower level supplies the data
3	A writes x	S → M	S → I	write; B invalidates its copy
4	B reads x	M → S	I → S	read miss; A writes the block back and sends it to B
5	B writes x	S → I	S → M	write; A invalidates its copy

if they are on different CCDs, each one crosses the die-to-die links through the I/O die (section 8). Second, a block that is written alternately by two cores ping-pongs between them, so the cost of the hop is paid repeatedly. Software that places communicating threads on the same CCD avoids this, which is one reason EPYC processors expose their CCD structure to the operating system (section 8). Broadcasting does not scale, so larger systems use a *directory* that records, for each block, which caches hold a copy, and sends messages only to those sharers [1, sec. 12.4.6].

The textbook's directory keeps the sharers as a bit vector with one bit per cache [1, sec. 12.4.6]. Its size shows why directories are themselves a design problem. For eight private caches, one bit each per 64-byte (512-bit) block adds $8/512 \approx 1.6\%$ to the storage of every tracked block; for 64 caches it adds $64/512 = 12.5\%$, and for 128 caches 25%. This is a worked example of the textbook's simple scheme, not a description of AMD's implementation, which the sources used here do not detail; but it explains why coherence tracking grows harder as more caches, and more dies, share one memory.

AMD's Zen designs combine both ideas at different scales. Within a Zen 3 CCD, eight cores share one L3; across CCDs, each L3 is private to its die, and the dies communicate through the I/O die over Infinity Fabric [11]. EPYC 7003 can even expose each CCD's L3 as a separate NUMA domain to software [11]. The textbook's trade-off therefore reappears one level up: the per-CCD L3 is "private" with respect to the whole socket, which keeps it close and fast but means that data shared across CCDs must be kept coherent over a die-to-die link (section 8).

5.4 On-chip and on-package interconnection networks

On a single die, cores, cache banks and memory controllers are usually arranged as tiles, each with a *router*; the routers and the links between them form a network-on-chip (NoC) that carries coherence messages, LLC requests and responses, and memory traffic [1, sec. 12.6.1]. Two figures of merit describe a topology: the *bisection bandwidth*, the minimum number of links that must be cut to split the network into two equal halves, which measures both reliability and aggregate bandwidth; and the *diameter*, the longest of all shortest paths between nodes, which bounds latency [1, sec. 12.6.2]. Table 8 lists the textbook's values.

Table 8: Diameter and bisection bandwidth of common topologies with N nodes, from Sarangi [1, sec. 12.6.3]

Topology	Diameter	Bisection bandwidth
Chain	$N-1$	1
Ring	$N/2$	2
Fat tree	$2\log N$	$N/2$
Mesh	$2\sqrt{N}-2$	$\sqrt{N}$
Torus	$\sqrt{N}$	$2\sqrt{N}$
Hypercube	$\log N$	$N/2$
Butterfly	$\log N$	$N/2$

Worked example: topologies at chiplet scale

Table 9 evaluates the formulas of table 8 for eight nodes, the number of CCDs in a Rome or Milan package or of cores in a Zen 3 CCD, and for 64 nodes, the number of cores in the full processor; figure 9 draws three of the cases.

Table 9: Worked example: diameter and bisection bandwidth from the formulas of table 8

	$N = 8$		$N = 64$	
Topology	**Diameter**	**Bisection**	**Diameter**	**Bisection**
Ring	4	2	32	2
Mesh	—	—	14	8
Torus	—	—	8	16
Hypercube	3	4	6	32

At eight nodes the ring is perfectly adequate: no node is more than four hops away. At 64 nodes the same ring would put cores up to 32 hops apart with a bisection of only 2, which is why large monolithic processors use meshes. AMD's chiplet designs avoid the problem by never building a 64-node network on one die. Within each CCD, a ring connects the eight cores to the L3 slices [18]; across the package, the CCDs form a star around the I/O die, in which any two CCDs are two hops apart (figure 9). Chips and Cheese attributes AMD's L3 latency, 24–28 cycles lower on average than that of Intel's mesh-based server processors in its tests, to this choice: a ring has lower latency and higher bandwidth, while a mesh scales better for a large monolithic die [19]. The cost of the star is that its hub carries all traffic between dies, which is the bandwidth limit examined in section 8.2.

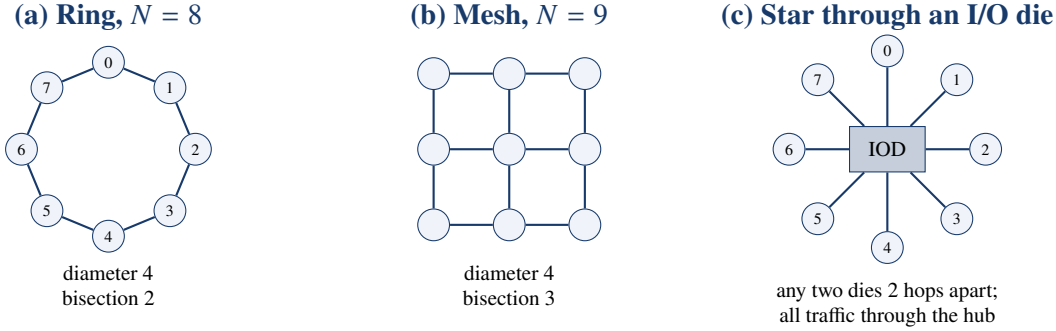


Figure 9: Three topologies. The ring and mesh follow Sarangi's definitions [1, sec. 12.6.3]; the star is the arrangement of Rome's eight CCDs around one I/O die (section 7). Within a CCD, Zen 3 and Zen 4 connect cores to L3 slices by a ring [18], [19].

What changes when a link leaves the die is the subject of Part III. On a die, a link is a set of short on-chip wires; between chiplets, it is a serialised link routed through the package, with its own transmitters, receivers and clocking. AMD's Rome processors connect every CCD to a single central I/O die, a star-like arrangement in which any two CCDs are two hops apart, which AMD credits with reducing the number of NUMA domains and the distances to memory compared with the previous generation [2].

Part III

Chiplet-Based Processor Architecture

Target length: about 25 pages. Draft status: all sections drafted, including organic, 2.5D and 3D connection options; illustrative latency, bandwidth, power and cost estimates need checking against further sources.

6 Why Large Monolithic Dies Become Costly

6.1 The reticle limit and die size

A single die cannot be larger than the area a lithography tool can expose in one field. AMD's Hot Chips 33 presentation plots server CPU and GPU die sizes growing towards this reticle limit over roughly the last fifteen years [5]. For its second-generation EPYC ("Rome") processor, AMD states that the 64-core top configuration would not be practical as a monolithic chip and would lie well beyond the maximum reticle size [2].

6.2 Manufacturing yield

Manufacturing defects are scattered across a wafer, so a larger die is more likely to contain one. The simplest model assumes that defects fall independently with a fixed average density D_0, which gives a Poisson yield e^{-AD_0} for a die of area A. Real defect densities vary across a wafer and from wafer to wafer, and the Poisson model ignores this. This paper therefore uses the yield model adopted in the 3D-IC cost analysis of Dong and Xie [20, Eq. 15]:

$$Y_{\text{die}} = Y_{\text{wafer}} \cdot \frac{1 - e^{-2AD_0}}{2AD_0}, \tag{8}$$

where A is the die area, D_0 the average defect density and Y_{wafer} the wafer yield. This is one of the yield functions derived by Murphy, who averaged the Poisson yield over a defect density that varies from chip to chip; this form assumes the density is spread uniformly between 0 and $2D_0$ [21, Eq. 11]. Appendix C derives it and compares it with Murphy's preferred intermediate model.

Table 10 gives an illustrative calculation with $D_0 = 0.2$ defects/cm^2 and $Y_{\text{wafer}} = 1$. It is a model result, not a measured processor result.

Doubling the die area lowers the modelled yield from about 82.4% to 68.8% (figure 10). Cost per good die rises faster than area for two reasons: fewer dies fit on each wafer, and a smaller fraction

Table 10: Illustrative Murphy-model yield for $D_0 = 0.2$ defects/cm^2 and $Y_{\text{wafer}} = 1$

Die area	$2AD_0$	Modelled yield Y_{die}
100 mm^2 (1 cm^2)	0.4	82.4%
200 mm^2 (2 cm^2)	0.8	68.8%

of them work. The exact increase depends on wafer cost, process node, defect distribution and test cost; appendix C derives the cost-per-good-die expression and shows why partitioning a design into smaller dies can lower the silicon cost even after paying for extra interface area.

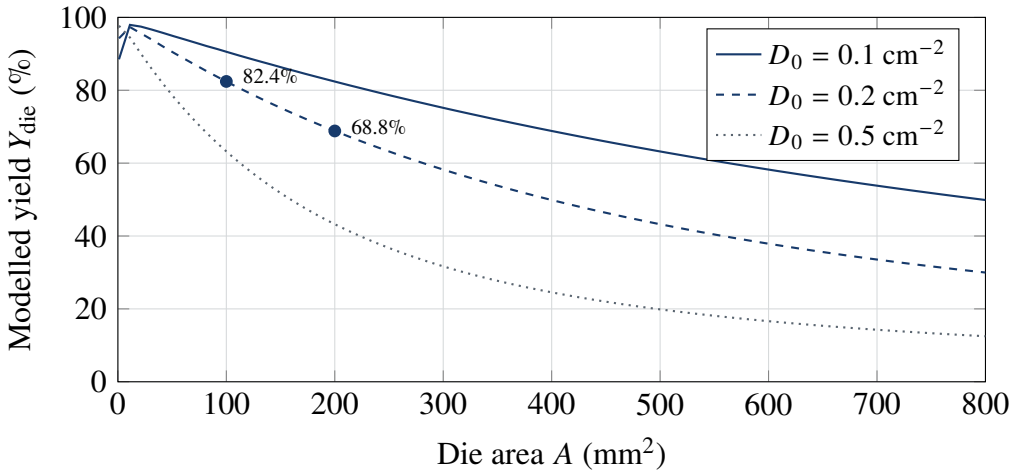


Figure 10: Murphy yield model (equation (8), $Y_{\text{wafer}} = 1$) for three illustrative defect densities. The marked points reproduce the worked example in section 6.2. Illustrative calculation, not measured processor data.

AMD's own material makes the same qualitative point. Its Zen 2 presentation lists "small die improve yield" among the motivations for the chiplet strategy [3], and its Hot Chips 33 presentation shows normalised cost per yielded square millimetre rising at each newer process node [5].

6.3 Uneven scaling of logic, SRAM and analog circuits

Moving a design to a newer node does not shrink every circuit equally. Naffziger *et al.* state that shrink factors at advanced nodes are much lower for analog circuitry than for digital logic and SRAM, and that memory interfaces and SerDes therefore gain very little performance, power or area from moving from 12 nm to 7 nm [2]. Building those circuits on the expensive leading-edge node wastes its cost. This observation is the basis of process-node specialisation (section 9).

Worked example: where the Zen 2 gains came from. AMD's Zen 2 presentation splits the doubling of instructions per unit of energy over the previous generation into three factors: 1.15× from higher IPC, 1.17× from design improvements and 1.47× from the 7 nm process [3, slide 4]. Multiplied together, $1.15 \times 1.17 \times 1.47 \approx 1.98$, which is AMD's "2×". The process contributed the largest single share, but only to the circuits that shrink with it. Logic and SRAM in the cores benefit; memory interfaces and SerDes, by Naffziger *et al.*'s account, barely do [2]. The chiplet

split therefore places the 7 nm process exactly where its 1.47× applies and spends the cheaper process everywhere else. The related, more recent observation that SRAM is falling behind logic is taken up in section 11.

7 Compute Dies, I/O Dies and Core Complexes

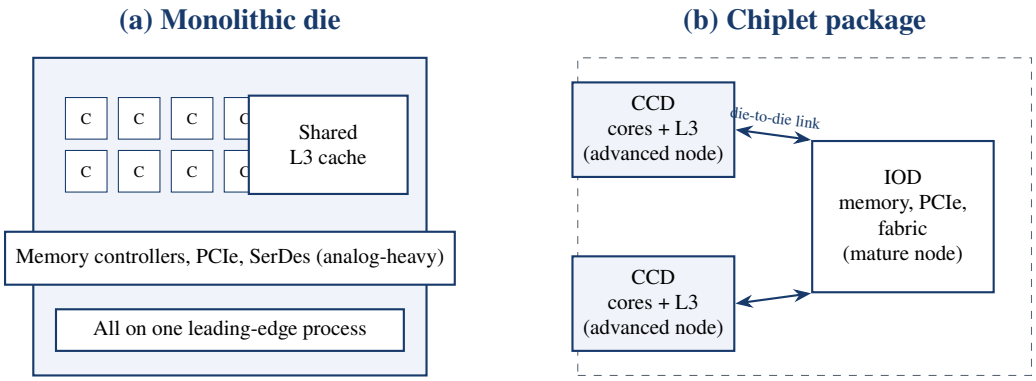


Figure 11: Schematic contrast between a monolithic processor die and a chiplet package with compute dies (CCDs) and an I/O die (IOD). Illustrative only; not to scale and not an actual AMD floorplan.

AMD's Zen 2 products ("Rome" for servers and "Matisse" for desktops) are built from three kinds of die [2], [3]:

- an eight-core CCD (Core Complex Die) of about 74 mm^2 on a 7 nm process, containing the cores and their L3 cache;
- a server IOD (I/O Die) of about 416 mm^2 on a 12 nm process;
- a smaller client IOD of about 125 mm^2, which in a standalone package also serves as the X570 chipset.

The same CCD is reused across server and desktop products; only the IOD and the number of populated CCDs change [2]. Within each Zen 2 CCD, the cores are grouped into two four-core CCX (Core Complex)s, each with up to 16 MB of L3 [3]. Inside a CCX, each core has a private 512 KB L2, and the 16 MB L3 is built from four 4 MB slices, one beside each core, with 32-byte-per-cycle paths throughout; the CCX doubled both the L3 capacity and the L1 load/store bandwidth of the first Zen generation [3, slide 12].

AMD stated the problem and the response in two lists [3, slide 13]. The challenges were a higher cost per mm^2 at the leading-edge node, analog and I/O circuits that "do not scale well" and are hard to port to a new process, and a wide range of markets to serve. The chiplet strategy answered each in turn: small dies to improve yield, analog and I/O on 12 nm to reduce cost and complexity, and a configurable set of parts.

Worked example: how much silicon is a 64-core Rome? AMD describes Rome as about 1000 mm^2 of cumulative die area in 7 nm and 12 nm [3, slide 19]. The die sizes reported by Naffziger *et al.* reproduce this: eight CCDs of 74 mm^2 and a 416 mm^2 server IOD give $8 \times 74 + 416 = 1008$ mm^2 [2]. Of that, 592 mm^2 (59%) is on the expensive 7 nm process and 416 mm^2 (41%) on the cheaper 12 nm. A single die of 1008 mm^2 would be beyond the reticle limit, and even its 592 mm^2 of compute alone would yield poorly (section 6.2).

The partition places latency-critical, logic-dense circuits—cores and caches—on the newest node, and places memory controllers, PCIe and other I/O, which scale poorly, on a cheaper mature node (figure 11).

Naffziger *et al.* describe how this partition was turned into products [2]. The server package places up to eight CCDs around a central IOD, in two-deep columns that were only possible because the organic-package link reaches farther than a silicon interposer would. Each CCD-to-IOD link is routed to the centre of the CCD to shorten the path to its L3. Not every server model populates all eight CCDs, so one package design yields a range of products. The desktop part reuses the same CCDs with a smaller IOD in the existing AM4 socket, and that IOD, packaged on its own, doubles as the platform's chipset. The previous server generation ("Naples") tied its interface clocks together, whereas Rome decouples the fabric, memory and I/O clocks so that each can be set independently, with a latency-optimised mode when fabric and memory clocks are equal [2]. Figure 12 sketches the resulting server package.

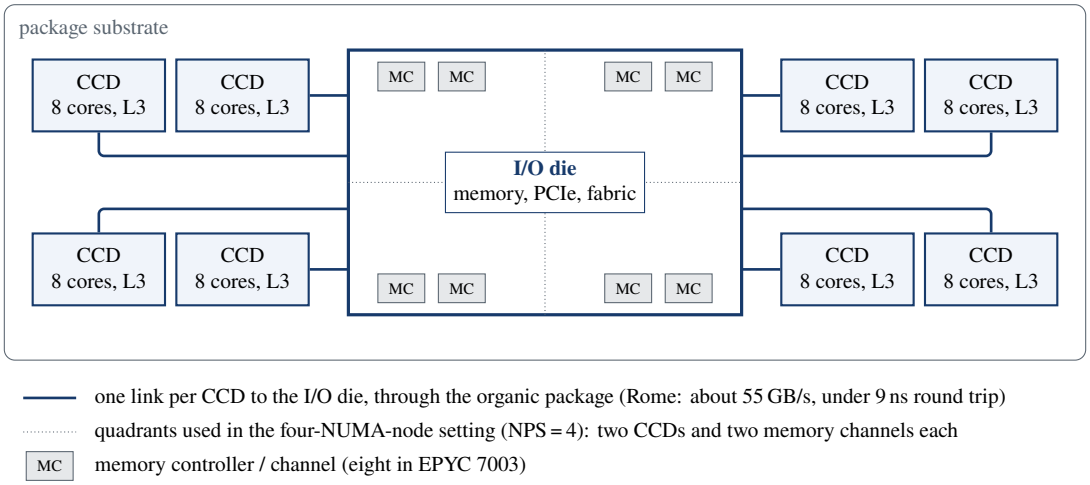


Figure 12: Schematic of an AMD EPYC 7002/7003-style package: eight CCDs in two-deep columns around a central I/O die, each with its own link to the I/O die [2], [11]. Positions, link routes and memory-controller placement are illustrative, not an actual AMD floorplan.

7.1 Inside the I/O die: memory, I/O and NUMA settings

AMD's EPYC 7003 overview describes what the I/O die of the Zen 3 server generation contains and how software sees it [11]. Each CCD connects to the IOD through its own Global Memory Interconnect (GMI) link. The IOD holds eight Unified Memory Controllers, one per DDR4 channel, each supporting up to two DIMMs, for up to 4 TB of memory per socket; eight x16 I/O links that provide up to 128 lanes of PCIe 4.0; and the Infinity Fabric links that join the dies. The fabric clock can run at up to 1600 MHz, which lets it run in step with DDR4-3200 memory, whose memory clock is also 1600 MHz, a pairing AMD says improves memory latency [11]. The IOD can be used with four, six or eight populated memory channels; eight is the most common and generally fastest configuration, and the six-channel option, with its own interleave mode, is new in this generation [11]. AMD's figure labels this IOD as 14 nm [11, Fig. 2-4], while Naffziger *et al.* give 12 nm for the Rome IOD [2]; the sources used here do not resolve the difference.

NUMA settings. Because a core's distance to each memory controller depends on where both

sit on the IOD, EPYC is a non-uniform memory access (NUMA) design, and a BIOS setting called NUMA Nodes Per Socket (NPS) lets the user choose how that structure is presented to software [11]. The setting also decides how memory addresses are spread, or *interleaved*, across channels: consecutive blocks of memory go to different channels in turn, so that a stream of accesses uses several channels at once. Table 11 summarises the options.

Table 11: NUMA settings of an EPYC 7003 processor with eight CCDs and eight memory channels, from AMD's description [11], with peak interleaved bandwidth worked out for DDR4-3200 (25.6 GB/s per channel)

Setting	Each NUMA domain contains	Channels interleaved	Peak bandwidth per domain
NPS1	the whole socket: all 8 CCDs, all channels, all PCIe devices	8	204.8 GB/s
NPS2	half the socket: 4 CCDs and 4 channels	4	102.4 GB/s
NPS4	one quadrant: 2 CCDs, 2 channels and 1 I/O hub	2	51.2 GB/s
LLC as NUMA	one CCD and its L3 (8 domains per socket)	—	—

Worked example: what the setting trades. In every NPS mode the socket's total memory bandwidth is the same, $8 \times 25.6 = 204.8$ GB/s, and so is the share per core when all 64 cores are busy, $204.8/64 = 3.2$ GB/s (table 13). What changes is locality and reach. Under NPS4, a thread whose memory is allocated in its own quadrant reaches the nearest two controllers, the shortest core-to-memory distance on the die, but its interleaved accesses can draw on at most 51.2 GB/s. Under NPS1, the same thread's accesses are spread over all eight channels and can draw on the full 204.8 GB/s, but on average three quarters of them go to controllers outside its own quadrant, including the diagonal quadrant that AMD identifies as the farthest [11]. NPS4 therefore suits many independent, well-placed workloads, and NPS1 suits a single program that needs bandwidth more than latency. The "LLC as NUMA" option goes one step further and tells the operating system which cores share an L3, so that threads that share data can be kept on one CCD and avoid the cross-die coherence traffic of section 5 [11]. The chiplet structure is thus exposed to software as a tuning choice: none of these settings would exist on a single die with uniform access to its memory controllers.

8 Infinity Fabric and Die-to-Die Communication

Splitting a processor into dies turns some on-chip wires into package-level links. AMD connects each CCD to the IOD with an IFOP (Infinity Fabric On-Package) serial link routed over 10–20 mm of organic package [2]. AMD reports, for Rome [2]:

- about 55 GB/s of effective bandwidth per IFOP link at a 1.46 GHz fabric clock;
- a round-trip latency from CCD to IOD and back of 13 fabric clocks, under 9 ns;
- longer reach than a 2.5D silicon interposer, which allowed two-deep columns of chiplets and hence eight CCDs in one package;

- a 19% average reduction in memory latency for the socket compared with the previous generation, attributed to the central IOD, decoupled clocks and the latency-optimised link.

The predecessor "Zeppelin" link achieved about 2 pJ/bit [2]. These are AMD-reported figures for specific configurations and are not independent measurements.

The link is engineered for low latency and power rather than maximum reach. Each IFOP lane runs at 14.6 GT/s with a 1.46 GHz fabric clock; it uses single-ended signalling to save power and package routing layers, forwards a clock alongside each bundle of data lanes to track skew, recovers each lane's timing with a digitally controlled clock-and-data-recovery circuit, and avoids active equalisation in favour of passive T-coils, again to save power [2]. The package itself had to be designed with the dies: the Rome package uses a 9-2-9 layer build-up (four signal-routing layers above the core) against 5-2-5 for Matisse, and the dense arrays of package vias must lie outside the footprints of the CCDs and IOD because of the high density of power vias beneath them [2]. These are the costs of a serial, package-level link that the 3D interface of section 15.2 avoids.

8.1 Latency, bandwidth and interconnect power

The textbook models of Part II show what these figures mean.

Latency. Every L3 miss in a CCD must now travel to the IOD, where the memory controllers sit, and back. In equation (6) this adds the die-to-die round trip (under 9 ns in Rome) to $\text{penalty}_{\text{LLC}}$ for every access that misses in the L3 [2]. The same hop appears whenever data must move between CCDs, for example when two threads on different CCDs share a cache block and the coherence mechanism of section 5 has to transfer it. This is why EPYC 7003 lets software treat each CCD's L3 as its own NUMA domain, and why its NPS settings group cores, memory channels and I/O into quadrants: the closest core-to-memory distance is within a quadrant and the farthest is between diagonal quadrants [11]. On a monolithic die the same traffic would cross on-chip wires instead.

The central IOD also changed how uneven memory latency is across the socket. In the previous generation, Naples, each of four dies had its own memory controllers, so a core saw three different memory distances; Rome's single IOD reduced this to two (table 12).

Table 12: Memory latency by NUMA distance, Naples (EPYC 7001) and Rome (EPYC 7002), as reported by AMD [2, Fig. 2.2.6]

Naples: 8 NUMA domains, 3 distances		**Rome: 2 NUMA domains, 2 distances**	
Domain	Latency (ns)	Domain	Latency (ns)
NUMA1	90	Near 2 channels	94
NUMA2	141	NUMA1	104
NUMA3	234	NUMA2	201
Average local	128	Local average vs Naples	0.81×

The fastest access became slightly slower (94 against 90 ns), because every access now crosses the IFOP link to the IOD, but the average fell by about a fifth because no access has to cross

to another compute die's memory controller inside the socket. This is the trade the chiplet organisation makes: a small, uniform penalty in place of large, uneven ones.

Bandwidth. A single link limits how much memory bandwidth one CCD can draw. As an illustration, eight 64-bit DDR4-3200 channels, the configuration of both Rome and the most common EPYC 7003 systems [3, slide 19], [11], provide $8 \times 3200 \times 10^6 \times 8\,\mathrm{B} = 204.8\,\mathrm{GB/s}$ at peak, the figure AMD gives for Rome [3, slide 19]. The desktop Matisse part has two such channels and 51.2 GB/s [3, slide 15]; with two CCDs and 16 cores that is $51.2/16 = 3.2\,\mathrm{GB/s}$ per core, exactly the share of a fully loaded 64-core Rome ($204.8/64$). The memory system was scaled with the number of compute dies, not the other way round. Naffziger *et al.* give about 55 GB/s of effective bandwidth per Rome CCD link [2], roughly a quarter of that figure. Both numbers now describe Rome, but one is a peak rate and the other an effective rate, so the ratio is only indicative, but it is consistent with AMD's statement that at least four CCDs are populated in most Rome configurations to balance overall bandwidth [2].

Power. Moving a bit across the package costs more energy than moving it across a die, because the link drives longer, more capacitive wires (equation (4)) and needs serialisers and receivers at each end. AMD reports about 2 pJ/bit for the predecessor link and states that the Rome link was designed to minimise power, avoiding active equalisation for that reason [2]. At 55 GB/s, 2 pJ/bit would correspond to roughly $55 \times 10^9 \times 8 \times 2 \times 10^{-12} \approx 0.9\,\mathrm{W}$ per fully loaded link; this is an order-of-magnitude illustration using the older generation's efficiency, not a measured Rome figure.

8.2 Scalability

The chiplet organisation scales by adding compute dies, but the IOD is shared. Adding CCDs increases core count and total L3 capacity, while memory channels and I/O stay fixed by the IOD and the socket. As more cores share the same memory bandwidth, the bandwidth per core falls, which raises the value of a large L3 that keeps traffic off the memory channels. AMD's own Rome rule of thumb, at least four populated CCDs for bandwidth balance [2], shows the opposite limit: with too few CCDs, the per-CCD links rather than the memory channels become the bottleneck. A star of point-to-point links to one IOD also has a fixed number of link ports, so the number of CCDs is bounded by the IOD design as well as by package area.

Worked example: balancing link and memory bandwidth

Naffziger *et al.* give about 55 GB/s of effective bandwidth per Rome IFOP link and state that at least four CCDs are populated in most configurations to balance overall bandwidth [2]. Table 13 shows why, comparing the total link bandwidth with the 205 GB/s peak of eight DDR4-3200 channels computed above. As before, the link figure is effective and the memory figure is peak, so the comparison is indicative only.

With two CCDs the links, not the memory, are the bottleneck; at four the two sides are roughly matched, which is AMD's rule of thumb; at eight, the memory channels limit, and each core's share of memory bandwidth halves. The last column is the point that connects Part III to Part IV: the more cores share a socket's memory channels, the more valuable it is to satisfy requests from a large on-package cache instead.

Table 13: Worked example: total CCD–IOD link bandwidth against peak memory bandwidth (55 GB/s per link [2]; 8 channels of DDR4-3200)

CCDs	Cores	Total link bandwidth	Limiting side	Memory bandwidth per core
2	16	110 GB/s	links	$110/16 \approx 6.9$ GB/s
4	32	220 GB/s	roughly balanced	$205/32 \approx 6.4$ GB/s
8	64	440 GB/s	memory channels	$205/64 \approx 3.2$ GB/s

Scaling beyond one socket

The same links extend to two sockets. Two identical EPYC 7003 processors connect through their external Global Memory Interconnect (xGMI) Infinity Fabric links, which use the same physical connections as the PCIe lanes [11]. Each link uses 16 lanes, so a socket that devotes four links to the other socket gives up 64 of its 128 lanes, leaving $2 \times 64 = 128$ lanes for the whole system; a designer who uses three links instead frees $128 - 48 = 80$ lanes per socket, 160 in total, at the cost of inter-socket bandwidth [11]. A dual-socket system has sixteen memory channels and can be configured with between 1 and 16 NUMA domains. The trade between I/O lanes and inter-processor bandwidth is the package-level version of the trade between link width and die area seen within the package (section 8.3).

8.3 How the dies are connected: organic substrate, 2.5D and 3D

Chiplets can be joined in three broad ways, which differ mainly in how finely their connections can be spaced (table 14).

- **On the organic package substrate** (a multi-chip module, sometimes called 2D packaging). Each die is mounted on the substrate with C4 solder bumps, and the substrate's copper traces connect the dies. This is the cheapest option and allows long links, but the connections are coarse.
- **2.5D integration.** The dies still sit side by side, but they are joined through a finer layer of wiring placed under or between them: a silicon interposer, a small embedded silicon bridge, or a fan-out redistribution layer. Connections are much denser, so a link of the same width carries more bandwidth with less energy per bit, but the extra silicon adds cost and limits how far apart the dies can be.
- **3D integration.** One die is stacked directly on another and connected through TSVs and micro-bumps or hybrid bonds (Part IV). This gives the densest and shortest connections.

AMD's choice for its chiplet CPUs was deliberate. Naffziger *et al.* contrast the organic-package SerDes link with a 2.5D interposer and with Intel's embedded bridge (EMIB), and state that its key benefit is longer reach, which allowed chiplets to be arranged in columns two deep; without that, eight CCDs would not have fitted around one IOD [2]. Rome and Matisse therefore connect their CCDs to the IOD through the organic substrate, and the later Zen CPUs studied in this paper use 3D stacking for the extra cache; the sources used here do not describe a change to the CCD–IOD connection in those later generations (section 8, Part IV).

2.5D integration is nevertheless part of AMD's wider packaging history (figure 13). Its own

Table 14: Die-to-die connection methods, with AMD's interconnect pitch targets [22, slide 13] and AMD examples

Method	Pitch	AMD examples	Main trade-off
Organic substrate (C4 bumps)	< 150 µm	EPYC 7002 and Ryzen 3000 CCD–IOD links [2]	long reach and low cost; coarse links
2.5D (interposer, bridge or fan-out)	< 40 µm	Fiji GPU with HBM on a silicon interposer (2015) [5, slide 9]; Ryzen AI Max PRO 300 [22, slide 8]	denser, lower-energy links; added silicon and shorter reach
3D (hybrid bonding)	< 9 µm	3D V-Cache [6]	densest, shortest links; heat and test complexity

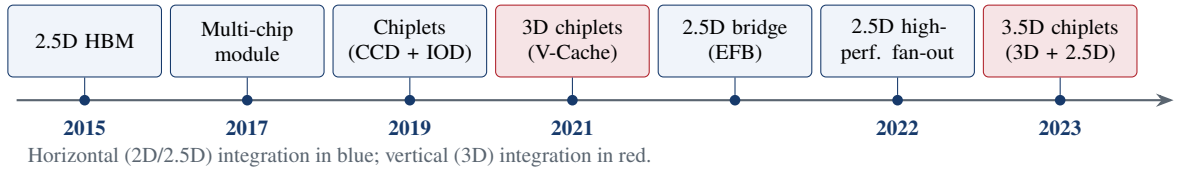


Figure 13: AMD's packaging steps as presented by AMD [22, slide 5]. The slide gives no separate year for the 2.5D bridge (EFB). The CPUs studied in this paper use the 2019 chiplet organisation and, from 2021, 3D-stacked cache.

timeline runs from 2.5D HBM (2015) through the multi-chip module (2017), chiplets (2019) and 3D chiplets (2021) to a 2.5D bridge (EFB), 2.5D high-performance fan-out (2022) and "3.5D" chiplets (2023) [22, slide 5]. The Instinct MI300 accelerator combines both: dies hybrid-bonded on top of I/O dies, which sit side by side on a 2.5D silicon interposer [22, slide 10]. Among Zen-based processors, the Ryzen AI Max PRO 300 series uses 2.5D fan-out packaging, which AMD describes as a "continued strategic use of 2.5D advanced packaging to reduce die-to-die power and latency" [22, slide 8]. Swaminathan names the parameters that decide between these options: cost and volume, *bandwidth density* (the area spent on die-to-die communication) and *energy per bit* [5, slide 7]. The organic substrate wins on cost and reach, 2.5D on bandwidth density and energy, and 3D on both of the latter at the price of the thermal and manufacturing issues of section 16.

9 Yield, Process-Node Specialisation and Cost

Chiplets combine two cost advantages. First, small dies yield better (section 6.2). Second, each die can use the process that suits it: AMD keeps memory interfaces and SerDes on 12 nm because they would see little benefit at 7 nm [2]. AMD also tests every CCD before assembly and can down-bin a CCD damaged during assembly, placing it in a known low-power state [2], which limits the cost of combining several dies in one package.

These benefits are not free. AMD describes the design as requiring package–silicon co-design, new interconnect circuits, a mixed-process package (with copper pillars on the 12 nm IOD to match die heights), and new test and power-management infrastructure [2].

The model of appendix C gives a rough sense of the silicon saving. Consider eight Rome-sized CCDs of 74 mm^2 each, 592 mm^2 in total, against a hypothetical monolithic die with the same compute area (ignoring the I/O, which in the chiplet design moves to a cheaper node). With $D_0 = 0.2\,cm^{-2}$ and the yield model of equation (8), a 74 mm^2 CCD yields about 87% and the 592 mm^2 die about 38%, so eight tested CCDs cost about 44% of the silicon of one good monolithic die; with $D_0 = 0.1\,cm^{-2}$ the figure is about 63%. These numbers are illustrative only: they ignore interface area, packaging, assembly yield and test cost, and Dong and Xie find an analogous effect for 3D stacking, where the extra bonding and test costs outweigh the yield gain for small designs, so that stacking pays only above a size threshold [20]. They also ignore the separate saving from building the I/O on a mature node [2]. The qualitative conclusion is robust, however: for a design of server size, the yield advantage of small dies is large.

9.1 Worked example: good dies per wafer

Yield is only half of the cost story; the other half is how many dies fit on a wafer. Dong and Xie estimate the number of dies per wafer as

$$N_{\text{die}} = \frac{\pi\,(\phi_{\text{wafer}}/2)^2}{A_{\text{die}}} - \frac{\pi\,\phi_{\text{wafer}}}{\sqrt{2\,A_{\text{die}}}}, \tag{9}$$

where ϕ_{wafer} is the wafer diameter and A_{die} the die area [20, Eq. 14]. The first term divides the wafer area by the die area; the second subtracts the partial dies lost around the circular edge, a loss that grows with die size. For a 300 mm wafer, table 15 applies equation (9) and the yield model of equation (8).

Table 15: Worked example: dies and good dies per 300 mm wafer ($D_0 = 0.2\,cm^{-2}$, $Y_{\text{wafer}} = 1$; illustrative)

Die	Dies per wafer	Modelled yield	Good dies	64-core sets per wafer
74 mm^2 CCD	$955.2 - 77.5 \approx 878$	86.6%	760	$760/8 = 95$
592 mm^2 monolithic	$119.4 - 27.4 \approx 92$	38.3%	35	35

A wafer of CCDs supplies the compute silicon for about 95 64-core processors, against 35 for the monolithic design, a factor of 2.7. With $D_0 = 0.1$ the factor is about 1.9, and with $D_0 = 0.3$ about 3.5. The edge loss alone is small for the CCD (8% of the gross count) but noticeable for the large die (23%). This calculation still ignores the I/O die, packaging and test, and assumes that a 592 mm^2 die could be made at all; at the full 64-core size, Naffziger *et al.* note that the monolithic design would exceed the maximum reticle size [2].

AMD's own estimate points the same way (figure 14). Naffziger *et al.* compare the cost of Rome chiplet products with hypothetical monolithic 7 nm designs of the same core count [2, Fig. 2.2.2]. Read from the chart, the monolithic designs cost roughly twice as much at 48 and 32 cores and about 1.8× at 16 cores, and the chiplet design scales its cost down with core count by leaving CCD positions empty ("depopulation") and using partly defective dies ("harvesting"). Two details of the chart should be noted: its legend describes the chiplet design as "7 nm + 14 nm", whereas the paper's text gives 12 nm for the I/O die, and the chart's axis is a normalised cost without units. The chart is AMD's model, not a disclosed cost, but its factor of about two agrees with the order of magnitude of table 15.

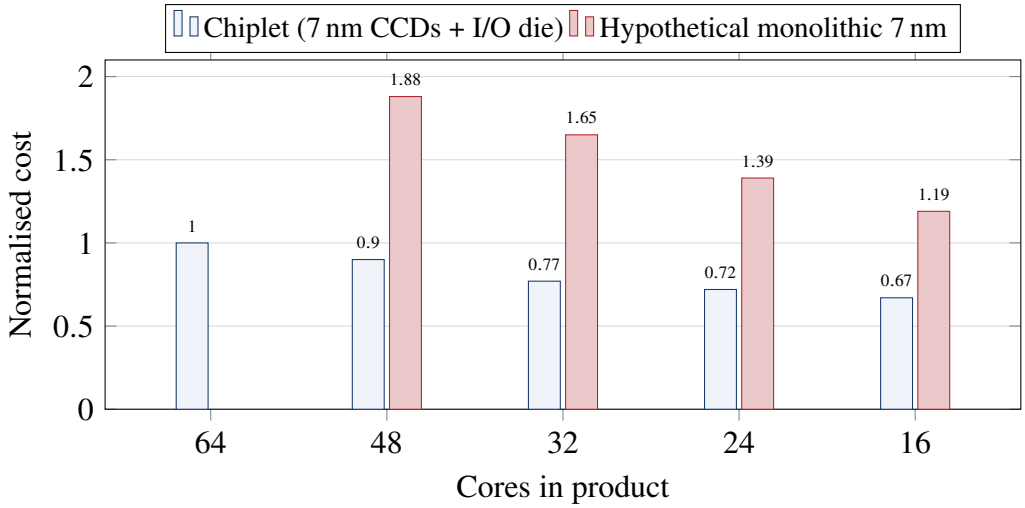


Figure 14: Cost against core count for Rome-generation chiplet products and hypothetical monolithic 7 nm equivalents, normalised to the 64-core chiplet part. Values were read by eye from AMD's chart [2, Fig. 2.2.2] and are accurate only to about ±0.05. AMD shows no monolithic 64-core bar; the text states that such a die would exceed the maximum reticle size.

10 Summary of Part III

Applying the paper's three-part test to chiplets:

- **Bottleneck addressed.** Die size beyond the reticle limit, falling yield with area, and the poor scaling of analog and I/O circuits on advanced nodes (section 6).
- **Benefit.** Core counts that would be impractical on one die, much better silicon yield, I/O on a cheaper node, and one compute die reused across server and desktop products [2], [3].
- **New trade-offs.** An extra die-to-die hop on every L3 miss and on cross-CCD sharing, per-link bandwidth limits, interconnect power, NUMA effects that software may need to manage, and more complex package design and test.

One limit is untouched: each CCD still carries its own planar L3. Growing that cache runs into the same area, yield and wire-length costs at the scale of the compute die, which is the problem Part IV takes up.

Part IV

3D-Stacked Cache and AMD 3D V-Cache

Target length: about 25 pages. Draft status: drafted from AMD ISSCC 2022 slides and Mittal & Vetter; independent latency, thermal and workload evidence added.

11 Why Cache Capacity Matters

Sections 4.3 and 4.4 showed that a larger LLC lowers the miss rate by removing capacity misses, provided its hit time does not grow too much. Whether that matters depends on the workload: a program whose working set already fits in the cache gains little, while one whose working set lies just above the cache size can gain a great deal.

AMD's ISSCC 2022 presentation of 3D V-Cache opens with two charts that frame this part [6, slides 5–6]. The first, titled "Large L3 Caches Provide IPC Uplift", plots IPC uplift on a linear scale rising steadily as L3 size grows from 1× to 32×; the slide gives no values on the uplift axis and does not name the workloads. The second, "Silicon Area Scaling by Function", shows logic area continuing to shrink from 28 nm to 5 nm while SRAM scaling falls behind from about 7 nm and analog circuits shrink least; a companion chart on the same slide shows normalised cost per yielded mm^2 (for a 250 mm^2 die) rising from 45 nm to 5 nm. The slide summarises these as "SRAM scaling & cost = barriers to large on-die caches". This extends Naffziger *et al.*'s 2020 observation that analog circuits scale worst [2]: by the 5 nm generation SRAM, too, has become comparatively expensive.

Both charts are vendor material, and the first has no stated workloads or values; neither is evidence that every program benefits. Part V tests the claim against published measurements.

The textbook's miss classes explain why the benefit of a larger cache is uneven. Compulsory misses occur on the first access to a block and no cache size removes them; conflict misses depend on associativity; capacity misses occur because the cache cannot hold everything the program is actively using [1, sec. 11.3.2]. Only capacity misses fall as the cache grows, and they fall sharply only when the cache becomes large enough to hold a program's *working set*, the data it reuses over a period of time. Figure 15 sketches the consequence. A program whose working set fits in 32 MB sees almost no change from a 96 MB cache; one whose working set is far larger than 96 MB also sees little change, because most of its data still does not fit; only programs whose working sets lie between the two sizes move from frequent misses to frequent hits. The useful range is set by the ratio of the two capacities, three times, rather than by the extra 64 MB alone.

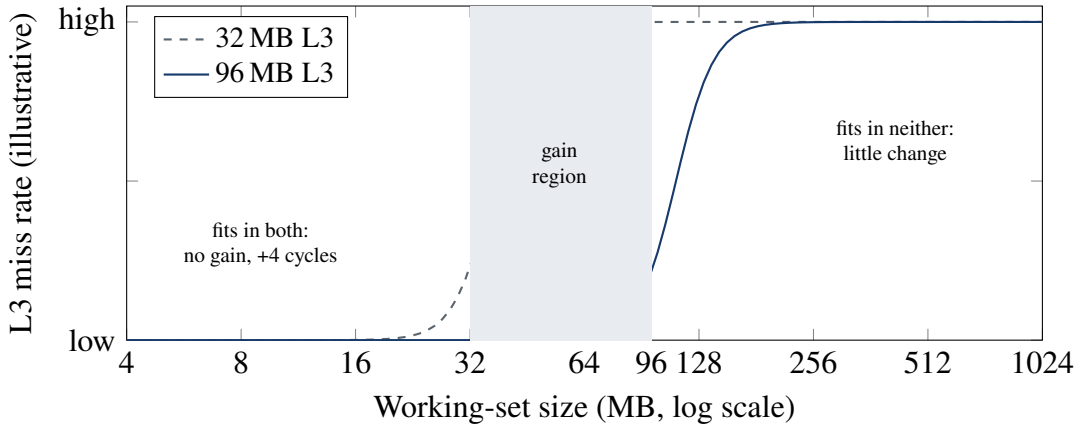


Figure 15: Conceptual shape of the L3 miss rate against working-set size for a workload with reuse, for a 32 MB and a 96 MB L3. The curves are illustrative, not measured; real transitions are smoother and depend on access pattern, associativity and sharing. They show the three workload classes of section 17.

12 Why Expanding Cache in Two Dimensions Became Difficult

Growing the L3 in the plane of the compute die meets four related obstacles.

12.1 Area and yield

SRAM cells no longer shrink at the rate of logic (section 11), so a cache of a given capacity occupies a growing share of the die at each new node. AMD lists cost, product flexibility and latency as the resulting barriers to large on-die caches [6, slide 6]. Mittal and Vetter add that, if the large caches needed by future workloads were built from SRAM, they could occupy up to 90% of the chip area [16, sec. 1]. A larger die yields fewer good dies per wafer (section 6.2), so doubling the L3 on a planar CCD would raise the cost of every CCD, including those sold for workloads that do not need the extra cache. It would also make the CCD less flexible as a building block reused across products.

12.2 Wire length and access latency

A larger planar cache spreads over a larger area, so the average distance between a core and the cache bank holding its data increases. Longer wires have more resistance and capacitance; their RC delay increases access latency, and the energy needed to charge their capacitance increases dynamic power (equation (4)). Inside each array the same effect appears at smaller scale: more cells per bit line mean more capacitance to drive, which is why large caches are divided into many small arrays and banks (section 4.2).

The effect is measurable even within today's L3. Zen 3 divides its L3 into eight slices, and measurements on a Ryzen 5 5600G show that a core reaches its nearest slice in 40 cycles but needs up to 20 cycles more, 60 in total, for the farthest one (figure 16) [13]. Because consecutive lines are spread across the slices, the average over all eight is what a program sees: $(0 + 6 + 1 + 10 + 6 + 13 + 10 + 20)/8 \approx 8$ extra cycles for core 0, or about 48 cycles in total, which matches the 47-cycle L3 latency measured on the same processor [13]. Distance on the

die is therefore already a visible part of the L3's hit time; a planar L3 three times larger would spread its slices over a larger area and lengthen the farthest paths further.



Figure 16: Measured extra latency, in cycles beyond the 40-cycle minimum, for a Zen 3 core to reach each of the eight slices of its L3, on a Ryzen 5 5600G [13]. The arrangement of slices follows the source's own table; it is a logical layout, not a floorplan.

In terms of equation (6), part of the miss-rate gain from a bigger cache is paid back as a longer hit time t_{LLC}. AMD makes the same argument for stacking: compared with spreading a large cache across the plane, 3D integration "improves effective memory latency" and "reduces long datapath and I/O's dynamic powers" [6, slide 7].

12.3 Power

Large SRAM caches consume dynamic power on every read and write and leakage power even when idle (section 3.2). Mittal and Vetter's survey of DRAM-cache architectures argues that the low density and high leakage power of SRAM make very large SRAM caches infeasible, and that such caches may need expensive cooling, such as liquid cooling, which adds substantial cost and complexity [16, sec. 1]. This is one reason researchers have explored denser technologies, such as stacked DRAM, for last-level caches.

12.4 Summary of the planar limits

Planar expansion therefore trades miss rate against die cost, hit latency and power all at once. The question for the next section is whether a separate cache die can break that coupling.

13 Side-by-Side Versus Vertically Stacked Cache Dies

If extra cache is to be made on its own die, it can be placed either *beside* the compute die in the package or *on top of* it (figure 17). Both are valid chiplet designs; the choice is a trade-off rather than a matter of one being impossible.

13.1 Placing the cache die beside the compute die

A side-by-side cache die is a perfectly workable chiplet design. It could be connected through the organic substrate or, more densely, through 2.5D packaging (section 8.3), but either way it

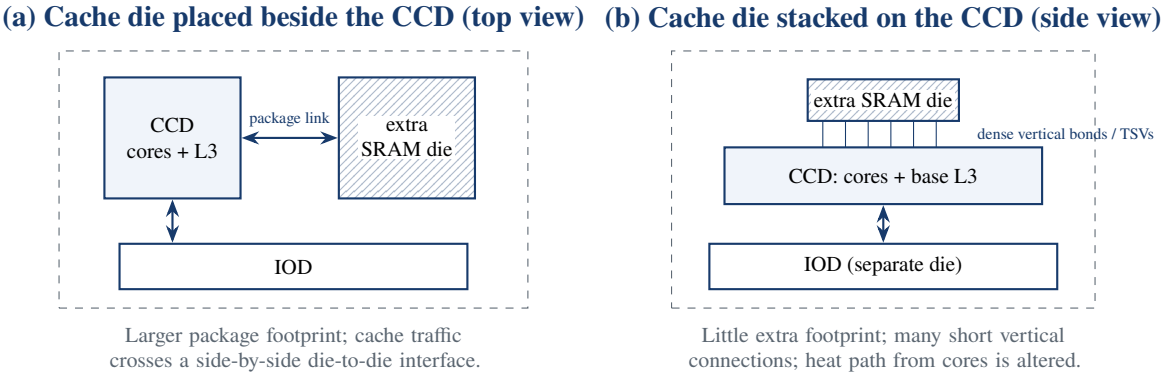


Figure 17: Two ways to add cache capacity to a compute die. Illustrative arrangements only; not actual AMD package layouts and not to scale.

has three costs:

- **Package footprint.** The extra die occupies additional area next to the CCD. This enlarges the *package*, even though it does not enlarge the compute die itself. The distinction between *die area* (one piece of silicon) and *package footprint* (the area of the assembled processor) is important throughout this part.
- **Interconnect.** Every access to the extra cache must cross a die-to-die interface through the package or an interposer. Package-level connections are far coarser than on-die wiring, which limits how many parallel connections—and hence how much bandwidth—the link can offer.
- **Latency and power.** Data must travel between the dies, adding interface circuitry and wire delay to every access to the extra cache, and energy to every bit moved.

A side-by-side die does not automatically push the CCD farther from the IOD: the package designer chooses where each die goes, and the cache die could be placed on the far side of the CCD. However, package area is limited, the CCD–IOD links are already routed with care to minimise memory latency [2], and every additional die competes for the same routing resources. At high signalling rates, longer and more numerous package routes bring RC delay, power, signal-integrity and routing-congestion problems. These can be mitigated with better interconnect design, buffering and packaging technology, so they are engineering costs rather than impossibilities.

The more important comparison is therefore not the CCD-to-IOD distance but the latency and bandwidth of the path *between the cores and the additional cache*.

AMD's own framing of the choice matches this analysis. Its ISSCC 2022 presentation illustrates a hypothetical processor whose large cache is split off onto a side-by-side chiplet, and states that "2.5D chiplets can provide product flexibility and reduce cost" but that "3D can be even better", because it improves effective memory latency, reduces long-datapath and I/O dynamic power, and "fits more transistors within a given package cavity size" [6, slide 7].

13.2 Stacking the cache die on the compute die

Vertical stacking changes the trade-off:

- **Footprint.** The cache die overlaps the CCD's existing footprint, so capacity grows with little additional package area.
- **Interconnect density.** The two dies can be joined by a very large number of short vertical connections, which supports the bandwidth a large cache needs and keeps the added distance small.
- **Thermal design.** The stack can obstruct heat flow from the compute die, depending on which die is on top and what lies above the cores. This is the central new engineering problem (section 16).

Table 16: Qualitative comparison of side-by-side and vertically stacked cache dies

Factor	Cache die beside the CCD	Cache die stacked on the CCD
Package footprint	Larger	More compact
Connections	Side-by-side package or interposer link	Dense vertical bonds and through-silicon vias
Cache access latency	Depends on the interface design	Can benefit from short, dense connections
Cooling	Easier to arrange	More difficult; depends on stack order
Manufacturing	Extra die plus package assembly	Extra die plus bonding, thinning and stacking

Table 16 summarises the comparison. The case for vertical stacking is not that side-by-side chiplets cannot work. It is that stacking can add substantial cache capacity without enlarging the package by the same amount, with potentially better interconnect characteristics. The question worth investigating is *when the extra complexity of 3D integration is worth the smaller footprint and denser interconnect*.

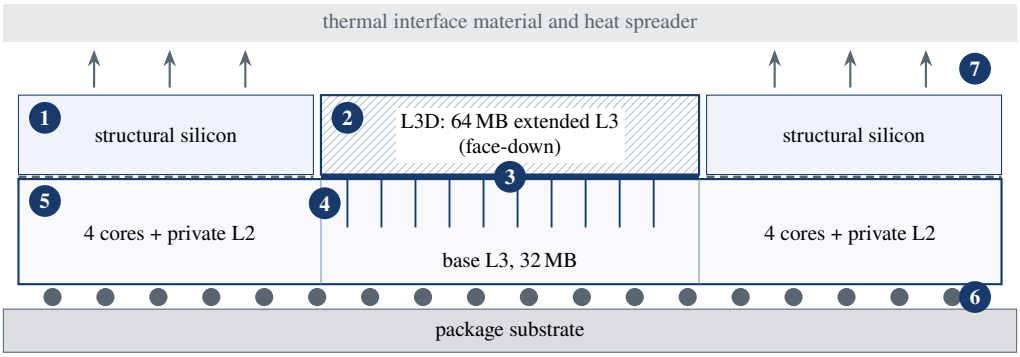
14 Stacked-Die Integration and Hybrid Bonding

14.1 The first-generation 3D V-Cache stack

AMD's Hot Chips 33 presentation describes the first-generation stack as a 64 MB L3 cache die attached to an up-to-eight-core Zen 3 CCD by a direct copper-to-copper bond, with TSV (through-silicon via)s carrying signals and power through the CCD [5]. AMD's EPYC 7003 documentation states that the L3 cache tiles are stacked on the central L3 portion of the base CCX [11]. The ISSCC 2022 presentation describes the three components of the stack and how they are assembled [6, slides 10–12, 14, 16–17]:

- the Zen 3 CCD, built on TSMC's 7 nm process with eight cores sharing 32 MB of L3 in an area of 81 mm^2, with V-Cache support designed in "from Day 1";
- the extended L3 die (L3D), also 7 nm, carrying 64 MB of L3 in 41 mm^2, roughly half the CCD area;
- structural silicon dies, which provide mechanical support for the thinned CCD and a path for heat from the CPU cores.

The CCD is mounted face-down on the package substrate through conventional C4 bumps and connects to the L3D through TSVs. The L3D is mounted face-down on the CCD by hybrid bonding, using TSMC's SoIC process with a minimum TSV pitch of 9 µm, while the structural dies are oxide-bonded beside it. Support silicon brings the stack to the same height as an unstacked CCD, so server parts with and without V-Cache fit the same package. The cache die sits over the CCD's central L3 region [11] and the structural silicon over the cores, so the hottest circuits are covered only by passive silicon. Figure 18 shows the arrangement in cross-section.



1 Structural silicon dies, oxide-bonded to the CCD: support the thinned CCD and conduct heat from the cores.

2 L3D: 7 nm, 41 mm^2, 64 MB of L3 over the CCD's central L3 region.

3 Hybrid bond: direct copper-to-copper connection (TSMC SoIC) between CCD and L3D.

4 Through-silicon vias in the thinned CCD; minimum 9 µm pitch.

5 CCD: Zen 3, 7 nm, 81 mm^2, eight cores; the shared L3 grows from 32 to 96 MB.

6 C4 bumps connect the face-down CCD to the substrate.

7 Heat from the cores rises through the structural silicon to the heat spreader.

Figure 18: Schematic cross-section of the first-generation 3D V-Cache stack, drawn from AMD's descriptions [6, slides 10–12, 14, 16–17] and [11]. Not to scale; layer thicknesses and the number of TSVs are illustrative.

14.2 Hybrid bonding versus micro-bumps

Earlier 3D stacking typically joined dies with solder micro-bumps. Hybrid bonding instead bonds aligned copper pads and surrounding dielectric directly. AMD reports, from its internal engineering analysis [5]:

- a hybrid-bond pitch of 9 µm, compared with 50 to 36 µm for micro-bump 3D;
- more than 15× the interconnect density of micro-bump 3D;
- more than 3× better interconnect power efficiency than micro-bump 3D;
- lower TSV capacitance and inductance, which benefits signal and power delivery.

AMD's EPYC 7003 documentation similarly claims over 200× the interconnect density of 2D packaging and over 15× that of solder-bump 3D [11]. These are AMD's own comparisons and should be read as engineering claims.

Interconnect density matters architecturally for three reasons. First, *bandwidth*: the number of parallel connections between two dies limits how much data can cross between them per cycle, and a large L3 serving eight cores needs very wide paths; AMD's stack delivers more

than 2 TB/s [6, slide 20]. Second, *latency*: vertical connections a few micrometres long add very little wire delay, which helps explain why the extra 64 MB costs only four cycles. Third, *energy*: short, low-capacitance connections cost less energy per bit (equation (4)), which AMD quantifies as more than 3× better than micro-bump stacking [6, slide 15]. The low parasitics of hybrid bonding also let AMD use a simple digital signal interface between the dies rather than the serialising transceivers needed by package-level chiplet links [6, slide 26]. These are exactly the costs that section 13 identified for a side-by-side cache die.

14.3 Worked example: what a finer pitch buys

Pitch is the distance between the centres of neighbouring connections. If pads are laid out on a square grid of pitch p, a square millimetre holds

$$n = \left(\frac{1000\,\mu\text{m}}{p}\right)^2 \tag{10}$$

connections. At the 36 µm micro-bump pitch quoted by AMD this gives $(1000/36)^2 \approx 772$ connections per mm^2; at the 9 µm hybrid-bond pitch it gives $(1000/9)^2 \approx 12\,346$, sixteen times as many, which is consistent with AMD's claim of more than 15× the density [5]. Against C4 bumps at AMD's 150 µm limit [22, slide 13], about 44 per mm^2, the hybrid bond offers roughly 280 times as many, the same order as AMD's "over 200×" comparison with 2D packaging [11]. Because density grows with the square of the pitch reduction, each halving of the pitch gives four times as many connections (figure 19).

The architectural meaning becomes clear from the area involved. Suppose, for illustration only, that a 1 mm^2 strip of the CCD were given over entirely to signal connections. At the hybrid-bond pitch it could carry about twelve thousand signals; a 32-byte (256-bit) interface for each of eight cores in both directions needs $8 \times 256 \times 2 = 4096$ data wires, a third of that budget before clocks, control and spares. At the micro-bump pitch the same interface would need more than 5 mm^2, and at the C4 pitch more than 90 mm^2, larger than the whole CCD. This is a model calculation, not AMD's layout, but it shows why a wide, parallel, cache-speed interface between two dies is practical only with 3D bonding, and why package-level chiplet links must instead serialise data over a few fast lanes (section 8).

14.4 How a hybrid bond is made

Hybrid bonding joins two surfaces that each contain both dielectric and copper. AMD's packaging engineers describe the dielectric part as *fusion bonding* [8, slide 9]. The oxide surfaces are prepared so that they end in silanol (Si–OH) groups. When the two dies are brought together at room temperature, these groups hold the surfaces together with weak hydrogen bonds. Annealing then converts them into strong siloxane bonds,

$$\text{Si–OH} \;+\; \text{HO–Si} \;\longrightarrow\; \text{Si–O–Si} \;+\; \text{H}_2\text{O},$$

releasing water vapour [8, slide 9]. In hybrid bonding the copper pads start slightly recessed ("dished") below the oxide, so that the oxide surfaces bond first; during the anneal the copper diffuses across the interface and joins the pads [8, slide 9]. No solder is involved, which is why

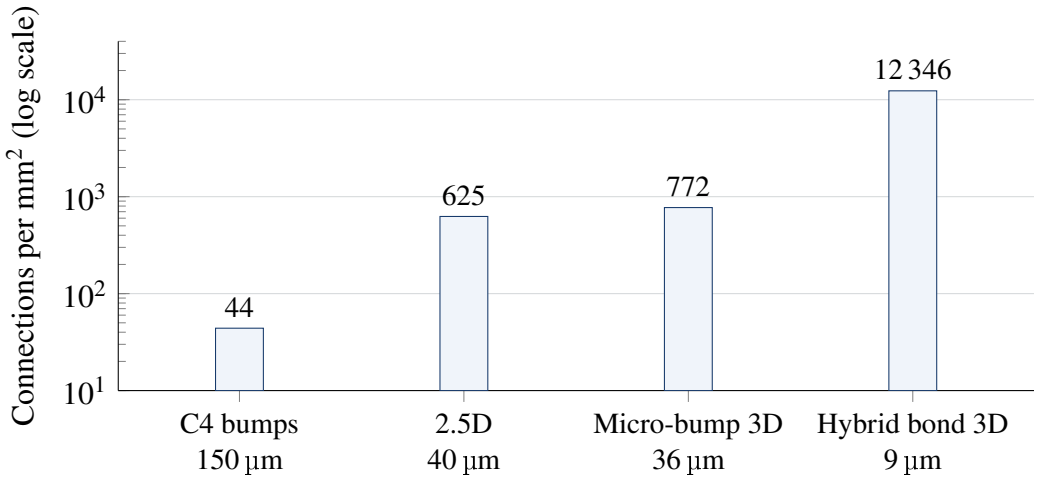


Figure 19: Worked example: connections per square millimetre for a square grid of pads at each pitch, $(1000/p)^2$ for a pitch p in micrometres. Pitches are AMD's stated limits for C4 bumps and 2.5D [22, slide 13], the finest micro-bump pitch AMD quotes, and the 9 µm hybrid-bond pitch [5]. Real layouts are not full square grids, so the values are upper bounds for comparison only.

the pitch can be so much finer than with micro-bumps.

Figure 20 shows one connection. In the first-generation stack, the copper pads on each die are called bond-pad metal (BPM). On the CCD, the BPM sits on the end of a TSV; on the L3D, a bond-pad via (BPV) connects the BPM to the die's top copper layer, M13 [6, slide 16]. The L3D has thirteen copper layers and one aluminium layer [6, slide 11]. For Zen 4, AMD changed this detail: the stack became explicitly face-to-back, with a 7 nm cache die on a 5 nm CCD, and the BPV lands on the top aluminium pad instead of the top copper layer [8, slides 11–12]. The process is TSMC's SoIC (System on Integrated Chips) [6, slide 16].

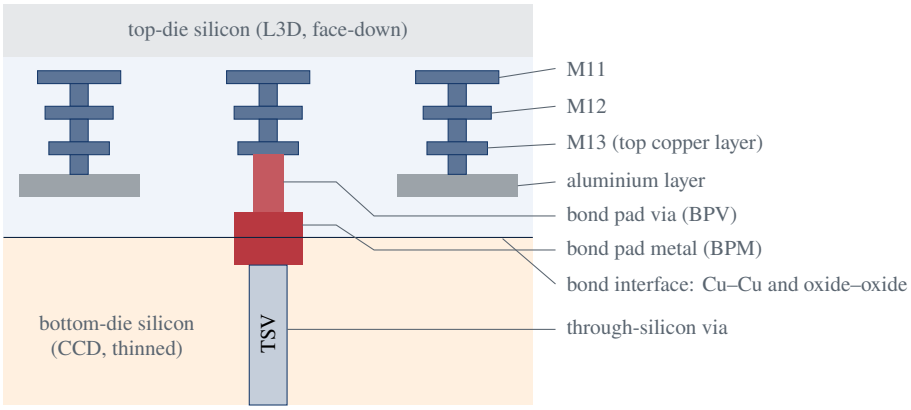


Figure 20: Schematic of one hybrid-bond connection in the first-generation stack, redrawn from AMD's description [6, slide 16]. Copper bond-pad metal (BPM) on the two dies is bonded directly; on the top die a bond-pad via (BPV) connects the pad to the top copper layer (M13), and on the bottom die the pad lands on a TSV. In the Zen 4 stack the BPV lands on the top aluminium pad instead [8, slides 11–12]. Not to scale.

15 Cache Access, Latency, Bandwidth and Hierarchy

With the stacked die, the L3 of a Zen 3 CCD grows from 32 MB to 96 MB, shared by up to eight cores [6], [11]; an EPYC 7003 processor with eight such CCDs carries up to 768 MB of L3 [11]. AMD reports up to 2 TB/s of bandwidth for the V-Cache [6]. The extra capacity remains part of the same L3 level, so the cache hierarchy above it (private L1 and L2 per core) is unchanged [11].

AMD's presentation gives the cost of the extra capacity in latency: the 96 MB L3 remains 16-way set associative with a 32 B/cycle interface to each core, delivers more than 2 TB/s of L3 bandwidth, and adds *four cycles* of latency [6, slide 20]. Independent measurements agree: Chips and Cheese measured roughly three to four extra cycles of L3 latency on Milan-X compared with Milan [19], about four extra cycles on the V-Cache CCD of the Zen 4 Ryzen 9 7950X3D [18], and the same four cycles on the Zen 5 Ryzen 7 9800X3D, whose cache die sits below the CCD [23]. AMD's bandwidth figure is a peak capability: on Milan-X, single-thread L3 bandwidth measured about 25% more bytes per cycle than Milan rather than the doubling AMD had announced [19]. Each die's portion of the L3 has its own data, tag and LRU arrays, so a tag lookup does not have to cross between dies before the data array is accessed [6, slide 20].

Equation (6) shows how to judge this trade-off. For an access that has already missed in L2, the extra capacity changes the average cost by approximately

$$\Delta T \;\approx\; \Delta t_{\mathrm{L3}} \;-\; \Delta m_{\mathrm{L3}} \cdot t_{\mathrm{mem}}, \tag{11}$$

where $\Delta t_{\mathrm{L3}} = 4$ cycles is the added hit latency, Δm_{L3} the reduction in the L3 miss rate and t_{mem} the main-memory latency in cycles. The textbook puts the stall for a miss that goes to main memory at 100–300 cycles [1, sec. 12.4.8]. Over that range, the larger L3 pays off once it removes more than about $4/300 \approx 1.3$ to $4/100 = 4$ percentage points of L3 misses. A workload whose working set already fits in 32 MB gains nothing and pays the four cycles on every L3 hit; a workload whose working set lies between 32 MB and 96 MB can gain substantially.

15.1 Worked example: applying the trade-off to a measured workload

Lam's fixed-frequency measurements of the Ryzen 9 7950X3D give enough data to apply equation (11) to a real program [18]. For libx264 video encoding, the plain CCD had 1.48 L3 misses per thousand instructions (MPKI); Lam's charts give an L3 hit rate of 61.55% and an IPC of 1.36 on the plain CCD, against 71.86% and 1.43 on the V-Cache CCD. (The article's text gives the second hit rate as 72.8%, but its chart and its stated 16.75% improvement both correspond to 71.86%, which is used here.)

1. **L3 accesses.** If 38.45% of L3 accesses miss and this gives 1.48 misses per thousand instructions, the program makes $1.48/0.3845 \approx 3.85$ L3 accesses per thousand instructions.

2. **Misses removed.** With V-Cache, $3.85 \times 0.2814 \approx 1.08$ of those accesses still miss, so about $1.48 - 1.08 = 0.40$ misses per thousand instructions are removed.

3. **Cycles saved.** At the textbook's 100–300 cycles per memory access [1, sec. 12.4.8], that saves $0.40 \times (100 \text{ to } 300) \approx 40$ to 120 cycles per thousand instructions.

4. **Cycles added.** Every L3 access pays four more cycles: $3.85 \times 4 \approx 15$ cycles per thousand instructions.

5. **Net effect.** About 25 to 105 cycles saved per thousand instructions.

At an IPC of 1.36, a thousand instructions take $1000/1.36 \approx 735$ cycles, so the model predicts 3–14% fewer cycles. The measurement lies inside that range: at an IPC of 1.43 the same work takes about 699 cycles, 4.9% fewer.

The measurement can also be turned around. The observed saving of about 36 cycles per thousand instructions, plus the 15 cycles of added L3 latency, implies that each removed miss saved about $(36 + 15)/0.40 \approx 130$ cycles. That is at the low end of the textbook's 100–300-cycle range, which is expected: an out-of-order core overlaps some memory misses with other work (section 3), so removing a miss saves less than its full latency. The worked example therefore uses no assumed values except the textbook's range for memory latency, and the measurement narrows even that.

The same arithmetic explains the small loss Lam observed in a flight simulator with only 0.35 MPKI, falling to 0.28 with V-Cache [18]. Removing 0.07 misses per thousand instructions saves at most 7–21 cycles, which can be outweighed by four extra cycles on each of its L3 hits. The model, the measurement and the miss-class argument of section 11 agree.

15.2 Inside the cache interface

AMD's ISSCC presentation describes how the stacked half of the L3 is organised and connected, and the details explain why the latency cost is so small (figure 21).

Where the connections are. On the CCD, two columns of signal TSVs run along either side of the central 32 MB L3, between the L3 and the two rows of four cores [6, slide 10]. The L3D has matching "CCD signal interface" columns directly above them [6, slide 11]. The L3 already sits at the centre of the CCD, with every core's 32 B/cycle interface entering it from the side [6, slides 19, 21], so the stacked cache is reached through the same central region rather than through a new path across the die. AMD states that support for V-Cache was integrated in the CCD "from Day 1" [6, slide 10]: the TSV columns occupy CCD area in every Zen 3 CCD, including those sold without a cache die.

How the stacked arrays are organised. The L3D contains 512 data macros of 128 KB each and 1088 tag and LRU macros of 6 KB each [6, slide 25]. The first figure checks: $512 \times 128\,\text{KB} = 64\,\text{MB}$. The arrays use a high-density SRAM bit cell and two supply rails, one for the bit cells and one for the peripheral circuits, with "extensive power reduction features" [6, slide 25]. The L3 is protected by DECTED (double-error-correcting, triple-error-detecting) ECC [6, slide 19].

Worked example: the cost of tags. The tag and LRU macros hold $1088 \times 6\,\text{KB} = 6528\,\text{KB}$, about 10% of the 64 MB of data. With the 64-byte lines used throughout Zen 3 [11], the L3D holds $64\,\text{MB}/64\,\text{B} = 1\,048\,576$ lines, so the tag, state and replacement storage averages $6528 \times 1024 \times 8/1\,048\,576 \approx 51$ bits per line. The address split of section 4.5 shows where those bits go. A 64 MB, 16-way cache with 64-byte lines has $2^{26}/(2^6 \times 16) = 65\,536 = 2^{16}$ sets, so 6 address bits select the byte and 16 select the set. With the 48-bit physical addresses AMD documents for Zen 3 EPYC processors [14, p. 10], each tag would need $48 - 16 - 6 = 26$ bits,

leaving roughly 25 bits per line for coherence state, replacement information and error-correction bits. The overhead is significant: a cache die is not all data storage.

An open detail. The combined 96 MB L3 is 16-way set associative [6, slide 20]. With 64-byte lines that is $96\,\text{MB}/(64\,\text{B} \times 16) = 98\,304 = 3 \times 2^{15}$ sets, which is not a power of two, so the simple indexing of section 4.5, in which a fixed group of address bits selects the set, cannot cover it directly. Because each die's portion of the L3 has its own data, tag and LRU arrays [6, slide 20], the base and stacked portions may be indexed separately, but AMD's materials do not say how addresses are divided between them. Lam, examining the Zen 4 version, describes each nominally 4 MB L3 slice as having its own interface to an 8 MB extension on the stacked die, with its own tags and LRU state, and suggests that the controller *probably* chooses between the base and stacked portions using a subset of the address bits [18]. That arrangement would split each slice's lines between the two dies in a 1:2 ratio and would explain the non-power-of-two total, but it remains an informed inference rather than an AMD statement.

Worked example: checking the bandwidth claim. Each core has a 32 B/cycle interface to the L3, and AMD describes the stacked connection as a 32 B/cycle bidirectional bus [6, slides 20–21]. If all eight cores transfer 32 bytes per cycle in each direction at an assumed L3 clock of 4 GHz, the total is

$$8 \times 32\,\text{B} \times 2 \times 4 \times 10^{9}\,\text{s}^{-1} \;\approx\; 2.0\,\text{TB/s},$$

consistent with AMD's "more than 2 TB/s" [6, slide 20]. The figure is therefore an aggregate peak across all cores and both directions. A single thread uses only one core's interface, which may explain why Chips and Cheese's single-thread measurement on Milan-X found a much smaller gain, about 25% more bytes per cycle, than the headline suggested; the article itself notes that the clock speed in that test was estimated [19].

A simple signal interface. Because hybrid-bond connections have low parasitics, each signal crosses between the dies through an ordinary digital path: a flip-flop and driver on one side, the TSV, electrostatic-discharge protection and an isolation stage on the other, and a receiving flip-flop [6, slide 26]. No serialiser, equaliser or separate link clock is needed. By contrast, the CCD-to-IOD link of section 8 is a serial interface with a round trip of 13 fabric clocks. This difference is the main reason a stacked cache can add only four cycles while a die-to-die hop through the package costs several nanoseconds (under 9 ns for the round trip in Rome [2]), against about 1 ns for four cycles at 4 GHz.

The L3 remains private to each CCD, so a thread running on one CCD does not benefit from another CCD's stacked cache.

16 Thermal Management, Power Delivery and Manufacturing Complexity

Stacking changes the heat path described in section 3.2: heat from the cores must now pass through additional silicon and bonding layers before it reaches the heat spreader. The first-generation design limits the penalty by placing the cache die over the CCD's central L3 region [11] and covering the cores only with structural silicon, whose stated purposes include "thermal dissipation for CPU cores" [6, slide 12]. AMD also credits hybrid bonding with "superior thermal conductance" compared with micro-bump stacking [6, slide 15], although this is an

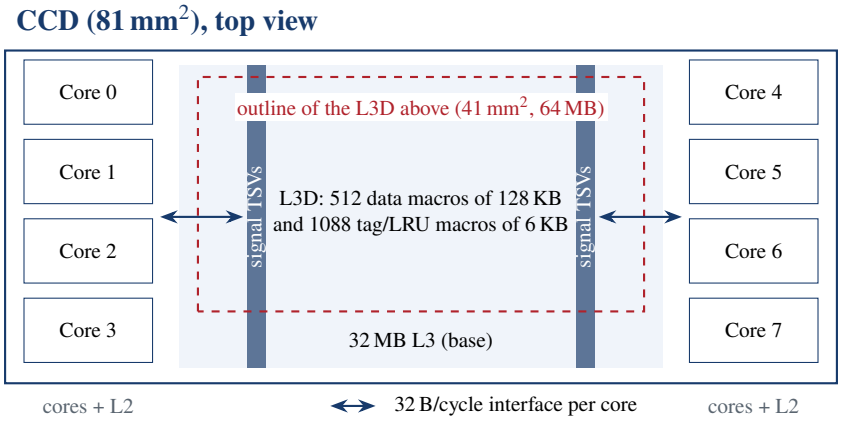


Figure 21: Top view of how the L3D connects to the CCD, redrawn from AMD's descriptions [6, slides 10–11, 19–21, 25]. Two columns of signal TSVs run along the CCD's central L3, and matching interface columns on the L3D lie directly above them. Each core reaches the L3 over a 32 B/cycle interface. Positions and sizes are schematic; the L3D outline is drawn only approximately over the L3 region.

AMD claim based on internal simulation. Earlier academic work gives a sense of scale. Black *et al.* simulated an extra 8 MB of SRAM stacked on a Core 2 Duo-class processor: the peak temperature rose from 88.35 °C to 92.85 °C, whereas a 32 MB stacked DRAM cache raised it only to 88.43 °C, because SRAM has the higher power density [24]. These are 2006 simulations of an older stacking technology and are indicative only.

Equation (5) gives a way to reason about these numbers. Treat a core's hot spot as one grid point with power P and an effective thermal resistance R to ambient, so that its temperature rise is $\Delta T = RP$. Any layer added above the core adds its own resistance in series, raising R and therefore ΔT for the same power. As an illustration with assumed values only: a hot spot that runs 60 K above ambient through its original heat path runs 66 K above ambient if the added layer raises R by 10%. Because leakage rises with temperature, the extra 6 K also raises P slightly, and the feedback loop of section 3.2 increases the final temperature further. The designer then has two choices: accept the higher temperature, or lower voltage and frequency until P falls enough to compensate. The measured clock and voltage limits of the first-generation parts (section 16.2) are consistent with the second choice, although the sources attribute them at least partly to the stacked cache's lower voltage tolerance rather than to temperature alone [18]. Black *et al.*'s simulations show the same effect in reverse: the stacked SRAM die adds both its own power and a layer, so its 4.5 K rise is much larger than that of a lower-power stacked DRAM [24].

16.1 Power delivery through the stack

Power must also be delivered upwards, through the CCD, to the cache die. The Zen 3 CCD has three primary supplies: an ungated supply (RVDD), a per-core gated supply (VDD), and a gated supply for the L2 and L3 SRAM arrays (VDDM) [6, slide 23]. The L3D needs two of these, RVDD and VDDM, and receives them through dedicated power TSVs placed in channels between the CCD's array macros [6, slide 24]. Inside the L3D the same split continues: VDDM powers the bit cells and RVDD the peripheral circuits [6, slide 25]. Supplying the bit cells separately lets them run at the voltage that keeps SRAM data stable, while the periphery can follow the rest of the logic. Each of these power paths is a set of TSVs that occupies CCD area and must carry current with little voltage drop. Hybrid bonding's lower TSV capacitance and

inductance help here as well [5].

16.2 Evidence of the thermal cost and AMD's response

Measurements of shipping parts show that the first-generation arrangement did carry a cost. In the first generation the CCD is thinned to about 20 µm, and the stacked parts accepted a lower voltage limit: the Zen 4 Ryzen 7 7800X3D has a maximum of 1.2 V [23]. Clock speeds fall accordingly. On the Ryzen 9 7950X3D, which pairs one V-Cache CCD with one plain CCD, the plain CCD clocks about 7% higher on average, and under an all-core load on an L3-sized data set the V-Cache CCD sustained 4.8 GHz against 5.15 GHz [18]. On servers, the 96-core EPYC 9684X with V-Cache has a lower all-core boost than the otherwise similar EPYC 9654 (3.42 against 3.55 GHz) and a higher default power (400 against 360 W), although its base clock is higher (2.55 against 2.40 GHz) and both reach the same 3.70 GHz maximum boost [25].

The Zen 4 generation kept the first-generation arrangement: a 5 nm CCD and a 7 nm cache die, both face-down, joined by hybrid bonding at a 9 µm minimum pitch, with structural dies alongside [8]. For the Zen 5 Ryzen 9000X3D series AMD inverted the stack. AMD states that the 64 MB cache "has been relocated below the processor, which puts the core complex die (CCD) closer to the cooling solution" [10], and its packaging engineers describe the new arrangement as placing the CPU die on top of the stack with face-to-back hybrid bonding at below 9 µm pitch [22, slides 6–7]. The two structural dies that sat beside the cache die, directly over the cores, are gone (figure 22); AMD's schematic still shows a structural silicon layer above the whole stack [22, slide 7]. Independent testing found that the Ryzen 7 9800X3D held its 4.7 GHz base clock while running at 95 °C, and characterised the change as removing the first generation's biggest weakness, the extra thermal resistance of the support silicon over the cores; a boost-clock deficit of about 300 MHz against the equivalent non-V-Cache part remains [23]. The redesign is consistent with heat removal having been a major constraint of the first design, which Cozma calls its biggest weakness [23]. It was not the only one: both Cozma and Lam describe the first-generation stacked cache as less tolerant of high voltage, which also limited boost clocks [18], [23]. How the clock penalty divides between the two causes is left open (section 33).

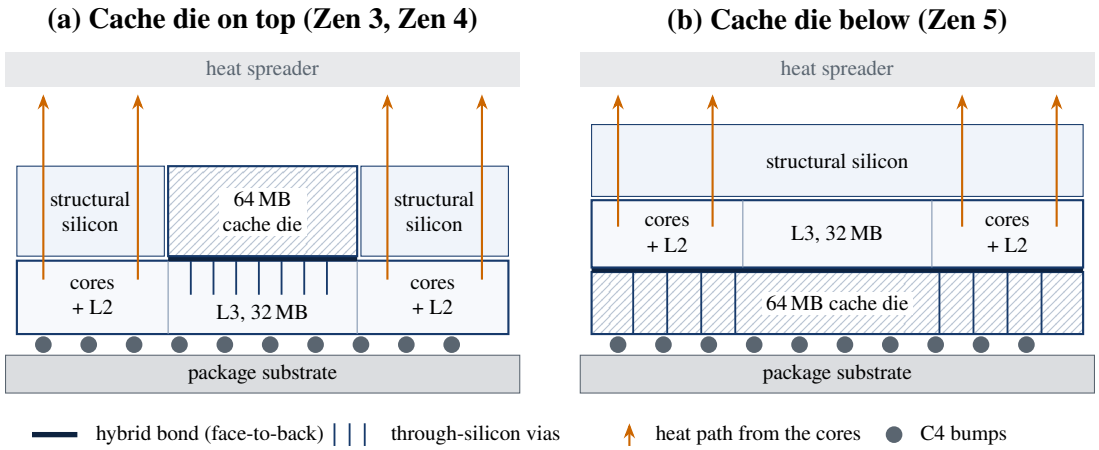


Figure 22: The two stack arrangements of 3D V-Cache, drawn from AMD's comparison [22, slide 7] and descriptions [6], [8], [23]. In (a) the cache die sits over the CCD's central L3 and the cores lie under structural silicon dies; in (b) the cache die, now the full width of the CCD, carries the through-silicon vias and the cores sit directly beneath a single structural layer. Schematic; not to scale.

AMD's own naming is inconsistent: one 2024 AMD presentation calls the Zen 4 stack the "2nd generation" [8], while AMD's 2024 press release and a 2025 presentation use the same label for the inverted Zen 5 stack [10], [22]. This paper therefore names designs by product rather than by generation.

16.3 Manufacturing: thinning, assembly, yield and test

Stacking adds steps that a planar die does not need. The CCD is thinned, to about 20 µm in the first generation [23]; a die that thin cannot support itself, which is one reason for the structural silicon [6, slide 12]. Support silicon then restores the stack to the height of an ordinary CCD, so that server packages with and without V-Cache are the same [6, slide 17]. Each of these steps costs money and adds a new way for a part to fail.

Die-to-wafer or wafer-to-wafer? Dies can be bonded one at a time onto a wafer of other dies (die-to-wafer, D2W), or two whole wafers can be bonded together (wafer-to-wafer, W2W). Dong and Xie summarise the trade-off: D2W can achieve a higher yield because each die can be tested first and only *known-good dies* (KGD) stacked, whereas W2W needs no test before bonding and is easier to align, with higher throughput, at the cost of yield [20, sec. IV.C]. AMD's 2024 comparison adds that in D2W the top and bottom dies need not be the same size and that D2W allows more heterogeneous integration, while W2W requires dies of the same or similar size [8, slide 10]. V-Cache stacks a 41 mm^2 cache die on an 81 mm^2 CCD, dies of different sizes from different wafers, so it falls on the D2W side of this comparison.

Dong and Xie's cost model shows why testing before bonding matters. For an N-layer stack bonded die-to-wafer, the cost of the bare stack is

$$C_{\mathrm{D2W}} = \frac{\sum_{i=1}^{N} \left(C_{\mathrm{die},i} + C_{\mathrm{KGD\,test}}\right) / Y_{\mathrm{die},i} \;+\; (N-1)\, C_{\mathrm{bonding}}}{Y_{\mathrm{bonding}}^{N-1}}, \tag{12}$$

while for wafer-to-wafer bonding the die yields multiply:

$$C_{\mathrm{W2W}} = \frac{\sum_{i=1}^{N} C_{\mathrm{die},i} \;+\; (N-1)\, C_{\mathrm{bonding}}}{\prod_{i=1}^{N} Y_{\mathrm{die},i}\; Y_{\mathrm{bonding}}^{N-1}} \tag{13}$$

[20, Eqs. 16–17]. In W2W, a defect in either die scraps the whole stack, so the stack's yield is as low as that of a single large die; Dong and Xie find W2W stacking always more expensive than 2D in their study, and D2W cheaper than W2W from the yield perspective [20, sec. IV.C].

Worked example: does stacking save silicon? Compare a hypothetical planar CCD with all 96 MB of L3 on one die against the actual two-die stack, using the yield model of equation (8) with $D_0 = 0.2\,\mathrm{cm}^{-2}$ and $Y_{\mathrm{wafer}} = 1$. The planar die is taken as the sum of the two real dies, $81 + 41 = 122\,\mathrm{mm}^2$; this ignores the area of the TSVs and the possibility of a denser layout, so it is only an approximation. Table 17 gives the result.

With known-good-die testing, the stack consumes about 10% less silicon than the planar die, as long as bonding loses fewer than about 10% of stacks ($139/154 \approx 0.90$). With $D_0 = 0.1$ the saving shrinks to about 5%, and with $D_0 = 0.3$ it grows to about 14%. Without testing, the stack has almost exactly the planar die's yield, as Dong and Xie's analysis predicts, and gains nothing. The silicon saving is modest; thinning, bonding and testing all cost extra, so on this measure

Table 17: Worked example: silicon consumed per good part for a planar 96 MB CCD and the stacked design (Murphy model, $D_0 = 0.2\,\mathrm{cm}^{-2}$; illustrative)

Design	Die area(s)	Modelled yield	Silicon per good part
Planar CCD with 96 MB L3	122 mm^2	79.1%	154 mm^2
Stack, dies tested first (D2W)	81 + 41 mm^2	85.4% and 92.2%	94.8 + 44.5 = 139 mm^2 $\div Y_{\text{bonding}}$
Stack, untested (W2W)	81 + 41 mm^2	$0.854 \times 0.922 =$ 78.8%	$122/0.788 = 155\,\mathrm{mm}^2$ $\div Y_{\text{bonding}}$

alone stacking is not obviously cheaper. The stronger economic argument is the one AMD gives: *product flexibility* [6, slide 6]. The same 81 mm^2 CCD is sold with or without a cache die, so only the products that need 96 MB pay for it, while a planar 122 mm^2 CCD would charge every product for the larger die.

Reliability. A bonded interface must survive years of heating and cooling. AMD reports that packaged Zen 4 V-Cache parts passed 96 hours of unbiased highly accelerated stress testing, 2500 temperature cycles and 1000 hours of high-temperature storage, with no apparent damage to the bond interface [8, slide 14]. These are AMD's qualification results, not independent tests.

Co-design. AMD's own summary of the project is that "product definition and design [were] co-optimized up front with technology development" [6, slide 33]. The TSV columns, power channels and isolation circuits had to be in the CCD before the first V-Cache product existed, and the CCD's floorplan, with the L3 at the centre and the cores at the edges, made a cool, central place for the cache die. Stacked cache was therefore a design decision taken years before it shipped, not a packaging option added at the end.

17 Workloads That Benefit From a Larger LLC

AMD's figures for V-Cache in games come from two different comparisons. Its Hot Chips 33 comparison of a 12-core 3D-chiplet prototype with the 12-core Ryzen 9 5900X, both fixed at 4.0 GHz, reports gains from 1.04× to 1.25× across five games, 15% on average [5, slide 19]. Holding frequency fixed is valuable because it isolates the cache effect, but the result covers only the disclosed games and configuration and should not be generalised.

The ISSCC 2022 presentation adds product results, again from AMD's own testing [6, slides 29, 31, 35]. The eight-core Ryzen 7 5800X3D is reported as about 15% faster in games than the twelve-core Ryzen 9 5900X at 1080p with high settings, ranging from up to 1.36× in one title to a tie in another. On servers, a 16-core EPYC with V-Cache completed about 66% more electronic-design (RTL simulation) jobs per hour than a 16-core EPYC 73F3 without it. The spread from a tie to 1.36× already illustrates the workload dependence predicted in section 4.4; section 24 discusses what else differs between the compared parts.

The miss classification of section 4.4 predicts which workloads should benefit. A larger L3 removes capacity misses, so the gain depends on where a workload's working set lies relative to the 32 MB and 96 MB capacities:

- **Working set already within 32 MB.** Few capacity misses remain to remove, and every L3 hit pays the extra four cycles. Little or no gain, possibly a small loss.
- **Working set between 32 MB and 96 MB, with reuse.** Many accesses that went to memory now hit in the L3. This is where large gains are expected.
- **Working set far larger than 96 MB, or streaming with little reuse.** The data still does not fit or is not reused, so the miss rate barely changes.

Mittal and Vetter note that modern scale-out workloads such as business analytics and web search can demand hundreds of megabytes of last-level cache [16, sec. 1]. The spread in AMD's own gaming results, from a tie in one title to 1.36× in another [6, slide 29], and the 66% gain in RTL simulation [6, slide 31], fit this picture, but they are vendor results and Part V examines them more critically.

Independent measurements support this classification. Lam measured each CCD of a Ryzen 9 7950X3D separately, with boost disabled and both CCDs held at 4.2 GHz, and used performance counters to compare IPC and L3 hit rate [18]. Because clock, core design, memory and software are identical, the differences isolate the cache. Workloads with many L3 misses gained most: IPC rose by over 19% in *Call of Duty: Black Ops Cold War*, whose hit rate rose by 47%, and by 13% in *Cyberpunk 2077* (from 1.26 to 1.43). Workloads with few misses to remove gained little: libx264 video encoding, with 1.48 L3 misses per thousand instructions, gained 4.9%, less than the plain CCD's 7% clock advantage, and a flight simulator with only 0.35 misses per thousand instructions lost a little IPC, which Lam suggests is likely due to the added latency. These are the first two classes above, measured directly (figure 23); section 15.1 works through the libx264 case with equation (11).

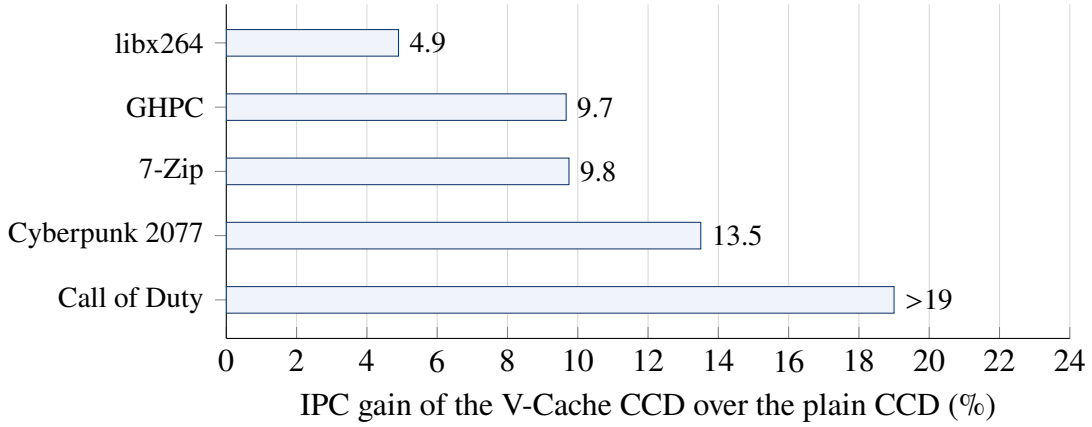


Figure 23: IPC gain from 3D V-Cache with clock, core design and memory held equal: the two CCDs of one Ryzen 9 7950X3D, both fixed at 4.2 GHz, measured with performance counters by Lam [18]. Cyberpunk 2077 is computed from the reported IPC of 1.26 and 1.43; the Call of Duty value is reported only as "over 19%". A sixth workload, a flight simulator with very few L3 misses, lost a small, unquantified amount of IPC and is not plotted.

HPC studies show the third class. On a memory-bound proxy application, Domke *et al.* found that Milan-X was up to 3.4× faster than Milan, but only for a subset of problem sizes [26]. Lawenda *et al.* found the same pattern in OpenFOAM: Milan-X was up to 1.54× faster than Milan for the mesh size that best fits its larger cache, the advantage fell to 1.24× for the largest mesh, and on some smaller meshes Milan-X was up to 19% slower; the larger L3 cut the L3 miss rate by up to 30 percentage points [27]. On desktop software, Phoronix's 420-test comparison

of the eight-core Ryzen 7 5800X3D with the eight-core 5800X found large V-Cache leads in a minority of tests, while the 5800X won, mostly by a couple of percent, where the cache could not be used and its slightly higher clocks paid off [28].

18 Summary of Part IV

Applying the three-part test to 3D-stacked cache:

- **Bottleneck addressed.** The area, yield, wire-length and power costs of growing an SRAM cache in the plane of the compute die, made worse by SRAM's slowing area scaling (section 12).
- **Benefit.** Three times the L3 per CCD (32 to 96 MB) for four extra cycles of latency, with more than 2 TB/s of bandwidth and no change to the package footprint or the cache hierarchy above the L3 [6], [11].
- **New trade-offs.** An altered heat path from the cores, power delivery through TSVs, die thinning, bonding and test complexity, a second die to pay for, and a benefit that depends strongly on the workload.

The worked examples of this part make these points quantitative. The 9 µm bond pitch gives about sixteen times the connection density of micro-bumps (section 14.3), which is what makes a wide, parallel, cache-speed interface possible and keeps the latency cost to four cycles. Applied to Lam's libx264 measurements, the simple trade-off of equation (11) predicts a gain of the right size (section 15.1). The yield model suggests that stacking with tested dies saves only about a tenth of the silicon of an equivalent planar die (table 17), so the economic case rests mainly on product flexibility. And the heat path, which the first generation managed with structural silicon and lower clocks, was important enough for AMD to invert the stack in Zen 5 (section 16.2). Part V now turns from the mechanism to the evidence across products.

Part V

AMD Case Study and Quantitative Analysis

Target length: about 25 pages. Draft status: method, AMD-reported evidence and confound grid drafted; independent evidence, Zen 2 to Zen 3 comparison and summary added.

19 From Zen 2 Chiplets to Zen 3 and 3D V-Cache

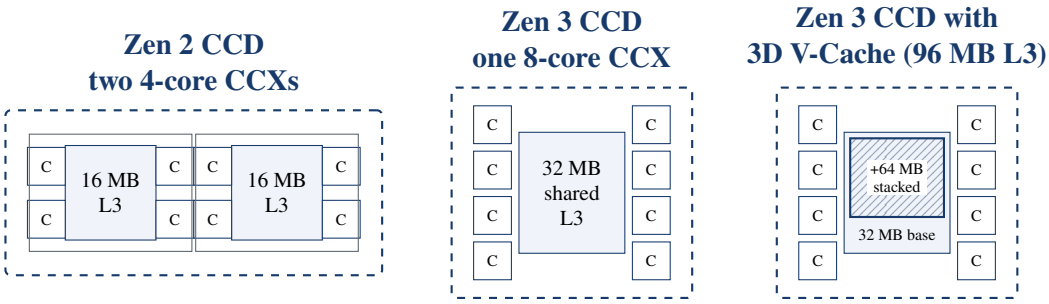


Figure 24: Schematic progression of L3 organisation per compute die from Zen 2 to Zen 3 and Zen 3 with 3D V-Cache, based on AMD's descriptions [3], [6], [11]. Logical organisation only; not a floorplan.

Table 18 and figure 24 summarise the changes in cache organisation that matter for this paper, using only AMD's published descriptions.

Two steps stand out. Zen 3 did not add L3 capacity per CCD but changed its *sharing*: each core can now use the whole 32 MB rather than only its own CCX's 16 MB [3], [11]. 3D V-Cache then tripled the capacity seen by each core without changing the CCD's footprint or the sharing structure [6], [11]. These two steps give a natural experiment for the effect of L3 capacity, as long as other differences are controlled (section 24).

19.1 Zen 2 to Zen 3: a same-core-count comparison

The Zen 2 to Zen 3 step can be examined with two eight-core, single-CCD desktop parts. The Ryzen 7 3700X (Zen 2) and Ryzen 7 5800X (Zen 3) both have eight cores, sixteen threads and 32 MB of L3, but the 3700X splits its L3 into two 16 MB halves, one per four-core CCX, while the 5800X shares all 32 MB among its eight cores [3], [11]. They differ in other ways too: base

Table 18: L3 organisation per compute die across the generations studied (AMD-published figures)

	Zen 2 CCD	**Zen 3 CCD**	**Zen 3 CCD with 3D V-Cache**
Cores per CCD	8	8	8
CCX organisation	Two 4-core CCXs	One 8-core CCX	One 8-core CCX
L3 shared by one core	Up to 16 MB (its CCX)	32 MB	96 MB
L3 per CCD	2×16 MB	32 MB	32 MB base + 64 MB stacked
Private L2 per core	512 KB	512 KB	512 KB
CCD area	74 mm^2	81 mm^2	81 mm^2 + 41 mm^2 L3D
Added L3 latency	—	—	+4 cycles
Sources	[2], [3]	[6], [11]	[6], [11]

and maximum boost clocks of 3.6 and 4.4 GHz against 3.8 and 4.7 GHz, and a TDP of 65 W against 105 W [29].

AMD attributes a 19% IPC increase to Zen 3, citing the unified eight-core complex with direct access to 32 MB of L3 among the core improvements. The figure comes from 25 workloads run at a locked 4 GHz on the eight-core Ryzen 7 3800XT and 5800X, both with DDR4-3600 memory [4, note 4]. Because clock, core count and memory are held equal, this is a cleaner comparison than most vendor figures. It still cannot separate the shared L3 from the many other changes to the Zen 3 core.

Independent testing of the two retail parts shows the combined effect. With the same DDR4-3200 CL14 memory, TechSpot measured the 5800X ahead of the 3700X by 22% in Cinebench R20 multi-core, 24% and 19% in 7-Zip compression and decompression, 16% in Blender, 30% in V-Ray, 26% in Corona, 12% in code compilation, 6% in DaVinci Resolve, 14% in Premiere Pro, 27% in Photoshop and 23% in After Effects [30]. During the multi-core Cinebench run the 5800X held about 4.55 GHz, and it drew about 20 W more than the 3700X [30]. These are whole-processor differences. The 5800X runs at higher clocks under a higher power limit and has a redesigned core, so the gains cannot be credited to the unified L3 alone. The gains are broad rather than concentrated in cache-sensitive work: the largest is in V-Ray, a multi-threaded renderer (30%), and large gains also appear in Photoshop (27%), which the reviewer describes as relying heavily on single-core performance, which points to the core and clock as much as to the cache [30]. Cache measurements give a more direct view. The FPS Review ran AIDA64's cache and memory benchmark on both processors with identical DDR4-3600 CL16 memory and the same fabric clock (1800 MHz) [12]. Memory read bandwidth was the same, about 50.7 GB/s on both, and memory latency fell from 68.8 ns on the 3700X to 61.8 ns on the 5800X. L3 read bandwidth was also nearly identical (633 against 631 GB/s). L3 latency, however, *rose* from 9.9 ns to 10.9 ns [12]. At the core clocks the tool reported during the runs (4.40 and 4.85 GHz), this is roughly 44 cycles against 53 cycles. The figures come from screenshots of a single run on each processor, so the cycle counts are only approximate. Chips and Cheese's latency measurements across Zen generations agree: unifying the two 16 MB caches of a Zen 2 CCD into one 32 MB cache added 7 to 8 cycles of L3 latency, whereas doubling the size of each L3 slice from Zen to Zen 2 added about 5 cycles and tripling it with V-Cache only 3 to 4, which suggests that enlarging slices is cheaper in latency than merging separate caches into one [19].

This result fits the trade-off described in section 12. Doubling the cache a core can reach, from 16 to 32 MB, removes misses for working sets between the two sizes and the traffic between CCXs, but a larger shared array serving more cores takes longer to reach. The 5800X's faster single-threaded applications, in which the 32 MB working-set limit rarely matters, therefore most likely owe their gains to the core and clock, not to faster cache access. Isolating the benefit of the unified L3 itself would require fixed-clock measurements of workloads with working sets between 16 and 32 MB, or of communication between cores in different CCXs, which none of the sources in hand provides.

Two later generations extend the same pattern (table 19). With Zen 4, AMD kept the stack unchanged in principle but moved the CCD to 5 nm while the cache die stayed at 7 nm [8]; on servers, up to twelve CCDs per package each reach 96 MB of L3 with V-Cache, for up to 1152 MB per socket [9]. With Zen 5, AMD inverted the stack so that the cache die sits below the CCD (section 16.2).

Table 19: 3D V-Cache across Zen generations (AMD-published figures unless noted)

	Zen 3	**Zen 4**	**Zen 5**
Example parts	Ryzen 7 5800X3D; EPYC 7773X	Ryzen 7 7800X3D; EPYC 9684X	Ryzen 7 9800X3D
CCD / cache die	7 nm / 7 nm	5 nm / 7 nm	4 nm (TSMC) / not stated
Stack order	cache die on top of CCD	cache die on top of CCD	CCD on top of cache die
L3 per V-Cache CCD	96 MB	96 MB	96 MB (32 + 64)
Largest socket L3	768 MB	1152 MB	—
Added L3 latency	3–4 cycles (measured)	about 4 cycles (measured)	4 cycles (measured)
Sources	[6], [11], [19]	[8], [9], [18]	[10], [22], [23], [31]

AMD lists the Ryzen 7 9800X3D with 96 MB of L3, cores made in TSMC's 4 nm process and an I/O die in 6 nm [31]; it does not state the process used for the cache die.

20 Comparing Architectural Configurations

The comparisons in this part rely on pairs of processors that differ mainly in L3 capacity. Table 20 lists the pairs used and what else differs between them.

21 Method for Analysing Published Evidence

The quantitative analysis uses published research and benchmarks only (figure 25). This is the most manageable option given that the group does not have access to the relevant hardware. Its

Table 20: Processor pairs that differ mainly in L3 capacity

Pair	Cores	L3	Other differences	Source
EPYC 7763 vs 7773X (Zen 3)	64	256 vs 768 MB	base clock 2.45 vs 2.20 GHz; same 3.5 GHz boost	[7]
Ryzen 7 5800X vs 5800X3D (Zen 3)	8	32 vs 96 MB	5800X clocks slightly higher	[28]
7950X3D: plain CCD vs V-Cache CCD (Zen 4)	8 per CCD	32 vs 96 MB	plain CCD about 7% faster at stock; identical when both are fixed at 4.2 GHz	[18]
EPYC 9654 vs 9684X (Zen 4)	96	384 vs 1152 MB	all-core boost 3.55 vs 3.42 GHz; base 2.40 vs 2.55 GHz; same 3.70 GHz maximum; default power 360 vs 400 W	[25]

limitation is dependence on the experimental conditions and measurements that others chose to publish; the analysis can only be as controlled as its sources.

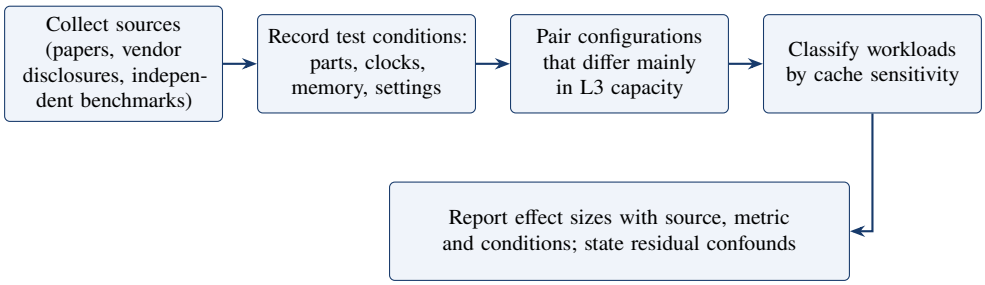


Figure 25: Planned evidence-analysis procedure for Part V (published research and benchmarks only; no new hardware measurement).

Each source is recorded with: the processors compared; clock frequencies (fixed or boost); memory type, speed and timings; power limits; software and settings; the metric (for example, frames per second, IPC, runtime or throughput); and whether the source is the vendor or an independent party. Results are reported as relative changes with the source, metric and conditions stated, never as a single headline number.

Source selection. Sources are admitted in three tiers: peer-reviewed papers; independent benchmark publications that state their hardware, settings and method; and vendor material, which is used only when labelled as such. Every source used must have its PDF in the project's source folder. Where sources conflict, the one with the better-controlled comparison (section 24) is preferred and the disagreement is reported. Averages across workloads are formed as geometric means of performance ratios, so that no single workload dominates.

22 Published Benchmarks and Performance Measurements

The evidence falls into two groups: AMD-reported results (table 21) and independent measurements (table 22).

The independent results agree with AMD's on direction but not on size: they show a wider

Table 21: AMD-reported performance figures relevant to this study (vendor claims, not independent measurements)

Claim	Conditions stated	Source
About 15% faster gaming, 8-core Ryzen 7 5800X3D vs 12-core Ryzen 9 5900X; per-title results from a tie to 1.36×	Six games at 1080p, high preset; both with DDR4-3600 and RTX 3080; AMD testing, Dec. 2021	[6, slides 29, 35]
About 66% more RTL-simulation jobs per hour (24.4 to 40.6), 16-core EPYC with V-Cache vs 16-core EPYC 73F3	Synopsys VCS; same reference platform; AMD testing, Sept. 2021	[6, slides 31, 35]
15% average gaming uplift, 12-core 3D-chiplet prototype vs Ryzen 9 5900X	Both at the same fixed frequency; disclosed games only	[5]
15% IPC improvement from Zen to Zen 2	Many simultaneous changes, including doubled L3	[3]
19% IPC improvement from Zen 2 to Zen 3 (one AMD test, reported in both sources)	25 workloads at a fixed 4 GHz, 3800XT vs 5800X, equal memory; but a redesigned core as well as the unified L3	[4], [6, slides 10, 35]
19% average memory-latency reduction, Rome vs Naples	Central IOD, decoupled clocks, optimised IFOP	[2]

Table 22: Independent measurements of the effect of 3D V-Cache

Result	Conditions	Source
IPC +4.9% to +19% across five workloads; one workload slightly negative	7950X3D, each CCD at a fixed 4.2 GHz; performance counters	[18]
Up to 3.4× on MiniFE for a subset of problem sizes	EPYC 7773X vs 7763, 64 cores each	[26]
OpenFOAM from 0.81× to 1.54× (motorBike) and up to 1.47× (urban air pollution); L3 miss rate down by up to 30 points	EPYC 7773X vs 7763, dual socket, clocks not fixed	[27]
gem5 +7.6%, libx264 and 7-Zip about +5% on one CCD; Milan-X clocked about 5% lower	Milan-X vs Milan cloud instances, clocks estimated	[19]
Of 350 significant differences among 420 tests, 106 favour the 5800X3D (12 by 50% or more) and 244 the 5800X (all but 3 by under 10%)	Ryzen 7 5800X3D vs 5800X, Linux	[28]
L3 latency +3 to +4 cycles	Milan-X, 7950X3D, 9800X3D	[18], [19], [23]

spread, including small losses, and the best-controlled of them, Lam's fixed-frequency per-CCD measurements, shows single-digit to high-teen IPC gains rather than uniform double-digit speedups.

22.1 Latency through the whole hierarchy

The clearest picture of what the stacked cache changes comes from a latency sweep, in which a test program walks through a region of memory of increasing size and measures the time per load. Chips and Cheese ran such a sweep on Milan and Milan-X cloud instances (figure 26) [32]. The two curves coincide through L1 (4 cycles) and L2 (about 12 cycles). In the L3 range Milan-X is slightly slower, 51.14 against 47.49 cycles, the three to four extra cycles discussed in section 15. The curves separate at 32 MB: Milan's latency climbs towards memory, while Milan-X's stays flat until beyond about 64 MB and then rises as its 96 MB L3 is exceeded. Between those sizes a load costs about 51 cycles on Milan-X, while on Milan it rises to roughly 250–300 cycles by 64–96 MB, five to six times as long. Beyond 96 MB both go to memory, at 342 cycles for Milan and 313 for Milan-X; in nanoseconds Milan-X's memory latency was also lower (89 against 106 ns), and the article notes that Milan-X was boosting better than Milan in this test [32].

A third source agrees. On a Zen 3 Ryzen 5 5600G, which has a 16 MB L3 on a single die, 7-cpu.com measured 4 cycles for L1, 12 for L2, 47 for L3 and 47 cycles plus 74 ns for memory, about 375 cycles at the chip's 4.45 GHz [13]. The same source shows a second effect that latency sweeps can mix up with cache misses: with ordinary 4 KB pages, latency at 512 KB was 21 cycles, against 12 with 2 MB pages, although the data still fits in L2; the source's table attributes part of the rise to misses in the 64-entry first-level data TLB, which can no longer hold the address translations, and leaves the exact 4 KB penalty unstated [13]. Sweeps such as Chips and Cheese's therefore measure cache and TLB effects together, and the steps in figure 26 are not purely cache capacities.

These measurements also check the textbook's numbers. The measured memory latency of 313–342 cycles lies just above the 100–300-cycle range assumed in section 15.1, so on these processors a removed miss saves at least as much as that example allowed. With the measured values, equation (11) gives a break-even of $3.65/313 \approx 1.2$ percentage points of L3 misses removed for Milan-X to come out ahead.

22.2 A working-set sweep: MiniFE

Domke *et al.* ran the MiniFE finite-element proxy application on dual-socket Milan and Milan-X systems with the same 64 cores per socket, 280 W TDP and 16-channel DDR4 memory, while scaling the problem from 100^3 to 400^3 grid points [26]. Figure 27 shows the result: a sweep through working-set sizes of exactly the kind sketched in figure 15.

- At the smallest sizes ($n = 100$ and 110) Milan-X is about 10% *slower*. Both systems hold the data in L3, so the difference is probably Milan's higher base clock (2.45 against 2.20 GHz); section 24.1 tests this.
- From $n = 120$ the advantage rises steeply, to 3.4× at $n = 160$. In this range Milan's 512 MiB of L3 (two sockets) no longer holds the data, while Milan-X's 1536 MiB still does.

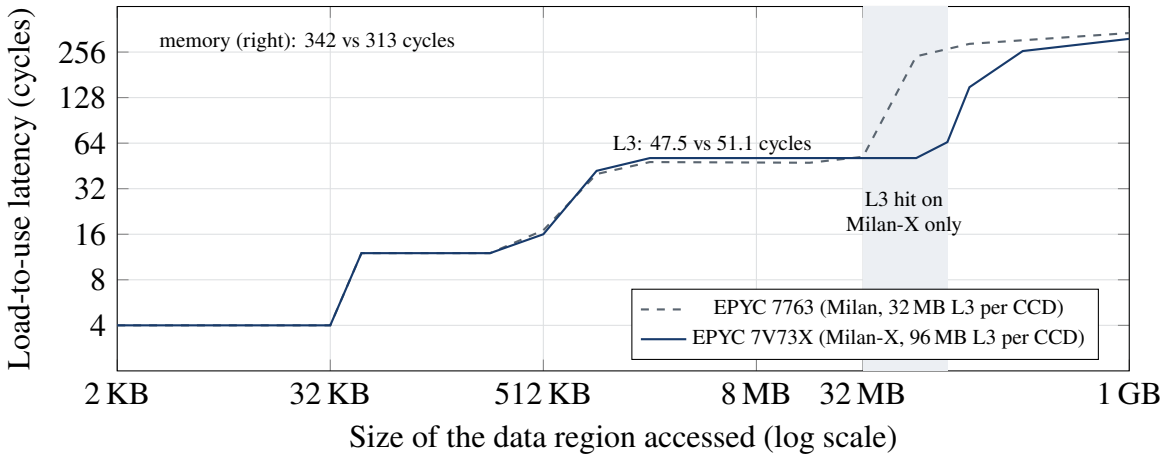


Figure 26: Cache and memory latency against data size for Milan and Milan-X, redrawn from Chips and Cheese's measurements [32]. The labelled values (L1 4 cycles, L2 about 12 cycles, L3 47.49 and 51.14 cycles, memory 342 and 313 cycles) are taken from the chart's own labels; the transitions between them were read by eye and are approximate. In the shaded range, from 32 MB to about 96 MB, Milan-X still hits in its L3 while Milan already goes to memory.

- Beyond $n = 170$ the advantage falls quickly and settles at about 1.04× from $n \approx 330$ onwards, once the data exceeds both caches.

Worked example: inferring the working set. The curve can be used to estimate how much data each grid point needs, with no information about MiniFE's data structures. Milan's own performance falls sharply between $n = 120$ and 130 [26, Fig. 1], that is, between 1.73 and 2.20 million points; if this is where the data outgrows 512 MiB, each point needs about $512 \times 2^{20} / (1.73 \text{ to } 2.20 \times 10^6) \approx 245$–$310$ bytes. Milan-X's performance falls between $n = 170$ and 180, between 4.91 and 5.83 million points; with 1536 MiB of L3 that gives about 275–330 bytes per point. The two independent estimates agree, which supports the reading that the speedup curve is a cache-capacity effect: about 300 bytes per grid point fit in three times as many points' worth of L3 on Milan-X. The estimate is approximate, because not all of the L3 holds useful data and the two sockets' caches are not perfectly shared.

22.3 OpenFOAM in detail

Lawenda *et al.*'s OpenFOAM study adds measured miss rates and clock speeds [27]. For the dual-socket 64-core Rome, Milan and Genoa systems, all with 256 MB of L3 per socket, the L3 miss rate rose from about 10–15% on the smallest motorBike meshes to about 80% on the largest. Milan-X, with 768 MB per socket, cut the miss rate by up to 30 percentage points on the mid-sized to large meshes. Its performance relative to Milan ranged from 0.81× to 1.54× across the eight mesh sizes: slower on some small meshes, fastest at the "mhigh" mesh of 6.7 million cells, and falling back to 1.24× on the largest, 39.4-million-cell mesh, where the data exceeds even Milan-X's cache. Expressed per core, Milan-X had the advantage between about 15 000 and 100 000 cells per core, and its best utilisation came at about 14 800 cells per core against about 4700 for Rome and Milan. The newer generation shows the same pattern: Genoa-X ranged from 0.98× to 1.60× against the 96-core Genoa [27].

The study's measured clocks show the confound discussed in section 24: during the runs, Milan

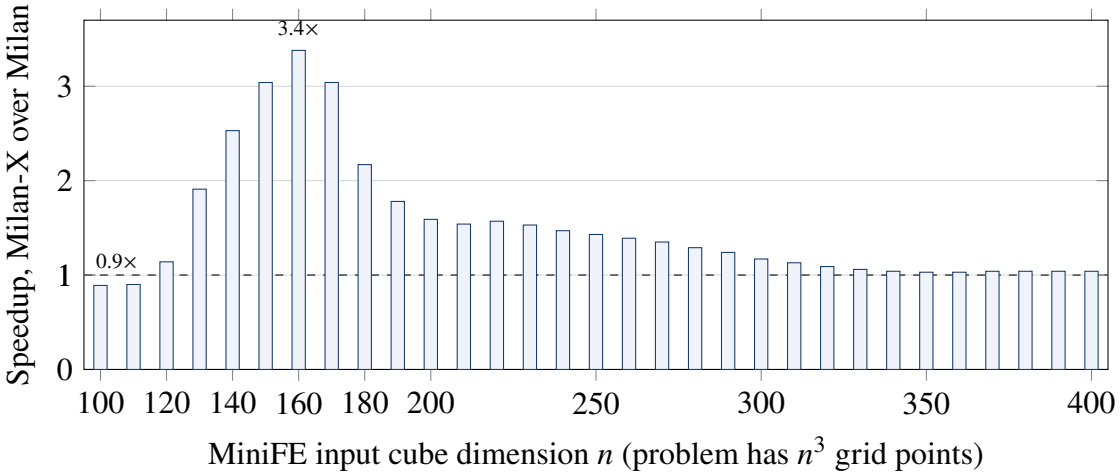


Figure 27: MiniFE on a dual-socket EPYC 7773X (Milan-X, 2×768 MiB L3) against a dual-socket EPYC 7763 (Milan, 2×256 MiB L3) as the problem size grows. Values were read by eye from Domke *et al.*'s figure [26, Fig. 1] and are accurate to about ±0.05. Milan-X runs at a lower base clock (2.20 against 2.45 GHz).

ran at 2.9–3.2 GHz and Milan-X at 2.8–3.0 GHz [27, Table 5]. Milan-X's losses on the small meshes, where both systems' caches suffice, are therefore at least partly a clock effect, as in MiniFE.

22.4 Breadth: 350 desktop results

Phoronix compared the eight-core Ryzen 7 5800X3D and 5800X across 420 benchmarks on Linux and charted the 350 with a statistically significant difference [28]. Classifying each result by the colour of its bar in that chart gives the distribution in figure 28. The 5800X was faster in 244 tests, but all except three of those margins were under 10% and the median was about 4%, consistent with its higher clocks. The 5800X3D was faster in 106 tests, with a median margin of about 11%; 58 of its wins were 10% or more and 12 exceeded 50%, led by ASKAP's tConvolve gridding kernel (294%), Zstd compression, oneDNN, CloverLeaf and OpenFOAM (68%). Phoronix's own summary describes the 5800X winning "a small set of tests"; the chart shows that the 5800X won more tests but by much smaller margins. The distribution is the clearest available picture of the workload dependence: for most desktop software the stacked cache costs a few percent, and for a minority it is worth tens of percent or more.

23 Cache-Sensitive Versus Cache-Insensitive Workloads

The evidence sorts workloads into three groups, matching the working-set argument of section 17:

- **Strongly cache-sensitive:** memory-bound HPC and engineering codes whose working sets fall between the base and stacked capacities (MiniFE, OpenFOAM at mid-sized meshes, RTL simulation) and games with high L3 miss rates. Gains range from tens of percent to more than 3× [18], [26], [27].
- **Mildly sensitive:** workloads with moderate miss rates, such as compression and video

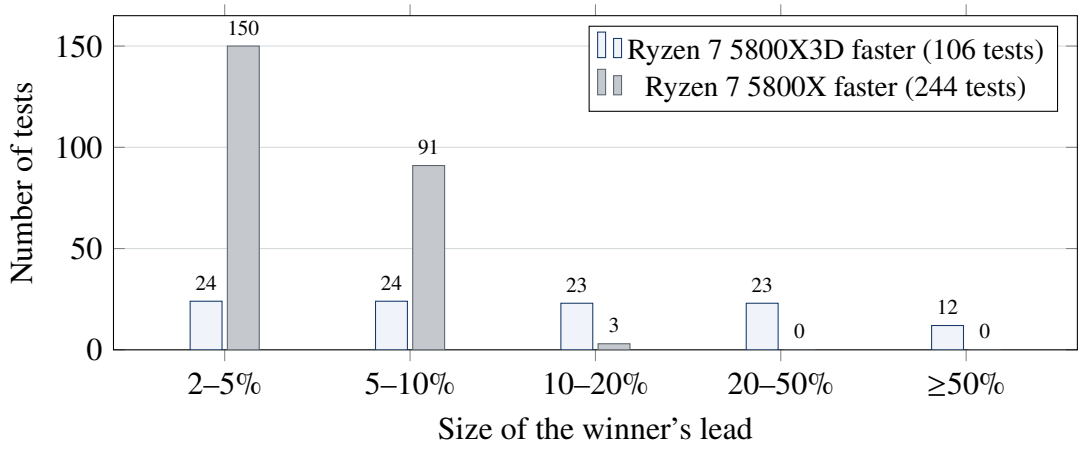


Figure 28: Distribution of the 350 statistically significant differences among Phoronix's 420 benchmarks of the Ryzen 7 5800X3D and 5800X [28]. The winner of each test was identified from the colour of its bar in Phoronix's comparison chart and matched to the chart's printed percentages; this classification is ours. Each bin includes its lower bound (for example, 5–10% means at least 5% and less than 10%).

encoding, gain around 5–10% at equal clock [18], [19], which a V-Cache part's lower clock can cancel.

- **Insensitive or negative:** workloads whose working sets already fit (small OpenFOAM meshes, the flight simulator in Lam's tests) or that are compute-bound gain nothing and can lose slightly from the added latency or lower clocks [18], [27], [28].

The useful predictor is therefore not the application type but its L3 miss rate on the base configuration: in Lam's results, the workloads with the most L3 misses gained most and those with the fewest gained least or lost [18].

24 Separating Cache Effects From Other Factors

A difference in measured performance between two processors can rarely be attributed to cache size alone. Using equation (1) and equation (6), the main confounding factors are:

- **Clock frequency.** If the larger-cache part runs at a different frequency, f in equation (1) changes as well as IPC. Fixed-frequency comparisons, like the one AMD reports for its prototype [5], remove this factor.
- **Core count and topology.** More cores or a different number of CCDs changes throughput and the cache available per core. AMD's headline desktop comparison sets an eight-core, single-CCD Ryzen 7 5800X3D against a twelve-core, two-CCD Ryzen 9 5900X [6, slide 29], so it mixes cache capacity with core count, CCD topology and clock speed. Its server comparison holds the core count at 16 but uses the EPYC 73F3 as the baseline, an F-series model that AMD positions for high frequency [6], [11].
- **Memory bandwidth and latency.** Memory speed, timings and the fabric clock change the miss penalty in equation (6) and can make a larger cache appear more or less valuable.
- **Concurrent microarchitectural changes.** Generational comparisons mix many changes. AMD's 15% Zen-to-Zen 2 IPC figure, for example, combines a new branch predictor, a

larger op cache, wider floating-point paths and a doubled L3, among others [3]; it cannot be credited to the cache.

- **Power limits and software.** Package power limits, operating-system scheduling across CCDs, and application versions all affect results.

Table 23 applies this checklist to the comparisons available so far. None of AMD's published product comparisons controls every factor. The fixed-frequency prototype comparison controls clock speed but not core count, and the server comparison controls core count but not frequency. The independent comparisons are graded the same way below the line.

Table 23: Which confounding factors each available comparison controls (✓ = controlled, × = not controlled, ~ = estimated and normalised, ? = not stated; AMD-reported rows above the line, independent below)

Comparison	Clock	Cores	Memory	Same core design
12-core 3D prototype vs Ryzen 9 5900X, fixed frequency [5]	✓	✓	?	✓
Ryzen 7 5800X3D vs Ryzen 9 5900X [6, slides 29, 35]	×	×	✓	✓
16-core EPYC with V-Cache vs EPYC 73F3 [6, slides 31, 35]	×	✓	?	✓
Zen to Zen 2 IPC [3]	?	?	?	×
Zen 2 to Zen 3 IPC, 3800XT vs 5800X at 4 GHz [4]	✓	✓	✓	×
7950X3D V-Cache CCD vs plain CCD, both at 4.2 GHz [18]	✓	✓	✓	✓
EPYC 7773X vs 7763 [26], [27]	×	✓	✓	✓
Milan-X vs Milan, one CCD [19]	~	✓	?	✓
Ryzen 7 5800X3D vs 5800X [28]	×	✓	?	✓
Ryzen 7 5800X vs 3700X [30]	×	✓	✓	×

The prototype comparison holds both clock (4.0 GHz) and core count (12) constant but does not state the memory configuration [5, slide 19]. Among the independent results, Lam's per-CCD measurements control every listed factor, which makes them the strongest available evidence for the effect of the stacked cache itself [18].

24.1 Worked example: correcting for clock speed

Where clocks were not fixed, a rough correction is possible if the clocks during the test are known. If performance scaled linearly with frequency, dividing a measured speedup by the ratio of clock speeds would give the per-clock effect. Memory-bound code does not scale linearly, because memory latency in nanoseconds does not improve with core clock, so the true per-clock effect lies between the raw ratio and the corrected one. Table 24 applies this to the comparisons whose clocks were measured or stated.

The correction changes the picture in two ways. For MiniFE's smallest input the whole 11%

Table 24: Worked example: speedups bounded by a clock correction (corrected = measured ÷ clock ratio)

Comparison	Clock ratio (V-Cache / plain)	Measured	Clock-corrected
OpenFOAM, best mesh [27]	2.9/3.05 ≈ 0.95 (midpoints of measured ranges)	1.54×	1.62×
OpenFOAM, worst mesh [27]	≈ 0.95	0.81×	0.85×
gem5, one CCD [19]	3.15/3.3 ≈ 0.95 (estimated)	1.076×	1.13×
MiniFE, smallest input [26]	2.20/2.45 ≈ 0.90 (base clocks)	0.89×	0.99×

loss disappears, which supports the reading that Milan-X loses there only because of its clock. For OpenFOAM's worst mesh a loss of about 15% remains even after correction, so the clock alone does not explain it; Lawenda *et al.* do not analyse that case, and the four extra cycles of L3 latency cannot account for so large a difference on their own. This is a question the published evidence leaves open. Because base clocks and measured ranges are crude stand-ins for the actual frequency during each run, the corrected values are indicative only.

25 Summary of Part V

The evidence answers RQ4 as follows. The stacked L3 costs about four cycles of L3 latency in every generation measured, and in its first two generations also several percent of clock speed. In exchange it removes capacity misses for working sets between 32 and 96 MB per CCD. Where those misses dominate, the benefit is large: from about 10–20% higher IPC in miss-heavy games to more than 1.5× in memory-bound simulation, and up to 3.4× for one HPC proxy at favourable problem sizes. Where they do not, the benefit is small, absent or slightly negative. Separating the cache effect from other factors requires either fixed-clock comparisons or performance-counter evidence; most vendor comparisons provide neither, and the best-controlled independent study available covers only a handful of workloads. The confidence of these conclusions is therefore high for the direction and mechanism, and moderate for the size of the effect in any particular workload.

Part VI

Comparative Synthesis, Limitations and Future Directions

Target length: about 20 pages. Draft status: comparison, interactions and remaining bottlenecks drafted; conclusions and open questions drafted.

26 Monolithic, Chiplet-Based and Vertically Stacked Designs Compared

Table 25 compares the three organisations against the bottleneck–benefit–trade-off test of section 1.4, with the measured or modelled values from Parts III–V where they exist.

27 Other Ways to Enlarge the Last-Level Cache

3D V-Cache is one of several ways to give a processor more last-level cache. Table 26 places it beside the alternatives for which the sources give numbers.

A side-by-side cache in practice. Intel's Broadwell processors offer a real example of the side-by-side cache die discussed in section 13. They placed a 128 MB embedded-DRAM die next to the processor die and used it as a fourth-level cache. Lam measured its latency at over 30 ns (34.18 ns in his chart, against 10.65 ns for the 7950X3D's V-Cache CCD) and its bandwidth at about 50 GB/s, a quarter of Broadwell's own L3 bandwidth, and explains why: reaching the eDRAM involves another trip over the on-chip ring bus and then Intel's OPIO package link, which on the related Haswell design is a 64-bit full-duplex interface at 6.4 GT/s, providing 51.2 GB/s in each direction [18]. Because the eDRAM was too slow to be part of the L3, the L3 stayed in place to protect the cores from its latency, and Broadwell even used part of the L3's data array to hold the tags for the L4 [18]. Lam contrasts this with V-Cache: each nominally 4 MB slice of the Zen 4 L3 gets its own interface to an 8 MB extension on the stacked die, complete with tag and LRU arrays, so the stacked SRAM can extend the L3 rather than add a level [18]. He also notes that Zen 4's own Infinity Fabric link between dies provides only about 64 GB/s of read bandwidth per CCD, little better than Broadwell's link, and concludes that package traces are not well suited to L3 bandwidths [18]. This is the comparison of table 16 with real numbers: the side-by-side cache had about three times the latency of the stacked one and, in Lam's multithreaded test, about a thirteenth of its bandwidth (53 against 702 GB/s), and it had to sit at a different level of the hierarchy [18].

Table 25: Comparison of the three design approaches (model values illustrative; measured values from the cited sections)

	Monolithic die	**Chiplets (2D package)**	**Chiplets with stacked cache**
Main limit addressed	—	Die size, yield, uneven node scaling	Planar cost and wire length of a large LLC
Communication	All on-die	Die-to-die links through the package	Adds dense vertical links between cache and compute dies
Memory latency	Shortest on-die paths	Extra die-to-die hop to memory	Fewer trips to memory for cache-sensitive work
Cost and yield	Falls steeply with die area	Smaller dies, mixed nodes, tested before assembly	Extra die and bonding step; capacity only where sold
Thermal	One heat source per die	Heat spread over several dies	Stack alters heat path from cores
New trade-off	Scaling ceiling	Latency, link power, packaging	Heat, test complexity, workload dependence
64-core compute sets per wafer (model)	about 35	about 95	about 95, plus cache dies
L3 per compute die (Zen 3)	—	32 MB	96 MB
L3 hit latency (Milan / Milan-X)	—	47.5 cycles	51.1 cycles
Clock (Zen 4 desktop, same package)	—	reference	about 7% lower

Table 26: Ways to enlarge the last-level cache, with the figures the sources report (different studies, nodes and conditions; compare orders of magnitude, not exact values)

Approach	Reported figures	Main drawback	Source
Planar SRAM L3	32 MB per Zen 3 CCD; about 47 cycles	Die area, yield and wire length grow with capacity	[13], [32]
Stacked SRAM (3D V-Cache)	64 MB in 41 mm^2; +4 cycles; >2 TB/s	Heat path, clocks, bonding cost	[6], [19]
On-package eDRAM L4 (Intel Broadwell, side by side)	128 MB; over 30 ns; about 50 GB/s	Too slow to join the L3; must be a separate L4	[18]
Stacked DRAM cache (simulated, 2006)	32 MB; average 13% fewer cycles per memory access; off-die traffic −66%; +0.08 °C	Higher latency than SRAM; large tag store	[16], [24]
STT-RAM (simulated, 16 MB)	5.0 against 11.4 mm^2 for SRAM; leakage 65 against 463 mW; write 11.9 against 3.3 ns	Slow, energy-hungry writes	[17]

Denser memory technologies. The two simulated alternatives trade speed for density in different ways. Kim *et al.*'s circuit-level estimates for 16 MB caches show STT-RAM needing less than half the area of SRAM (5.0 against 11.4 mm^2) and about a seventh of the leakage power (65 against 463 mW), with a similar read latency (6.9 against 6.6 ns) but a write latency more than three times longer (11.9 against 3.3 ns) and about six times the write energy (1.13 against 0.18 nJ) [17, Table 1]. That is why they place write-intensive blocks in a small SRAM region, an approach that gave a 26% average speedup over a baseline placement policy in their simulations [17]. Black *et al.*'s 32 MB stacked DRAM cache reduced the cycles per memory access of their benchmarks by 13% on average and up to 55%, cut off-die bandwidth and bus power by 66%, and raised the peak temperature by only 0.08 °C, much less than stacked SRAM, because DRAM dissipates less power [24]. Neither has appeared in a mainstream CPU as an L3 extension; V-Cache's advantage is that it changes the cache's location but not its technology, so it adds capacity at nearly SRAM speed.

28 Interactions Between Chiplet Organisation and Cache Design

The two techniques studied in this paper are not alternatives but layers of one design. Three interactions stand out.

The chiplet makes the stack practical. Stacking is applied to a small, self-contained compute die rather than to a large monolithic processor. Because the CCD is small, a defective stack loses only one CCD's worth of silicon. Because the same CCD is used across the product line, AMD could design V-Cache support into it "from Day 1" and then offer stacked and unstacked versions of the same die [6, slide 10]. And because the stacked CCD is built to the same height as a plain one, server parts with and without V-Cache share a package [6, slide 17].

Stacking moves the cost of the cache to where it is wanted. On a planar die every CCD would pay for a 96 MB L3 whether or not its workload needs it, which is the "product flexibility" barrier on AMD's own list [6, slide 6]. With stacking, the base CCD stays at 81 mm^2 and the 41 mm^2 cache die is added only to the products that benefit.

The chiplet boundary also bounds the cache. The L3 remains private to each CCD (section 5), so a thread gains only from the stacked cache on its own die, and data shared across CCDs still crosses the die-to-die link. A larger per-CCD L3 reduces trips to memory through the IOD, which partly offsets the chiplet's main latency cost (section 8), but it does nothing for traffic between CCDs.

Mixing stacked and plain CCDs creates a scheduling problem. Because stacking is per CCD, a product can combine a V-Cache CCD with a plain one, as the 16-core Ryzen 9 7950X3D does. The two CCDs then have different strengths: one has three times the L3, the other clocks about 7% higher, and neither is better for every program. At the time of Lam's testing, AMD's default policy placed ordinary applications on the higher-clocked CCD, so a user had to set processor affinity by hand to use the cache [18]. The chiplet organisation thus turns a hardware choice into an operating-system decision about where each thread should run.

29 Workload-Dependent Trade-Offs and Remaining Bottlenecks

Taken together, Parts III–V suggest a simple map of where each technique helps. Throughput workloads with many independent threads favour more cores, which chiplets supply. Workloads whose working sets lie just beyond a CCD's planar L3 favour stacked cache. Latency-sensitive single-threaded code with a small working set favours clock frequency, which a stacked part may sacrifice (section 24).

Figure 29 turns this map into a sequence of questions for the stacked cache.

Worked example: cache against clock on one chip

The Ryzen 9 7950X3D makes the trade-off concrete, because its two CCDs differ only in the stacked cache and in clock speed: the plain CCD runs about 7% faster [18]. If performance scaled linearly with clock, the V-Cache CCD would win whenever its IPC gain at equal clock exceeded 7%. Table 27 applies this rule to Lam's fixed-clock measurements.

The linear rule overstates the plain CCD's advantage, because memory-bound programs do not speed up in proportion to clock; Lam makes this point for libx264, expecting the two CCDs to come close [18]. The table's net values are therefore conservative for the V-Cache CCD. Even so, the result shows why AMD placed ordinary applications on the higher-clocked CCD by default, and why the choice still depends on the program [18].

Three bottlenecks remain after both techniques are applied: memory bandwidth per core, which falls as CCDs are added behind a fixed IOD; the die-to-die hop for every L3 miss and for sharing between CCDs; and heat removal from stacked dies. The alternative technologies of section 27 attack the capacity problem from another direction, at a cost in speed; 3D V-Cache instead keeps

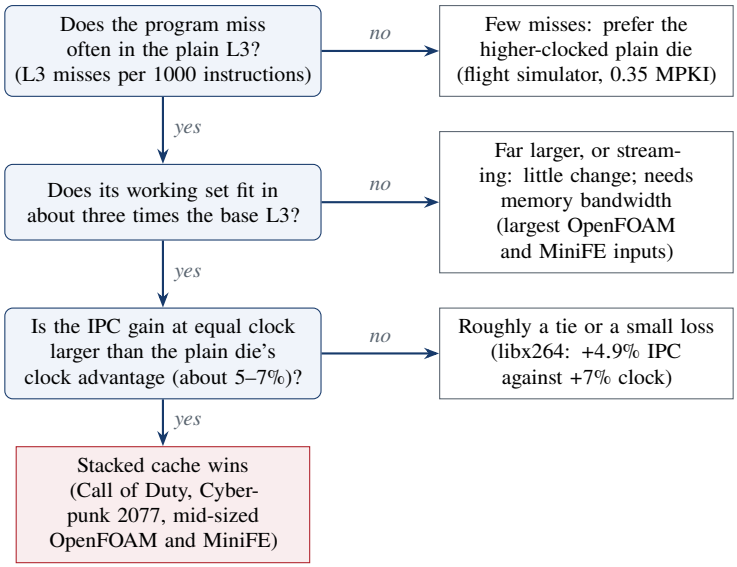


Figure 29: When does a stacked L3 pay off? A decision sequence summarising the evidence of Parts IV and V; the examples are measured cases from [18], [26], [27]. Cost, which also favours the plain die, is not shown.

Table 27: Worked example: IPC gain of the V-Cache CCD at equal clock [18] against the plain CCD's 7% clock advantage (net = $(1 + \text{gain})/1.07 - 1$)

Workload	IPC gain at 4.2 GHz	Net, with plain CCD 7% faster	Better CCD
libx264	+4.9%	−2.0%	plain
GHPC	+9.7%	+2.5%	V-Cache
7-Zip	+9.8%	+2.6%	V-Cache
Cyberpunk 2077	+13.5%	+6.1%	V-Cache
Call of Duty	> +19%	> +11%	V-Cache

SRAM and changes only its placement, which is why it could ship in a mainstream product with the latency cost held to four cycles. The independent evidence of Part V confirms that these alternatives differ chiefly in which workloads they serve, not in a single ranking: stacked SRAM gives the lowest-latency capacity, but only up to 96 MB per CCD in current products.

30 Future Directions in Heterogeneous Integration and Advanced Packaging

AMD's Hot Chips 33 presentation sketches integration schemes that go beyond stacking cache on cores, including memory stacked on compute, cores stacked on cores or on uncore logic, and splitting or "folding" circuits across dies; AMD presents these as schemes that advanced packaging could enable but monolithic design could not [5]. These are vendor projections, not products.

AMD's more recent statements point the same way. Its 2023 EPYC white paper asks whether the same bonding technology could add "specialized processing units to the CPU die" rather than only cache [9], and a 2025 AMD packaging talk lists, among future directions, high-speed standardised die-to-die interfaces (UCIe), co-packaged optics and module sizes that grow beyond what a single wafer-scale process provides, with 3D interconnect pitch below 9 µm and 2.5D pitch below 40 µm [22, slides 12–13]. An independent review of chiplet research reaches a compatible conclusion from the other side: substantial progress has been made, especially in high-performance computing, but chiplet systems are still far from delivering cost, performance, power and design agility together, and package, architecture and interconnect must be co-optimised by chip designers, package designers and design-tool vendors jointly [33].

These directions raise architectural questions that the first-generation evidence already hints at. Stacking logic on logic would concentrate far more power than stacking SRAM, which was itself thermally significant (section 16.2). Black *et al.* simulated exactly this: splitting a high-performance core's floorplan across two stacked dies removed enough wire to cut power by 15% and raise performance by 15%, but the power density rose by 1.3× and the peak temperature by 14 °C, or 26 °C if there were no power savings and the stacking doubled the power density; scaling voltage and frequency down to reach the original temperature still left an 8% performance gain with 34% less power [24]. The thermal cost is roughly three times that of their stacked SRAM cache (4.5 °C), which suggests why commercial stacking has started with cache. Standard interfaces would allow chiplets from different designers to be combined, but each added die-to-die hop adds latency of the kind measured in section 8. And as packages grow, testing each die before assembly becomes more important for yield (appendix C).

31 Limitations of This Study

- The quantitative analysis relies on published measurements and is limited by their experimental conditions and by the availability of comparable data.
- Much of the detailed technical information about AMD designs comes from AMD itself; such claims are attributed to AMD and treated as engineering claims rather than independent results.

- The study works at the architectural level and does not model physical design, packaging processes or thermal behaviour in detail.
- The best-controlled independent evidence for the cache effect (fixed-clock, per-CCD performance counters) covers only a handful of workloads on one processor; most other comparisons change clock speed as well.
- Several independent sources are technical web publications rather than peer-reviewed papers; they are used for measurements whose method they state, and archived locally, but they are not refereed.
- The thermal argument rests on a 2006 simulation study, measured clock and voltage limits of shipping parts, and AMD's description of its own redesign; no published thermal measurement of a hybrid-bonded stack was available.
- The cost model uses 1964 and 2009 yield and cost formulations with illustrative parameters; no public cost model for hybrid-bonded cache stacking was found.

32 Conclusions

RQ1: Why is a single large die an increasingly costly way to scale? Because yield falls faster than linearly with area (section 6.2), the reticle limits the largest die (section 6), and analog, I/O and now SRAM circuits gain little from new process nodes (sections 6.3 and 11). Each of these makes the last square millimetres of a large die the most expensive.

RQ2: What does partitioning into compute and I/O dies change? AMD places the cores, their private L2 caches and the shared L3 on the CCDs, made on the newest node, and the memory controllers, I/O and the fabric that links the CCDs on the IOD, made on an older node; it connects them through the organic substrate because that reach lets eight CCDs surround one IOD, keeping 2.5D and 3D connections for places where density matters more than reach (RQ2a, sections 7 and 8.3). The split makes high core counts practical, improves silicon yield substantially and moves I/O to a cheaper node, at the cost of a die-to-die hop on every L3 miss and on all communication between CCDs, per-link bandwidth limits and NUMA behaviour that software may need to manage (Part III).

RQ3: Why does LLC capacity matter, and what does stacking change? A larger LLC removes capacity misses, but in the plane of the die it costs area, yield, wire length and power (section 12). Stacking adds 64 MB per CCD for about four cycles of latency, with dense, short connections and no growth in package footprint; its price is a harder heat path, which AMD first managed with structural silicon and lower clocks and then addressed by inverting the stack (section 16.2).

RQ4: Where does the stacked cache help, and how can its effect be isolated? It helps workloads whose working sets fall between the base and stacked capacities, often substantially, and does little or slightly harms others. Its effect can be isolated only by fixed-clock comparisons or performance-counter measurements, and the best such evidence supports the mechanism strongly but covers few workloads (section 24).

Taken together, the two techniques address different limits. Chiplet organisation is a general answer to the cost and size limits of monolithic scaling. Stacked cache is a targeted answer to the capacity limit of the last-level cache, whose value depends on the workload. AMD Zen

shows that they work best together: the small, reusable compute die makes stacking practical, and stacking lets that die serve both cache-hungry and clock-hungry markets.

33 Open Research Questions

- When is the extra complexity of 3D integration worth its reduction in footprint and its denser interconnect, compared with a side-by-side cache chiplet built with today's 2.5D packaging rather than the older links of Broadwell's eDRAM?
- How should operating systems and runtimes place threads and data when L3 capacity differs between compute dies?
- What limits the stacking of logic on logic, as opposed to SRAM on logic, from a thermal and power-delivery point of view?
- What is the stacked-cache benefit across a broad, representative workload set when clock speed and core count are held constant?
- How much of the first-generation clock penalty was thermal and how much electrical, and how fully does the inverted stack remove it?
- What would a current, public cost model for hybrid-bonded cache stacking show about where it pays off?

References

[1] S. R. Sarangi, *Basic Computer Architecture*, Version 3.09. Author's online edition, 2025.

[2] S. Naffziger, K. Lepak, M. Paraschou, and M. Subramony, "AMD chiplet architecture for high-performance server and desktop products," in *2020 IEEE International Solid-State Circuits Conference (ISSCC)*, IEEE, 2020, pp. 44–45. DOI: 10.1109/ISSCC19947.2020.9063103

[3] D. Suggs, D. Bouvier, M. Clark, K. Lepak, and M. Subramony, "AMD Zen 2," in *2019 IEEE Hot Chips 31 Symposium (HCS)*, IEEE, 2019.

[4] Advanced Micro Devices, Inc. "AMD Launches AMD Ryzen 5000 Series Desktop Processors: The Fastest Gaming CPUs in the World," AMD Investor Relations (press release). [Online]. Available: https://ir.amd.com/news-events/press-releases/detail/972/amd-launches-amd-ryzen-5000-series-desktop-processors-the-fastest-gaming-cpus-in-the-world

[5] R. Swaminathan, "Advanced Packaging: Enabling Moore's Law's Next Frontier Through Heterogeneous Integration," in *2021 IEEE Hot Chips 33 Symposium (HCS)*, Virtual: IEEE, 2021. [Online]. Available: https://hc33.hotchips.org/assets/program/tutorials/2021%20Hot%20Chips%20AMD%20Advanced%20Packaging%20Swaminathan%20Final%20%2020210820.pdf

[6] J. Wuu et al., "3D V-Cache: The implementation of a hybrid-bonded 64MB stacked cache for a 7nm x86-64 CPU," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, IEEE, 2022, pp. 428–429. DOI: 10.1109/ISSCC42614.2022.9731565

[7] M. Larabel. "AMD EPYC 7773X "Milan-X" Benchmarks Show Very Strong HPC Performance Upgrade," Phoronix. [Online]. Available: https://www.phoronix.com/review/amd-epyc-7773x-linux

[8] A. Alam, T. Chen, and R. Swaminathan, "2nd Generation 3D V-Cache Enablement," presented at the IMAPS 20th Device Packaging Conference (Fountain Hills, AZ), Mar. 2024. [Online]. Available: https://imapsource.org/api/v1/articles/129075-2nd-generation-3d-v-cache-tm-enablement.pdf

[9] Advanced Micro Devices, Inc., "4th Gen AMD EPYC Processor Architecture," Advanced Micro Devices, Inc., White paper, second edition, Jun. 2023.

[10] Advanced Micro Devices, Inc. "The Gaming Legend Continues — AMD Introduces Next-Generation AMD Ryzen 7 9800X3D Processor," AMD Investor Relations (press release). [Online]. Available: https://ir.amd.com/news-events/press-releases/detail/1225/the-gaming-legend-continues-amd-introduces-next-generation-amd-ryzen-7-9800x3d-processor

[11] Advanced Micro Devices, Inc., "AMD EPYC 7003 processor microarchitecture overview," Advanced Micro Devices, Inc., Publication 57075, 2022. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/epyc-technical-docs/white-papers/overview-amd-epyc7003-series-processors-microarchitecture.pdf

[12] B. Justice. "AMD Ryzen 7 5800X vs Ryzen 7 3700X Performance Review," The FPS Review. [Online]. Available: https://www.thefpsreview.com/2021/10/06/amd-ryzen-7-5800x-vs-ryzen-7-3700x-performance-review/

[13] 7-cpu.com. "AMD Zen3," 7-cpu.com. [Online]. Available: https://www.7-cpu.com/cpu/Zen3.html

[14] Advanced Micro Devices, Inc., "Using SEV with AMD EPYC Processors," Advanced Micro Devices, Inc., User guide 58207, Rev. 1.0, Mar. 2023. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/developer/58207-using-sev-with-amd-epyc-processors.pdf

[15] B. Wang, "Design and Implementation of Cache Memory with Dual Unit Tile/Line Accessibility," *Mathematical Problems in Engineering*, vol. 2019, no. 1, Jan. 2019, Art. no. 9601961. DOI: 10.1155/2019/9601961

[16] S. Mittal and J. S. Vetter, "A Survey Of Techniques for Architecting DRAM Caches," *IEEE Transactions on Parallel and Distributed Systems*, vol. 27, no. 6, pp. 1852–1863, Jun. 1, 2016. DOI: 10.1109/TPDS.2015.2461155

[17] B. Kim, Y. Kim, P. Nair, and S. Hong, "Exploiting Data Compression for Adaptive Block Placement in Hybrid Caches," *Electronics*, vol. 11, no. 2, Jan. 12, 2022, Art. no. 240. DOI: 10.3390/electronics11020240

[18] C. Lam. "AMD's 7950X3D: Zen 4 Gets VCache," Chips and Cheese. [Online]. Available: https://chipsandcheese.com/p/amds-7950x3d-zen-4-gets-vcache

[19] G. Cozma. "Deep Diving Zen 3 V-Cache," Chips and Cheese. [Online]. Available: https://chipsandcheese.com/p/deep-diving-zen-3-v-cache

[20] X. Dong and Y. Xie, "System-level cost analysis and design exploration for three-dimensional integrated circuits (3D ICs)," in *2009 Asia and South Pacific Design Automation Conference*, Yokohama, Japan: IEEE, Jan. 2009, pp. 234–241. DOI: 10.1109/ASPDAC.2009.4796486

[21] B. Murphy, "Cost-size optima of monolithic integrated circuits," *Proceedings of the IEEE*, vol. 52, no. 12, pp. 1537–1545, 1964. DOI: 10.1109/PROC.1964.3442

[22] W. Lambert, "Heterogenous Integration in High Volume: Now and Looking Forward," presented at the IMAPS 2025 Device Packaging Conference (Phoenix, AZ), 2025. [Online]. Available: https://imapsource.org/article/147811.pdf

[23] G. Cozma. "AMD's 9800X3D: 2nd Generation V-Cache," Chips and Cheese. [Online]. Available: https://chipsandcheese.com/p/amds-9800x3d-2nd-generation-v-cache

[24] B. Black et al., "Die Stacking (3D) Microarchitecture," in *2006 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'06)*, Orlando, FL: IEEE, Dec. 2006, pp. 469–479. DOI: 10.1109/MICRO.2006.18

[25] Advanced Micro Devices, Inc., "AMD EPYC 9004 Series Processors," Advanced Micro Devices, Inc., Data sheet, 2023. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/datasheets/amd-epyc-9004-series-processors-datasheet.pdf

[26] J. Domke et al., "At the Locus of Performance: Quantifying the Effects of Copious 3D-Stacked Cache on HPC Workloads," *ACM Transactions on Architecture and Code Optimization*, vol. 20, no. 4, pp. 1–26, Dec. 31, 2023. DOI: 10.1145/3629520

[27] M. Lawenda, Ł. Szustak, L. Környei, F. C. C. Galeazzo, and P. Bratek. "Evaluating the impact of the L3 cache size of AMD EPYC CPUs on the performance of CFD applications." arXiv: 2505.17934, pre-published.

[28] M. Larabel. "AMD Ryzen 7 5800X vs. Ryzen 7 5800X3D On Linux 6.0 Benchmarks," Phoronix. [Online]. Available: https://www.phoronix.com/review/amd-5800x3d-linux6

[29] Advanced Micro Devices, Inc., "AMD Ryzen 3000 and 5000 Series Processors," Advanced Micro Devices, Inc., Datasheet, 2022. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/products/processors/ryzen/5000/amd-ryzen-5000-datasheet.pdf

[30] S. Walton. "AMD Ryzen 7 5800X Review," TechSpot. [Online]. Available: https://www.techspot.com/review/2134-amd-ryzen-5800x/

[31] Advanced Micro Devices, Inc. "AMD Ryzen 7 9800X3D Desktop Processor," AMD (product specifications). [Online]. Available: https://www.amd.com/en/products/processors/desktops/ryzen/9000-series/amd-ryzen-7-9800x3d.html

[32] G. Cozma. "AMD's V-Cache Tested: The Latency Teaser," Chips and Cheese. [Online]. Available: https://chipsandcheese.com/p/amds-v-cache-tested-the-latency-teaser

[33] Y. Liu, X. Li, and S. Yin, "Review of chiplet-based design: System architecture and interconnection," *Science China Information Sciences*, vol. 67, no. 10, p. 200 401, Oct. 2024. DOI: 10.1007/s11432-023-3926-8

A Plagiarism Report

[TODO: No plagiarism-check evidence has been supplied for this draft. Each version needs its own evidence; Draft 1's report does not apply. Add the tool's displayed result for this draft and describe it only as what the tool displayed.]

B Reading Indices

[TODO: No readability indices have been supplied for this draft. Add the values exactly as displayed by the analysis tool. Readability indices estimate linguistic complexity only; they do not measure technical correctness or the strength of the argument.**]**

C Yield and Cost-per-Good-Die Model

This appendix derives the illustrative cost model used in section 6.2. It is a textbook-style model for reasoning about trends, not a model of any real AMD product or foundry process. The yield expressions follow Murphy [21] and Dong and Xie [20]; the numerical examples are this paper's own.

C.1 From Poisson to Murphy yield

If yield-killing defects fall independently and uniformly with a *fixed* density D per unit area, the number of defects on a die of area A is Poisson-distributed with mean AD. A die is good only if it has no defects, so its yield is

$$Y_{\text{Poisson}}(A, D) = e^{-AD}.$$

In practice the defect density is not fixed: it varies from chip to chip and from slice to slice. Murphy therefore averaged the Poisson yield over a normalised distribution $f(D)$ of defect densities, $Y = \int_0^\infty e^{-DA} f(D)\, dD$, and solved it for three simplified distributions [21, Eqs. 7–11]. If D is spread uniformly (a "rectangular" distribution) between 0 and $2D_0$, so that its mean is D_0, then

$$Y_{\text{Murphy}}(A) = \frac{1}{2D_0} \int_0^{2D_0} e^{-AD}\, dD = \frac{1 - e^{-2AD_0}}{2AD_0}.$$

This is Murphy's Eq. 11, and multiplying it by the fraction of wafers that survive processing, Y_{wafer}, gives the expression Dong and Xie use (their Eq. 15), which is equation (8).

Murphy also considered a narrower, "triangular" distribution, which gives

$$Y_{\text{triangular}}(A) = \left(\frac{1 - e^{-AD_0}}{AD_0} \right)^2.$$

He regarded the zero-width (Poisson) and rectangular cases as extremes that are "almost certainly inappropriate" and used the triangular form for most of his results, while noting that the three forms differ little at high yields [21, p. 1539]. This paper keeps the rectangular form because it is the one used in the 3D-IC cost model of Dong and Xie; for the 1 and 2 cm^2 dies of the worked example the choice changes the result by less than one percentage point (82.4 against 82.1%, 68.8 against 67.9%), but for the 592 mm^2 monolithic die used in Parts I and III the rectangular and triangular forms give 38.3% and 34.4%, a gap of about four points that slightly favours the monolithic design. Because the averaged expression is never smaller than e^{-AD_0} for the same mean density, Murphy's model is less pessimistic than the Poisson model for large dies.

With $D_0 = 0.2\,\text{cm}^{-2}$ and $Y_{\text{wafer}} = 1$: for $A = 1\,\text{cm}^2$, $Y = (1 - e^{-0.4})/0.4 \approx 0.824$; for $A = 2\,\text{cm}^2$, $Y = (1 - e^{-0.8})/0.8 \approx 0.688$. For comparison, the triangular model gives 0.821 and 0.679, and the Poisson model 0.819 and 0.670.

C.2 Cost per good die

Let C_w be the cost of a processed wafer and $N(A)$ the number of whole dies of area A that fit on it. Ignoring edge losses, $N(A) \approx W/A$ for usable wafer area W. Dong and Xie use a more exact

dies-per-wafer expression that also counts the partial dies lost at the wafer edge [20]. The cost per good die is

$$C_{\text{good}}(A) = \frac{C_w}{N(A)\,Y(A)} \approx \frac{C_w}{W} \cdot \frac{A}{Y(A)}.$$

Doubling the area therefore multiplies the cost per good die by approximately $2\,Y(A)/Y(2A)$. In the example above (from 1 to $2\,\text{cm}^2$ with $D_0 = 0.2\,\text{cm}^{-2}$), the factor is $2 \times 0.824/0.688 \approx 2.40$: the larger die costs about 2.4 times as much, not twice as much.

C.3 Splitting a design into chiplets

Suppose a design of area A is split into k chiplets of area A/k, each with an additional interface overhead δ. If each chiplet is tested before assembly, so that only known-good dies are packaged, the silicon cost of the assembled part is approximately

$$k \cdot C_{\text{good}}\!\left(\tfrac{A}{k} + \delta\right) \approx \frac{C_w}{W} \cdot \frac{A + k\delta}{Y\!\left(\tfrac{A}{k} + \delta\right)}.$$

Smaller chiplets yield better, so the denominator grows with k, while the overhead term $k\delta$ in the numerator also grows. As an illustration with $D_0 = 0.2\,\text{cm}^{-2}$, a $4\,\text{cm}^2$ design split into four chiplets of $1\,\text{cm}^2$ plus a $0.05\,\text{cm}^2$ interface each costs about $(4.2/0.817)/(4/0.499) \approx 0.64$ of the monolithic silicon, a saving of roughly 36%. The saving grows for larger designs and higher defect densities. Dong and Xie reach the same qualitative conclusion for die stacking: it becomes cheaper than a single planar die only above a size threshold, and testing each die before stacking (die-to-wafer bonding) avoids multiplying together the yields of all dies in the stack [20]. The model omits packaging, assembly yield, test cost and the use of different process nodes for different chiplets, all of which section 9 discusses qualitatively.

Index

2.5D integration, 43

AMAT, 26

cache miss, 28

dynamic power, 24

ED^2 metric, 25

hybrid bonding, 52

Infinity Fabric, 40

leakage power, 24

memory hierarchy, 26
Moore's law, 16
Murphy yield model, 36

performance equation, 22
plagiarism report, 86

readability indices, 87
reticle limit, 36

SRAM, 26

thermal resistance, 25

yield, 36, 88